Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Kevin Kudakwashe Murera

Thursday 13th August, 2026

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Kevin Kudakwashe Murera

Day of issue	:	TODO: date of issue
Day of release	:	TODO: date of submission

Reviewer	:	Prof. Dr. Karsten Berns
Supervisor	:	TODO: supervisor

Hereby I declare that I have self-dependently composed the Master Thesis at hand. The sources and additives used have been marked in the text and are exhaustively given in the bibliography.

Thursday 13$^{\text{th}}$ August, 2026 – Kaiserslautern

(Kevin Kudakwashe Murera)

Acknowledgements

TODO: acknowledgements.

Abstract

TODO: abstract. Hydraulic manipulators are hard to control precisely because the actuator response is nonlinear, load dependent, and uncertain. This thesis asks how the learned representation of that actuator affects closed-loop task performance, and whether conditioning that representation online on an inferred estimate of the currently active plant improves on a fixed learned model. Three control stacks are compared under matched task, observation, action, terrain, reward, seed and budget conditions, differing only in the actuation and adaptation path: PPO–ROM, in which the policy's desired cylinder motion is realized by a command-conditioned hydraulic reduced-order model; PPO–ID–FD, in which it is realized by a frozen inverse model followed by a frozen strict forward model; and PPO–ID(AAM)–FD, in which an Arm Adaptation Module infers a dynamics latent from causal observable history and conditions a small bounded adapter around the frozen inverse model.

Contents

1 Introduction **1**

1.1 Motivation and Application Context 1
1.2 Problem Statement 1
1.3 Research Questions 1
1.4 Objectives 1
1.5 Contributions 1
1.6 Scope and Limitations 1
1.7 Thesis Structure 1

2 Fundamentals and Related Work **3**

2.1 Manipulator Actuation and Control Fundamentals 3
2.1.1 Application Context and Implement Carriers 3
2.1.2 The Electro-Hydraulic Actuation Chain 4
2.1.3 High-Fidelity Simulation as a Data Source 5
2.2 Kinematics and Task-Space Tracking 5
2.2.1 Differential Kinematics 5
2.2.2 From Joint Space to Cylinder Space 6
2.3 Model-Based and Conventional Control Approaches 6
2.4 Data-Driven System Identification and Learned Dynamics 7
2.4.1 Learned Forward and Inverse Dynamics 7
2.4.2 Closed-Loop Identifiability 7
2.5 Reduced-Order Modeling 8
2.6 Domain Randomization and Latent Adaptation 9
2.6.1 Robustness Through Randomized Training 9
2.6.2 From Robustness to Adaptation 9
2.7 Reinforcement Learning and Structured Priors 11
2.7.1 Policy Optimization 11
2.7.2 Structured Priors for Policy Learning 11
2.7.3 Reinforcement Learning on Hydraulic Machines 12
2.8 Research Gap and Thesis Positioning 13

3 System Fundamentals and Problem Formulation 15
3.1 Manipulator Platform . . . 15
3.2 Kinematic Structure and Coordinate Mappings . . . 16
3.2.1 Joint Space and Cylinder Space . . . 16
3.2.2 Task Space and the Calibrated Cutter Jacobian . . . 18
3.3 Actuation and Measured State . . . 18
3.4 High-Fidelity Simscape Model . . . 19
3.4.1 The Command ROM as Deployment Actuator . . . 19
3.4.2 Mechanical Subsystem . . . 20
3.4.3 Hydraulic Subsystem . . . 21
3.4.4 Validated Scope of the Model . . . 23
3.5 IsaacLab Environment . . . 23
3.5.1 Simulation and Control Rates . . . 24
3.5.2 Environment Configuration Details . . . 24
3.6 The Verge-Following Task . . . 25
3.6.1 Task Definition . . . 25
3.6.2 Task Feasibility and Plant Capability . . . 25
3.6.3 Parked Diagnostic Workflow . . . 26
3.7 Control Architecture . . . 26
3.8 Problem Formulation . . . 26
3.9 Evaluation Criteria at System Level . . . 28
3.9.1 The Eight-Gate Task Contract . . . 28
3.9.2 Cylinder- and Model-Level Criteria . . . 29

4 Data Generation and Preparation 31
4.1 Dataset Objectives and Design Requirements . . . 32
4.2 Randomized Parameter Space of the Simscape Campaign . . . 32
4.2.1 Sampling Design . . . 33
4.2.2 Hydraulic Parameters . . . 34
4.2.3 The Compressibility Axis . . . 34
4.2.4 Mechanical Parameters . . . 34
4.3 Closed-Loop Data Collection . . . 34
4.3.1 The Controller as a Data-Collection Instrument . . . 34
4.4 Open-Loop Excitation Experiments . . . 35
4.4.1 Limitations of the Closed-Loop Batch . . . 35
4.4.2 Design of the Excitation Signal . . . 36
4.4.3 Plant Draws and Acceptance . . . 37
4.5 Combining Closed-Loop and Open-Loop Datasets . . . 37
4.6 Closed-Loop Calibration Data for the Command ROM . . . 39
4.7 The IsaacLab Adaptation Collection . . . 40
4.7.1 Provenance of the Adaptation Data . . . 40
4.7.2 The Physical Factor Vector . . . 40
4.7.3 Domains and Factor Design . . . 40
4.7.4 Collection Procedure . . . 41
4.7.5 The Frozen Dataset . . . 41
4.7.6 Learning Timebase and Hydraulic State . . . 41
4.8 Feature and Target Construction . . . 42
4.8.1 The Actuator Models . . . 42
4.8.2 Observable History of the Adaptation Models . . . 42
4.9 Preprocessing and Data Splits . . . 43
4.10 Dataset Diagnostics . . . 44

5 Learned Models and Online Adaptation 45
5.1 Modeling Overview and Common Notation . . . 45
5.1.1 Cylinder-Space Signals . . . 45
5.1.2 Two Actuator Representations . . . 46
5.1.3 Common Conventions . . . 46
5.1.4 Data Provenance . . . 48
5.2 Inverse Dynamics Model . . . 48
5.2.1 Input–Output Contract . . . 48
5.2.2 Architecture and Supervised Training . . . 49
5.2.3 Paired Control Fine-Tuning . . . 49
5.3 Forward Dynamics Model . . . 51
5.3.1 Input–Output Contract and Supervised Baseline . . . 51
5.3.2 Assessment of the Supervised Score . . . 51
5.3.3 Strict Open-Loop Excitation Dataset . . . 51
5.3.4 Strict Retraining Objective . . . 53
5.3.5 Validation of Valve Authority . . . 53
5.4 Reduced-Order Model . . . 54
5.4.1 Motivation and Design Position . . . 54
5.4.2 Input–Output Contract . . . 55
5.4.3 Grey-Box Structure . . . 55
5.4.4 Dataset and Training . . . 57
5.4.5 Offline Validation . . . 58
5.4.6 Closed-Loop Calibration Against the Deployment Plant . . . 59
5.5 System Dynamics Model . . . 59
5.6 Privileged Latent Representation . . . 60
5.7 Arm Adaptation Module . . . 61
5.8 Validation of the Adaptation Chain . . . 61
5.9 Latent-Conditioned Inverse-Model Adapter . . . 62
5.10 Training and Model-Selection Protocol . . . 63
5.11 Offline Model Evaluation . . . 63

6 Control Framework and Simulation Integration 65
6.1 The DLS Teacher and Its Two Roles . . . 65
6.1.1 Scope and Role of the Controller . . . 65
6.1.2 Privileged Information . . . 65
6.1.3 From Task Error to Cylinder Command . . . 67
6.1.4 Execution Through the Student Interface . . . 67
6.1.5 Teacher Performance . . . 68
6.2 Policy Formulation . . . 68
6.2.1 Policy Definition . . . 68
6.2.2 The Observation Vector . . . 69
6.2.3 The Action and Its Bounds . . . 70
6.2.4 The Reward Function . . . 71
6.2.5 Optimization . . . 73
6.3 Training Protocol and the Distillation Finding . . . 74

6.3.1 Distillation by DAgger . . . 74
6.3.2 Objective Mismatch in Unconstrained Fine-Tuning . . . 75
6.4 Reference-Anchored PPO . . . 77
6.4.1 Critic Warm-Up . . . 77
6.4.2 The Reference Anchor . . . 77
6.4.3 Trust Regions and Rollback . . . 78
6.4.4 The Block Ratchet . . . 78
6.4.5 Theoretical Status . . . 80
6.5 PPO–ROM: Certified Actuator Baseline . . . 80
6.6 PPO–ID–FD: Non-Adaptive Learned Baseline . . . 81
6.7 PPO–ID(AAM)–FD: Proposed Adaptive Method . . . 81
6.8 IsaacLab Integration and Timing . . . 81
6.9 Stability, Constraints and Fair-Comparison Rules . . . 81

7 Experimental Evaluation and Discussion 83
7.1 Hypotheses and Fairness Controls . . . 83
7.2 Model-Level Experiments . . . 83
7.3 System Response . . . 83
7.4 Delay . . . 83
7.5 Nonlinearity . . . 83
7.6 End-Effector and Mowing Performance . . . 83
7.7 Robustness and Generalization . . . 83
7.8 Adaptation Ablations . . . 84
7.9 Statistics and Reporting . . . 84
7.10 Integrated Discussion . . . 84
7.11 Threats to Validity . . . 84

8 Conclusion and Outlook 85
8.1 Summary of the Approach . . . 85
8.2 Answers to the Research Questions . . . 85
8.3 Main Contributions . . . 85
8.4 Limitations . . . 85
8.5 Future Work . . . 85

A System Parameters and Coordinate Mappings 87

B Dataset Manifests and Factor Ranges 89
B.1 Trajectories . . . 89
B.2 Execution and Logging . . . 89
B.3 Parameter Ranges and Dataset Audit . . . 90

C Model Architectures, Normalizers and Checkpoint Hashes 93

D PPO and IsaacLab Configuration 95

E Supplementary Results 97

Bibliography 99

1. Introduction

1.1 Motivation and Application Context

[TODO: write motivation]

1.2 Problem Statement

[TODO: write problem statement]

1.3 Research Questions

[TODO: state research questions RQ1–RQ5]

1.4 Objectives

[TODO: state objectives]

1.5 Contributions

[TODO: list contributions]

1.6 Scope and Limitations

[TODO: define scope and limitations]

1.7 Thesis Structure

[TODO: summarize chapters 2–8]

2. Fundamentals and Related Work

Chapter 1 argued that precise task-space tracking with a hydraulic mowing arm is limited less by kinematics than by the actuator. The path from a valve command to actual cylinder motion is nonlinear and load dependent, and it differs from one machine, season, or attachment to the next. This chapter collects the background needed to judge that argument and to understand the design decisions made later in the thesis. Section 2.1 introduces the machine class and its electro-hydraulic actuation. Section 2.2 recalls the kinematic tools behind the classical controller, and Section 2.3 reviews how conventional model-based control deals with hydraulic uncertainty. The remaining sections cover the learning methods this work builds on. Section 2.4 treats data-driven dynamics models and the pitfalls of identifying them from closed-loop data. Section 2.5 covers reduced-order and grey-box modeling, Section 2.6 domain randomization and latent adaptation, and Section 2.7 policy optimization with structured priors. Section 2.8 then states the gap that remains and how the three control stacks of this thesis (PPO–ROM, PPO–ID–FD, and PPO–ID(AAM)–FD) address it. Only material that is needed for the later chapters is included. The concrete system definition and problem formulation follow in Chapter 3.

2.1 Manipulator Actuation and Control Fundamentals

2.1.1 Application Context and Implement Carriers

The machine studied in this thesis belongs to a class that is easy to overlook because it is so common: hydraulic booms mounted on road-going implement carriers, doing repetitive maintenance work along traffic infrastructure. Section 3.1 shows the concrete instance, the MULAG FME 500 front mower on a Unimog carrier [MULAG Fahrzeugwerk, Heinz Wössner GmbH u. C
A multi-link boom carries a 1.2 m flail mowing head to the roadside verge, reaches up to about 6.5 m to either side of the vehicle, and must hold the head in a shallow band above the ground while the vehicle creeps along the road. The manufacturer's working envelope, reproduced in Figure 3.1, makes the difficulty concrete. The same arm that mows a flat verge beside the wheels must also reach over guardrails, down embankments to about

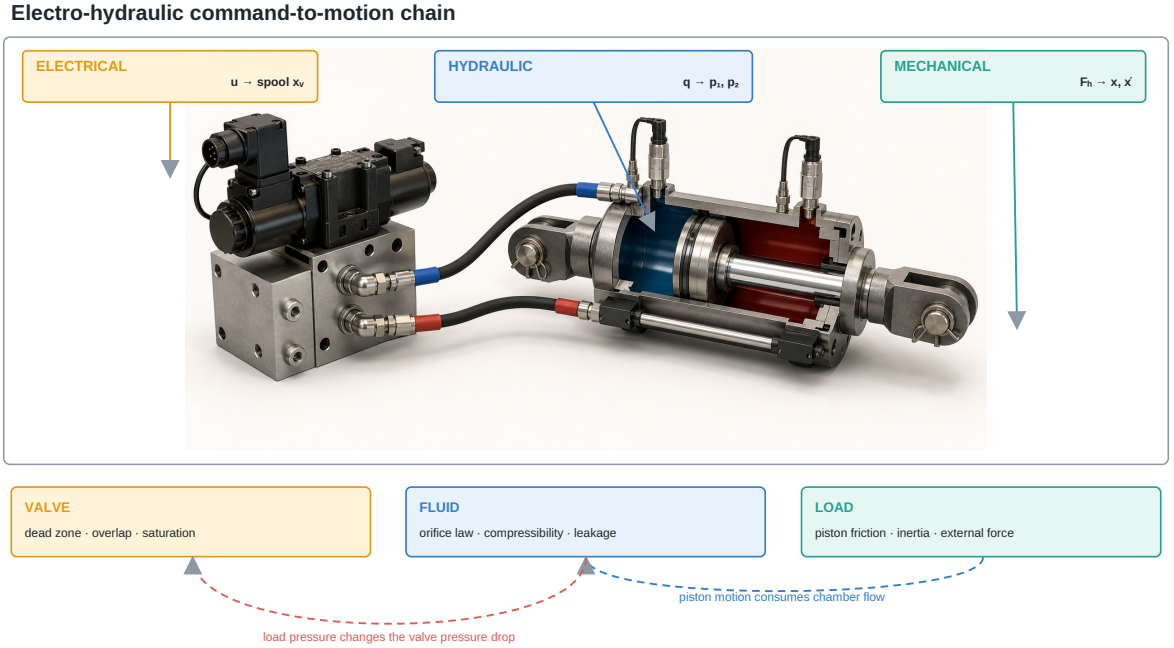

Figure 2.1: The electro-hydraulic actuation chain from valve command to mowing-head motion, grouped by energy domain. Tags beneath each domain name its dominant nonlinearities. The dashed return paths are the feedback loops that make the chain gain an operating point rather than a constant.

3 m below road level, and up to 8 m for hedge work, with the same linkage and the same operator. In practice, head height, tilt, and lateral position are adjusted continuously by a human through open-loop joystick commands. It is skilled and fatiguing work, done while also driving the vehicle. Automating the tracking part of this task is the practical motivation of this thesis.

Hydraulic actuation dominates this machine class for reasons that remain valid despite the rise of electric drives. Cylinders deliver very high force from small, rugged, and inexpensive components; they tolerate shock loads such as the mowing head striking a curbstone; and the carrier vehicle already provides hydraulic power through its power take-off. The price is paid in controllability. An electric joint drive responds to current within milliseconds and nearly linearly. A hydraulic cylinder responds through a chain of nonlinear energy conversions whose effective gain can change by an order of magnitude across the workspace. Mattila et al. [Mattila 17] survey two decades of research on this tension between the force density of hydraulic manipulators and the difficulty of controlling them precisely.

2.1.2 The Electro-Hydraulic Actuation Chain

Figure 2.1 traces the signal path that every command to the arm must traverse. A normalized electrical command displaces the spool of a proportional valve, which opens metering orifices toward the supply and tank lines. To first approximation the resulting flow follows the orifice law [Merritt 67], in which the flow through the valve is proportional to the spool position and to the square root of the pressure difference across the metering edge, scaled by the discharge coefficient and the orifice area gradient. The form is written

out in Section 3.4 with the coefficient values used by the plant model. Two of its properties shape everything that follows in this thesis. First, the flow gain depends on the square root of the pressure margin. The same spool displacement therefore moves the cylinder quickly under light load and barely at all when the load pressure approaches supply. The effective valve authority is an operating point, not a constant. Second, the relation is directional. Extending against gravity and retracting with it see different pressure margins and hence different gains. Around the spool null, machining overlap produces a dead zone in which small commands move nothing at all, and at the other end of the stroke the spool saturates [Jelali 03].

The delivered flow does not translate directly into motion either. Oil is compressible, the more so when air is entrained, so each cylinder chamber acts as a nonlinear spring. Its stiffness scales with the effective bulk modulus and inversely with the chamber volume, and the volume itself changes with piston position. Pressure relief valves clip the dynamics whenever a load spike or an aggressive command drives a chamber to its limit, which happens routinely in mowing when the head rests on the ground. Internal leakage bleeds pressure between chambers, seal friction opposes motion, and the load pressure feeds back into the valve flow through the orifice law. Taken together, this is the plant that the learned models of this thesis must represent: several interacting nonlinearities, some of them non-smooth (relief, dead zone, stiction), and parameters such as bulk modulus, leakage, and friction that drift with temperature and wear.

One structural point matters for the control architecture. The chain in Figure 2.1 ends in the mechanical linkage, where cylinder translation becomes joint rotation through lever arms that vary with configuration. Uncertainty entering at the valve is therefore filtered, amplified, and coupled by the mechanism before it appears as head-position error. This is why the models of Chapter 5 are formulated in *cylinder space*, directly across the hydraulic conversion stages, rather than in task space where hydraulic and kinematic effects blur together.

2.1.3 High-Fidelity Simulation as a Data Source

The hydraulic subsystems above can be modeled in detail in component-level simulation environments such as Simscape [The MathWorks, Inc. 24], and Chapter 4 builds on such a model of the FME 500 arm. It is tempting to close the loop and train a reinforcement-learning policy directly against this model, but the computational profiles do not match. The stiff pressure dynamics force small integration steps, and the solver processes one machine instance at a time, while modern policy optimization spreads its sample cost over thousands of plants simulated in parallel on a GPU [Makoviychuk 21]. This thesis resolves the mismatch by separation: the high-fidelity model is used offline as a data generator, and compact learned representations of the actuator carry its behavior into the parallel training environment. How this workflow fits together is shown later, in Figure 2.2, once its ingredients have been introduced.

2.2 Kinematics and Task-Space Tracking

2.2.1 Differential Kinematics

The mowing task is specified where the work happens, as a reference path for the cutter in vehicle coordinates. Task-space control therefore rests on the differential kinematics of the

arm [Siciliano 09], in which the manipulator Jacobian maps joint rates to tool velocity and its inverse yields the commands realizing a desired tool motion. The plain pseudo-inverse is fragile near singular configurations, which this arm visits routinely with the boom almost fully folded. The standard remedy, adopted unchanged here, is the damped least-squares inverse of Wampler [Wampler 86] and Nakamura and Hanafusa [Nakamura 86], which trades bounded tracking accuracy for bounded joint commands. It is conservative, well understood and cheap, which are the properties wanted in a layer that must never misbehave.

2.2.2 From Joint Space to Cylinder Space

Two complications specific to a hydraulic boom carry into the rest of the thesis. The actuated coordinates are cylinder lengths rather than joint angles, and each cylinder closes a triangle over its joint, so the cylinder-to-joint map is nonlinear and its lever arm varies over the stroke. The Jacobian used for control is therefore the composition of the task Jacobian with that map, and an error in one cylinder's realized motion appears in the task error of the whole arm. A passive fifth joint coupled to the head further constrains which task directions can be controlled independently. Chapter 3 works out both consequences and states the calibrated cutter Jacobian used throughout.

The kinematic layer, however accurate, assumes that a commanded cylinder velocity is a realized one. Section 2.1.2 argues against that assumption on this actuator class. The kinematic controller is therefore a *nominal* structure whose commands the hydraulic layer may or may not realize, and the learned models of this thesis live in that realization gap.

2.3 Model-Based and Conventional Control Approaches

Industrial practice on machines like the FME 500 is open-loop operator control: the joystick maps to valve current and the human closes every loop. The control literature offers a spectrum of alternatives, none of which alone settles the problem addressed here.

Classical cascade structures, an outer position loop commanding an inner velocity or pressure loop with fixed PI or PD gains, are the workhorse of hydraulic servo control [Jelali 03]. They are simple and robust, but fixed gains must be tuned for the worst case across the operating envelope, and Section 2.1.2 showed that the plant gain varies by an order of magnitude across that envelope. The result is a controller that is sluggish where the plant is weak or oscillatory where it is strong. On a boom whose lever arms change with every meter of reach, one gain set cannot be right everywhere.

Model-based nonlinear methods attack the problem directly. Computed-torque control cancels the rigid-body dynamics if they are known [Slotine 91], backstepping designs extend the idea through the pressure dynamics, adaptive versions estimate uncertain parameters online [Åström 95], and robust versions bound their effect. The line remains active: Yang et al. [Yang 26] pair a state-constrained controller with an explicit inverse of the valve dead-band, holding positioning accuracy while keeping the states inside prescribed bounds. That work is the model-based counterpart of a choice made in Chapter 5, where the same dead-band is represented by a learnable smooth element inside the reduced-order model rather than inverted analytically. Applied to hydraulic manipulators these designs achieve impressive tracking in laboratory settings, but their guarantees are conditioned on

structural model knowledge: valve flow curves, friction models, inertia parameters. That knowledge is expensive to obtain and to maintain for a commercial implement that is produced in variants and worked across seasons. Mattila et al. [Mattila 17] close their survey on the same point: model-based hydraulic control is only as accurate as its models, and the models are the bottleneck.

This thesis draws a specific lesson from that literature rather than discarding it. Structured controllers are kept where structure is reliable. The kinematic DLS layer of Section 2.2 is retained, because differential kinematics *is* accurately known. It serves as the privileged teacher that generates safe excitation and supervises policy distillation, and as the closed-loop controller used during identification data collection. Learning is directed at the part where first-principles modeling is the bottleneck, the mapping between cylinder-space commands and realized cylinder motion, and at the task-level controller that issues those commands (Section 2.7). The architecture is hybrid by design, not black-box.

2.4 Data-Driven System Identification and Learned Dynamics

2.4.1 Learned Forward and Inverse Dynamics

Where deriving a model is the bottleneck, one can measure it instead. In robot model learning [Nguyen-Tuong 11], two complementary mappings are distinguished. A *forward dynamics* model predicts the next state, or state increment, from the current state and the applied command, and an *inverse dynamics* model predicts the command that realizes a desired transition. Both contracts are stated precisely in Section 5.1, where they are fixed for the rest of the thesis. Neural networks are the standard function class when the underlying physics is non-smooth and history dependent, as it is here. For control, one-step accuracy is necessary but not sufficient. A model used inside a planner or a training loop is iterated, feeding its own prediction back as input, so small one-step errors compound over the horizon. A model selected purely by one-step loss can diverge within a few hundred milliseconds of recursive prediction [Nagabandi 18]. Multi-step rollout objectives, which iterate the model during training and penalize the accumulated error, counteract this compounding directly and are used for every dynamic model trained in this thesis. The probabilistic-ensemble line of work [Chua 18] addresses the same problem by additionally tracking how model uncertainty grows along the rollout. Where a single sample cannot summarize the relevant past, for instance pressure transients and valve history, recurrent architectures [Hochreiter 97, Cho 14] extend the forward model with an internal state that integrates a window of history. The reduced-order model of Chapter 5 takes this form.

2.4.2 Closed-Loop Identifiability

The dominant practical danger in learning a forward model is not underfitting but learning the wrong thing well. System-identification theory has long formalized when data determines a plant: informative experiments require inputs that excite the dynamics independently of the feedback law [Ljung 99, Isermann 11]. Data collected in closed loop violates this in a treacherous way. The controller computes the command from the state history, so command and state are strongly correlated, and a flexible model can

reach excellent prediction scores by exploiting the state history alone. It then behaves like a sophisticated persistence model that ignores the input channel. The failure is silent, because validation data from the same closed-loop distribution keeps rewarding the shortcut. Schoukens and Ljung [Schoukens 19] emphasize that for nonlinear systems the design of informative excitation is, if anything, more critical than for linear ones.

This is not a hypothetical concern in this thesis. It is a measured event. The first forward-dynamics model of Chapter 5, trained on closed-loop operation data, reached one-step R^2 near 0.999 while an input ablation showed that it had learned almost no dependence on the valve command at all. The open-loop excitation campaign of Chapter 4, valve sequences applied directly to the plant without any feedback controller in the loop, exists specifically to break this correlation and make the causal valve-to-motion path identifiable. The methodological rule that follows is applied to every learned model in this work: input-ablation tests with zeroed and shuffled commands are part of model acceptance, and a supervised score is never accepted as evidence of control authority on its own.

2.5 Reduced-Order Modeling

The term *reduced-order model* comes from a literature on projecting high-dimensional dynamical systems onto low-dimensional subspaces that preserve their dominant behavior, surveyed with its parametric variants by Benner et al. [Benner 15]. The motivation carries over directly: a model cheap enough to evaluate thousands of times per second, yet faithful enough that conclusions drawn from it transfer. Here the construction is learned rather than projected, reducing the full Simscape hydraulic model to a compact per-cylinder state of extension, extension velocity and pressure difference.

Between white-box projection and black-box regression lies the grey-box middle ground this thesis occupies. Physics-informed learning embeds known structure into the hypothesis class so that data is spent on what is unknown [Karniadakis 21], an approach Lutter et al. [Lutter 19] demonstrated for rigid-body dynamics by building Lagrangian structure into a network. The reduced-order model of Chapter 5 applies this to hydraulics: the orifice law, the pressure-buildup equation and the integrator chain are explicit layers with trainable effective parameters, while neural components supply only the valve realization, the configuration-dependent load and small bounded residuals. Two properties motivated this over a free-form sequence model. The trained parameters can be checked for physical plausibility, and the bounded residuals prevent an off-distribution input from producing an unphysical state jump, which matters in a policy-training loop that pushes its model off-distribution by design.

The closest precedent for embedding such a surrogate in policy training comes from legged robotics. Hwangbo et al. [Hwangbo 19] identified the rigid-body mechanics of their robot analytically, learned the response of its series-elastic actuators from measured data, trained a policy against the resulting hybrid simulation, and deployed it directly on the machine. Later work industrialized this recipe in GPU-parallel simulators [Rudin 22]. Yamamori et al. [Yamamori 26] have since inverted the premise, shaping the physical actuator's closed-loop response to match the idealized dynamics assumed in simulation. That option is unavailable on a commercial implement carrier whose valve and pump are not open to redesign, which is why the modeling burden here falls on the actuator

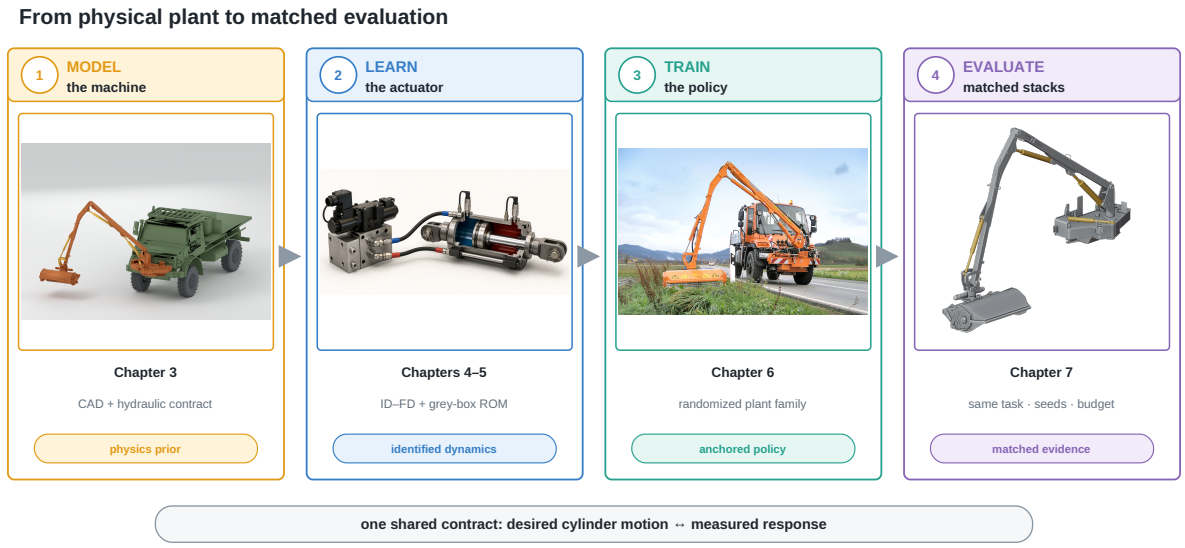

Figure 2.2: The four-step workflow of this thesis, following the division of labor introduced for legged robots by Hwangbo et al. [Hwangbo 19]. Each card names what the step contributes and the chapter that develops it.

representation. Figure 2.2 shows the same division of labor recast for hydraulics. The PhysX articulation plays the analytic rigid-body part, and the learned part is not a torque filter for a well-behaved electric drive but the entire hydraulic chain of Figure 2.1. The surrogate must therefore be hydraulic-aware and command-conditioned, realizing desired cylinder commands rather than filtering torques.

2.6 Domain Randomization and Latent Adaptation

2.6.1 Robustness Through Randomized Training

A model or policy trained on one simulated plant inherits every parameter of that plant as an implicit assumption. Domain randomization attacks this brittleness by training across a distribution of plants, originally over visual appearance for perception [Tobin 17], soon after over dynamics parameters such as masses, frictions, gains, and latencies for control [Peng 18]. The now-standard reading of those results is that a policy or model that must succeed on many plants converges to behavior that is *robust* across the family. For the hydraulic arm the corresponding family is physically meaningful rather than synthetic: effective bulk modulus varies with entrained air and temperature, valve gains and leakage drift with wear, and payload changes with the mounted attachment. Chapter 4 defines the randomized parameter space along these same axes.

2.6.2 From Robustness to Adaptation

Robustness, however, is a compromise by definition: one response that is acceptable everywhere is optimal nowhere. The alternative is to let the controller or model identify which member of the family it is currently facing and condition on that estimate (Figure 2.3). The teacher–student construction makes this trainable. During simulation the true

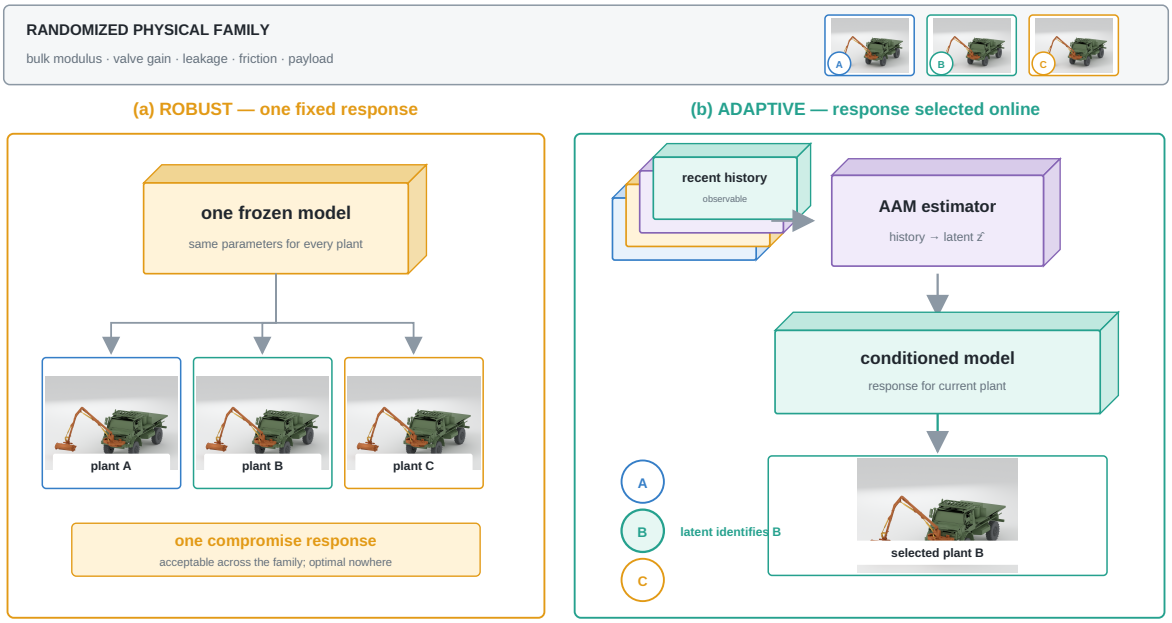

Figure 2.3: Two treatments of the same randomized plant family. (a) Randomized training alone gives a *robust* model: one fixed response, acceptable everywhere and optimal nowhere. (b) Latent conditioning adds *adaptation*: an estimator infers $\hat{\boldsymbol{z}}$ from recent measured history and the model changes its response. The estimator is trained against a privileged latent but deploys on observable signals alone.

randomized factors $\mathbf{e}$ are known, and a privileged encoder compresses them, or their dynamic consequences, into a low-dimensional latent $\boldsymbol{z}$. A student module then learns to regress $\boldsymbol{z}$ from signals that exist outside simulation, namely the recent history of states, commands, and responses. Lee et al. [Lee 20] used this scheme to walk over terrain whose properties the robot cannot observe directly. Kumar et al. [Kumar 21] sharpened it into rapid motor adaptation. A first phase trains the policy together with an encoder of the privileged environment parameters. A second phase trains an adaptation module to recover the same latent from the observable state–action history. At deployment only the history-based estimate is used, so the system adapts within fractions of a second and takes no gradient steps. Classical adaptive control pursues the same goal through online parameter estimation [Åström 95]. The learned variant replaces explicit parameter estimates with a task-shaped latent, trading physical interpretability for the ability to summarize many interacting parameters at once. The construction has since been carried into the dynamics model itself rather than the policy. Yu et al. [Yu 26] condition a neural-ODE dynamics model on a context vector inferred from state–action history, so the learned model rather than the controller is what adapts to the current plant. That is the position the AAM occupies here, and the reason this thesis measures the adaptation effect at the actuator layer instead of inside the policy.

The Arm Adaptation Module (AAM) of this thesis (Chapter 5) is this construction applied to hydraulic actuation, following the adaptive path of Figure 2.3(b). The privileged latent $\boldsymbol{z} = \mu(\mathbf{e})$ is a two-dimensional compression of the physical factor vector $\mathbf{e}$, validated by the system-dynamics surrogate of Chapter 5. The AAM reads its deployable estimate $\hat{\mathbf{z}}$

from a quarter second of causal, observable cylinder-signal history.

Where that estimate is *used* is a design decision that this thesis fixes explicitly, because it is easy to state loosely. The latent does not condition the reduced-order model. In the proposed stack, PPO–ID(AAM)–FD, the inferred latent conditions a small bounded residual adapter around the *frozen inverse-dynamics model*, leaving the forward model, the policy, and the PhysX plant unchanged. Conditioning the policy directly on $\hat{\mathbf{z}}$ is treated as an ablation rather than the default design (Chapter 7). Two claims are therefore kept apart throughout. A model trained across randomized plants is *robust*, not automatically adaptive, and evidence that a *privileged* latent carries useful dynamics information is not evidence that an *inferred* one improves control. Collapsing either pair is a common confound in the sim-to-real literature, and the experimental design of Chapter 7 is built to avoid both.

2.7 Reinforcement Learning and Structured Priors

2.7.1 Policy Optimization

Reinforcement learning frames control as sequential decision making: a policy $\pi_\vartheta(\boldsymbol{a} \mid \boldsymbol{o})$ maps observations to actions and is optimized to maximize expected cumulated reward [Sutton 18]. Among policy-gradient methods, proximal policy optimization (PPO) [Schulman 17] has become the default in robot learning for its simplicity and stable large-batch behavior. It constrains each update through a clipped surrogate objective, which limits how far the ratio between the new and the old policy may move before the improvement term stops paying. Advantages are estimated with generalized advantage estimation [Schulman 16]. The objective as configured for this thesis is written out in Section 6.2.5. What makes on-policy methods like PPO practical for this thesis is throughput. GPU-parallel simulators run thousands of environment instances at once [Makoviychuk 21], and locomotion policies have been trained this way in minutes [Rudin 22]. This is the regime in which the full hydraulic model cannot run (Section 2.1.3) and the learned actuator surrogates can.

2.7.2 Structured Priors for Policy Learning

Applying RL to the mowing arm from scratch would discard the reliable kinematic structure of Section 2.2 and force the policy to rediscover it from reward, with excursions a heavy boom cannot afford. Three constructions inject that structure, and they differ in *when* the classical controller acts.

Residual reinforcement learning adds the policy output to a hand-designed controller's command at runtime [Johannink 19, Silver 18], with the policy restricted to a bounded band. The nominal controller provides competent behavior from the first step, but it remains in the deployed loop, so every result is a property of the pair. Policy distillation moves the prior to training time instead: a privileged teacher, here the DLS controller reading quantities the deployed system cannot, generates demonstrations, and the student is fitted to them by supervised imitation before any reward is optimized [Ross 11]. At deployment the teacher is gone and the student is the whole controller. The third construction, developed in Section 6.4, keeps the prior for the whole of optimization without returning it to the runtime loop. The distilled student is frozen as a *reference*

policy and reward optimization is restricted to a neighbourhood of it, measured as a divergence between the two action distributions and evaluated only on states the learner itself visits.

This thesis uses the second and third. The deployed policy of all three stacks is a *sole* cylinder-motion controller, emitting the desired cylinder command with no nominal command added at runtime, and the DLS controller is retained only as teacher and as the closed-loop controller during identification. What bounds exploration is therefore not a residual band but the action contract itself, specified in Chapter 6.

Improving a policy that a simulator has already made competent is an active question. Yin et al. [Yin 25] retain the value function learned in simulation to guide exploration during real-world fine-tuning, treating the simulator as a source of structure rather than a plant to be abandoned. This thesis shares that premise and differs in what is retained: the policy itself, held as a fixed reference that bounds how far reward optimization may travel.

That construction answers a measured result rather than a design assumption. *Unconstrained* PPO fine-tuning from the distilled student raised total reward while degrading the primary tracking objective, consistently enough that distillation was for several development cycles treated as the *endpoint* rather than an initialization. The diagnosis is a mismatch of objectives rather than a failure of the optimizer: the reward is a proxy, acceptance is decided by physical gates, and the reward's optimum does not lie inside the gate-feasible set (Section 6.3). Anchoring is what allows the reported PPO–ROM policy to be a fine-tuned actor rather than a distilled one.

2.7.3 Reinforcement Learning on Hydraulic Machines

Applications of deep RL to hydraulically actuated arms are still few, and the closest of them sharpen rather than close the gap this thesis addresses. Egli and Hutter demonstrated RL-based trajectory tracking for the arm of a full-size hydraulic excavator, first in simulation-to-machine transfer experiments [Egli 20] and then as a general automation approach deployed on a walking excavator [Egli 22]. Spinelli et al. [Spinelli 24] extended that line to material handlers whose tool is underactuated, training in simulation and validating on a commercial machine, which shows the approach scales to arms where part of the mechanism is not directly commanded. The mowing head of this thesis is underactuated in the same sense, through its passive swivel joint. Maitan et al. [Maitan 26] attacked the problem from the opposite end, learning force control for a single hydraulic actuator directly on hardware and constraining exploration with a contraction-metric filter so that training on a real cylinder stays safe. Their work is the closest published statement of the difficulty this thesis handles in simulation: an unconstrained policy acting on a hydraulic cylinder is not safe by default, and something must bound it. Their key ingredient is shared with this work: a data-driven model of the machine's actuator response, learned from recorded operation and used as the training plant for the policy. The differences define where this thesis stands. Their actuator model is identified for one physical machine, so robustness across a plant family and online adaptation to a changing plant are outside its scope. Their policy is trained against that single model without a privileged teacher supplying the structured prior. And their evaluation compares against human operators and classical baselines rather than isolating the contribution of the actuator representation itself. The comparison that Chapter 7 is built around (same task, same policy architecture,

Table 2.1: Positioning of this thesis against the reviewed lines of work. Columns: (I) learned model of *hydraulic* actuator dynamics; (II) training data audited for causal input identifiability; (III) online latent adaptation to a changing plant; (IV) structured classical controller retained as a prior, with bounded action authority; (V) matched comparison isolating the actuator representation. (✓) = partially or implicitly addressed.

Line of work	I	II	III	IV	V
Model-based hydraulic control [Mattila 17, Jelali 03]	—	(✓)	(✓)	✓	—
Actuator networks for legged robots [Hwangbo 19, Rudin 22, Yamamori 26]	—	✓	—	—	—
RL for hydraulic machines [Egli 20, Egli 22, Spinelli 24, Maitan 26]	✓	(✓)	—	—	—
Latent online adaptation [Lee 20, Kumar 21, Yu 26]	—	—	✓	—	—
Residual RL and policy distillation [Johannink 19, Silver 18, Ross 11, Yin 25]	—	—	—	✓	—
This thesis (PPO–ROM / PPO–ID–FD / PPO–ID(AAM)–FD)	✓	✓	✓	✓	✓

same observation and action contracts, same terrain, reward, seeds and training budget, with only the actuator and adaptation path differing) has to the author's knowledge no published counterpart for hydraulic manipulators.

One property of a learned policy over a learned actuator cuts both ways and shapes the evaluation methodology of this thesis. Because the policy can compensate for a mediocre actuator model, closed-loop tracking error alone cannot distinguish a good model from a good patch over a bad one. The protocol of Chapter 7 therefore reports model-level rollout metrics, the magnitude and smoothness of the emitted commands, and the standing cylinder-tracking and actuator-prediction residuals alongside task error. A better actuator representation must show up as a smaller standing model error and quieter commands, not merely as equal tracking.

2.8 Research Gap and Thesis Positioning

Laid side by side, the threads of this chapter each solve part of the problem and leave a specific remainder (Table 2.1 names the works). Hydraulic control theory provides structure and explains why fixed models fail, but its precise methods presuppose plant knowledge that a commercial mowing implement does not come with. Learned dynamics models replace derivation with data, but the identifiability literature, and the measured failure recounted in Section 2.4.2, shows that closed-loop operation data silently produces models without control authority. Learned actuator surrogates embedded in parallel simulation are proven for electric drives but not for actuators dominated by pressure dynamics, relief saturation, and valve dead zones. Domain randomization yields robustness, and the teacher–student construction yields fast online adaptation, but these results live almost entirely in locomotion, where the adapted quantity is the contact with the world rather than the actuator itself. RL has reached full-size hydraulic arms, but without a structured prior, without adaptation, and without isolating the actuator representation. Residual RL and policy distillation both integrate learning with a structured controller, yet published evaluations rarely control for the fact that the policy can mask actuator-model deficiencies.

What is missing is the combination, summarized in Table 2.1: cylinder-space actuator dynamics learned from deliberately identifiable data, reduced to a physics-structured

command-conditioned model fast enough for massively parallel policy training, carrying a dynamics latent that is validated before deployment and recovered online from observable history alone, and evaluated in a framework where adaptive and non-adaptive actuator representations are compared under matched conditions. Assembling this on the MULAG FME 500 mowing arm is the contribution of this thesis. It takes the form of three control stacks differing *only* in the actuation and adaptation path between the policy's cylinder command and PhysX:

PPO–ROM. The policy commands desired cylinder motion, which the validated command-conditioned reduced-order model realizes. This is the certified actuator baseline and the current shippable stack.

PPO–ID–FD. The same policy interface drives the frozen inverse model followed by the validated strict forward model. This is the non-adaptive learned baseline. It tracks the line more finely than PPO–ROM and fails the ground-contact requirement, which makes it an informative negative result rather than a weaker copy.

PPO–ID(AAM)–FD. The AAM infers a dynamics latent from causal observable history, and that latent conditions a small bounded residual adapter around the frozen inverse model before the same frozen forward model and PhysX. This is the proposed adaptive method.

The scientific variable is therefore the actuator and adaptation path alone, with task, observation and action contracts, terrain, reward, training budget, test factors, seeds and acceptance gates held matched. Robustness and adaptation are measured as separate effects rather than conflated ones. The next chapter grounds this program in the concrete system.

3. System Fundamentals and Problem Formulation

Chapter 2 closed with a program: learn the hydraulic actuation of the FME 500 from identifiable data, wrap the learned models in a structured controller, and measure robustness and adaptation as separate effects. Before any of that can be made concrete, the system itself has to be pinned down. This chapter describes the manipulator and its two simulated representations exactly once, so that every later chapter can refer back here instead of re-explaining the plant. Sections 3.1 and 3.2 define the mechanism, its coordinate spaces, and the mappings between them. Section 3.3 fixes which signals exist, which are commanded, and which are merely estimated. Section 3.4 summarizes the high-fidelity Simscape model that generated the historical actuator-model data, and Section 3.5 the IsaacLab environment in which policies are trained, adaptation data is collected, and every result in this thesis is measured. Section 3.6 defines the verge-following task and the actuator ceiling that constrains it. Section 3.7 assembles the control architecture and the three stacks that differ inside it. Sections 3.8 and 3.9 state the control problem and the criteria the thesis is judged by.

3.1 Manipulator Platform

The MULAG FME 500 is modeled as a serial-link manipulator with five revolute joints. Figure 3.2 shows the kinematic topology. The chain starts at the base frame bolted to the Unimog's front attachment plate (link 0), continues through the slew column (link 1), the two boom segments (links 2 and 3), and the head carrier (link 4), and ends in the flail mowing head (link 5). Four double-acting hydraulic cylinders actuate the first four joints. Each cylinder closes a triangle over its joint, so cylinder extension and joint angle are related through the linkage geometry. The fifth joint, the rotation of the mowing head itself, carries no cylinder of its own in the configurations studied here: it is constrained by a swivel-link mechanism coupled to the head carrier, and the control task treats it as passive. Throughout the thesis the four actuated cylinders are indexed in the fixed order introduced in Chapter 5: slew (*Drehzapfen*), boom I (*Ausleger I*), boom II (*Ausleger II*), and head pivot (*Messerkopf-Schwenk*).

(a) (b)

Figure 3.1: The machine studied in this thesis, as delivered. (a) MULAG FME 500 front mower on a Unimog carrier during roadside mowing. (b) Manufacturer's working envelope, spanning roughly 8 m of height and reaching below road level. Images: MULAG Fahrzeugwerk [MULAG Fahrzeugwerk, Heinz Wössner GmbH u. Co. KG 26].

The geometry is not hand-approximated. Link shapes, masses, centers of mass, and full inertia tensors were extracted from the manufacturer's CAD data using the Blender add-on Phobos [von Szadkowski 20], which exports a consistent URDF description of the mechanism. This matters for a machine like the FME 500: the boom is a long, asymmetric weldment whose cross-inertia terms are significant, and a lumped-mass approximation would misrepresent the very load coupling that makes the hydraulics hard to control. The same CAD-derived description is the common ancestor of both simulated representations used in this thesis, the Simscape model of Section 3.4 and the IsaacLab articulation of Section 3.5, so their mechanical parameters agree by construction.

Table 3.1 lists the actuated joints and the extension ranges of their cylinders as observed across the training data. Two geometric facts recur throughout the thesis. First, the controlled point is not the origin of the head body: the cutter contact point sits 0.79 m below the *Messerkopf* body origin, and all task-space errors are measured at this offset tool tip. Second, the mowing pose of interest keeps the head facing straight down. The head-pivot cylinder holds a dedicated reference extension (0.589 m) to maintain this orientation, which effectively reserves the fourth cylinder for the orientation task and leaves cylinders 1–3 for positioning.

3.2 Kinematic Structure and Coordinate Mappings

3.2.1 Joint Space and Cylinder Space

The mechanism's configuration is described by the joint vector $\boldsymbol{q} \in \mathbb{R}^5$ and its velocity $\dot{\boldsymbol{q}}$. The actuated coordinates, however, are the cylinder extensions $\boldsymbol{x} \in \mathbb{R}^4$ introduced in Chapter 5. Each cylinder connects an attachment point $\boldsymbol{p}_A$ on link k to a point $\boldsymbol{p}_B$ on

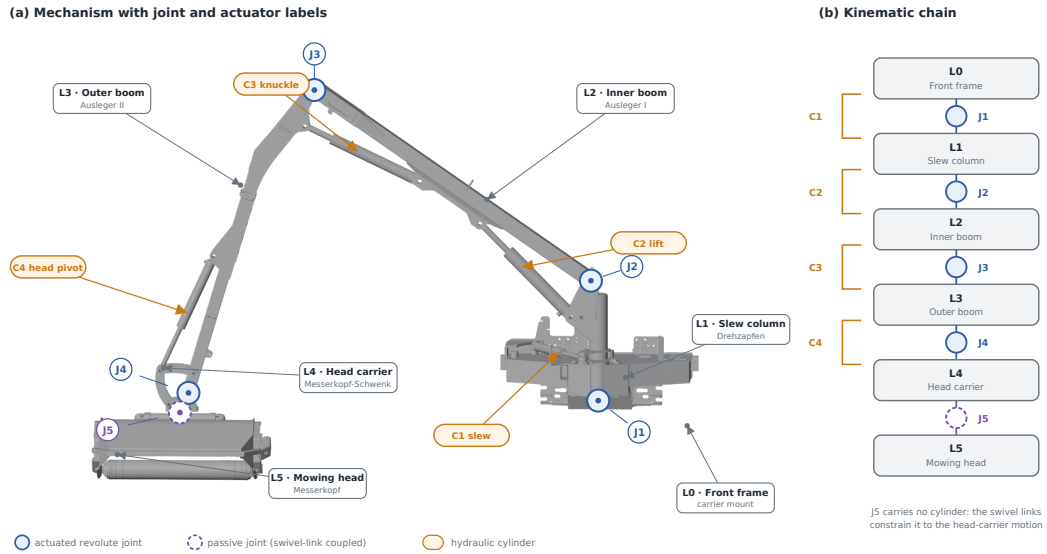

Figure 3.2: Kinematic topology of the MULAG FME 500, rendered from the CAD model. (a) The mechanism with every link (L0–L5), joint (J1–J5) and cylinder (C1–C4) called out; J5 is dashed to mark it passive. (b) The same structure as an abstract chain, with the passive fifth joint coupled through swivel links rather than a cylinder.

Table 3.1: Actuated joints of the modeled FME 500 and the extension ranges of their cylinders observed in the training data (Chapter 4).

i	Joint (German designation)	Function	Extension range [m]
1	*Drehzapfen*	slew about vertical axis	0.018 – 0.493
2	*Ausleger I*	inner boom elevation	0.275 – 0.791
3	*Ausleger II*	outer boom articulation	0.309 – 0.857
4	*Messerkopf-Schwenk*	head pivot (orientation)	0.269 – 0.689
5	*Messerkopf*	head rotation, passive (swivel links)	—

link $k{+}1$, both fixed in their links and exported from the CAD data. The instantaneous cylinder length and extension are

$$\ell_i(\boldsymbol{q}) = \big\|\boldsymbol{p}_B(\boldsymbol{q}) - \boldsymbol{p}_A(\boldsymbol{q})\big\|, \qquad x_i = \ell_i(\boldsymbol{q}) - \ell_{0,i}, \tag{3.1}$$

with $\ell_{0,i}$ the retracted reference length. Differentiating (3.1) gives the scalar mapping between joint motion and extension rate,

$$\dot{x}_i = J_{\mathrm{cyl},i}(\boldsymbol{q})\,\dot{\boldsymbol{q}}, \qquad J_{\mathrm{cyl},i} = \boldsymbol{e}_i^{\top}\big(\boldsymbol{J}_B - \boldsymbol{J}_A\big), \tag{3.2}$$

where $\boldsymbol{e}_i$ is the unit vector along the cylinder axis and $\boldsymbol{J}_A, \boldsymbol{J}_B$ are the translational Jacobians of the attachment points. The same mapping carries force back to torque: by the principle of virtual work, a cylinder force $F_{h,i}$ contributes the joint torque $\boldsymbol{\tau}_{\mathrm{act}} = \boldsymbol{J}_{\mathrm{cyl}}^{\top}\boldsymbol{F}_h$. The rows of $\boldsymbol{J}_{\mathrm{cyl}}$ are the lever arms discussed qualitatively in Chapter 2. They vary over the stroke, and with them the effective gain and stiffness that each cylinder presents to its joint.

3.2.2 Task Space and the Calibrated Cutter Jacobian

The task is defined at the cutter tip, expressed in the truck chassis frame (`base_link`). This choice is deliberate and matters in practice: the articulation root of the simulated arm is the slew base, which moves with the arm, and using it as the reference frame corrupts every tracking measurement. All references and errors in this thesis live in the truck frame, and the controlled point includes the $-0.79\,\mathrm{m}$ tool-tip offset of Section 3.1. Because the truck itself drives forward during the primary task (Section 3.6), the chassis frame translates along the road while the errors are measured inside it. The along-track direction is supplied by the vehicle and is not a controlled degree of freedom.

For control, the composition of task kinematics and cylinder linkage is captured in a single calibrated Jacobian $\boldsymbol{J}_c \in \mathbb{R}^{3\times 4}$ that maps cylinder extension rates to cutter-tip velocity in the truck frame. Rather than deriving it symbolically through the CAD chain, it was measured directly on the simulated plant: starting from the head-down mowing configuration, each cylinder target was stepped in turn and the settled cutter displacement recorded, giving central-difference estimates of every column. The measured matrix has rank 3 with singular values 8.24, 5.69, and 2.79, confirming that at the mowing operating point the four cylinders span exactly three independent task directions. This is the quantitative form of the observation from Section 3.1: with cylinder 4 committed to holding the head down, the remaining freedom moves the cutter in a three-dimensional workspace, and one direction of the nominal base frame is reachable only through vehicle motion. The damped least-squares controller of Chapter 6 inverts this measured $\boldsymbol{J}_c$, so the nominal layer and the plant share one geometric contract.

3.3 Actuation and Measured State

Every interface in this thesis draws from one signal inventory, and it is stated once here, together with who can see what. Table 3.2 summarizes the quantities, their dimensions, and their availability in the two simulation environments.

The separation encoded in that table drives several design decisions. The valve command $\boldsymbol{u}$ exists only inside the Simscape plant and its data exports. In IsaacLab no valve is simulated, so any model that needs valve information at runtime must either receive it from a learned inverse model (the PPO–ID–FD and PPO–ID(AAM)–FD paths) or predict it internally (the reduced-order model of the PPO–ROM path). The pressure difference $\Delta\boldsymbol{p}$ likewise has no native counterpart in the rigid-body simulation. At runtime it is a recursive internal state of the learned actuator models rather than a measurement, and it is deliberately withheld from the deployment-mirror inputs of Chapter 5 so that the causal history estimator cannot lean on a quantity a real machine would not publish. Only positions and velocities, of cylinders, joints, and the cutter tip, together with the terrain scan, are common ground, which is why the acceptance gates of Section 3.9 are formulated on these quantities.

The last three rows carry the rule that governs the whole adaptation argument. The factor vector $\mathbf{e}$ and the latent $\boldsymbol{z} = \mu(\mathbf{e})$ derived from it are privileged: they are known to the simulator, they are used to *train* and to *audit*, and they are never read by anything that claims to be deployable. Only $\hat{\mathbf{z}}$, inferred from causal observable history, may enter the adaptive stack. Commanded quantities carry a star and estimated ones a hat, following the notation of Chapter 5; $\hat{\mathbf{z}}$ exists only in PPO–ID(AAM)–FD.

Table 3.2: Signal inventory of the modeled system. "S" marks signals present in the Simscape data exports, "I" signals available live in the IsaacLab environment, "M" quantities that exist there only through the learned models of Chapter 5, and "P" *privileged* quantities that exist in simulation but must never enter a deployable policy or estimator input.

Signal	Dimension	Availability	Meaning
$\boldsymbol{u}$	$[-1,1]^4$	S, M	normalized proportional-valve commands
$\boldsymbol{x}$, $\dot{\boldsymbol{x}}$	$\mathbb{R}^4$	S, I	cylinder extensions and velocities
$\Delta\boldsymbol{p}$	$\mathbb{R}^4$	S, M	chamber pressure differences
$\boldsymbol{F}$	$\mathbb{R}^4$	S	logged rod forces
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	$\mathbb{R}^5$	S, I	joint positions and velocities
$\boldsymbol{p}_c$	$\mathbb{R}^3$	S, I	cutter-tip position, truck frame
$\boldsymbol{h}$	$\mathbb{R}^9$	I	forward terrain-height profile ahead of the cutter
$\boldsymbol{c} = [\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*]$	$\mathbb{R}^8$	I	desired cylinder command
$\mathbf{e}$	$\mathbb{R}^6$	P	physical randomization factors of the active plant
$\boldsymbol{z} = \mu(\mathbf{e})$	$\mathbb{R}^{d_z}$	P	privileged dynamics latent, $d_z = 2$
$\hat{\boldsymbol{z}}$	$\mathbb{R}^{d_z}$	M	AAM estimate of $\boldsymbol{z}$ from observable history

3.4 High-Fidelity Simscape Model

The primary data source for the *actuator* models of this thesis (the command-conditioned reduced-order model and the inverse/forward pair) is a multi-domain Simscape model of the FME 500 that I developed in the master project preceding this thesis. This section summarizes the parts of that model the thesis depends on. The modeling and validation work itself is part of the contribution of this research program.

The division of provenance is stated explicitly, because the data campaigns of Chapter 4 are easy to conflate and the three actuator paths do *not* share the same one. Table 3.3 states it once.

The Simscape model is therefore *historical* provenance for the bulk of every actuator model, and the sole provenance of the frozen ID and strict FD checkpoints, which are carried forward unchanged. It is not the whole story for the command ROM: after its Simscape training the ROM received a small, bounded, closed-loop calibration inside IsaacLab itself, described in Section 5.4.6. And it produced nothing at all for the adaptation chain: the system-dynamics surrogate, the privileged factor encoder, and the AAM are trained from scratch on data collected in IsaacLab through the command ROM and PhysX (Section 3.5).

3.4.1 The Command ROM as Deployment Actuator

One design decision governs everything downstream and is therefore stated explicitly: *the command ROM plus PhysX is the deployment plant of this thesis.* Inside the simulator it stands in for the real machine. Simscape is too slow to be that stand-in (Section 2.1.3) and the bare PhysX articulation has no hydraulics, so the hydraulically structured, PhysX-calibrated ROM is the best representation available at policy-training throughput. That is why it alone was calibrated against the deployment loop, and why the system-dynamics surrogate and the AAM describe *it* rather than Simscape.

Table 3.3: Provenance of each frozen actuator component. The command ROM is the only one that has been calibrated against the deployment plant; the inverse/forward pair is Simscape-identified throughout. This asymmetry is a property of the comparison and is carried into the discussion of Chapter 7, not glossed over.

Component	Identified on	Calibrated on
Command ROM	Simscape closed-loop export (grey-box, full model)	IsaacLab/PhysX closed-loop rollouts, via a bounded command-gated correction head only
Control ID	Simscape closed-loop export	—
Strict FD	Simscape closed-loop export, retrained on the strict open-loop excitation export	—
SD_0, $\mu(\mathbf{e})$, SD_z, AAM	IsaacLab/PhysX adaptation collection (from scratch)	—

The ROM therefore holds two roles at once: it is the plant, and it is the actuator of PPO–ROM. The consequence for the three-way comparison is carried into Chapter 7 rather than left implicit:

- PPO–ROM is the stack whose actuator *coincides with the plant*. It therefore measures how well the task can be done when the actuator model is, by construction, exactly right. It is the reference condition, not merely the best of three models.
- PPO–ID–FD substitutes an independently identified learned actuator pair for that plant model. It measures the cost of that substitution: what is lost when the actuator representation is identified elsewhere and must reproduce the deployment plant rather than be it.
- PPO–ID(AAM)–FD measures how much of that cost online adaptation recovers.

Read this way the ID transfer gap of Chapter 7 is an expected quantity rather than an anomaly. Read carelessly, PPO–ROM's advantage looks like evidence that a reduced-order model approximates better than an inverse–forward pair, which is *not* what the experiment can show. This qualification applies wherever PPO–ROM is quoted as the certified baseline.

3.4.2 Mechanical Subsystem

The mechanism is simulated as a rigid multibody system in Simscape Multibody [The MathWorks, Inc. 24 with the CAD-derived links, joints, and cylinder attachment frames of Section 3.1. The joint-space dynamics take the standard manipulator form

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_{\text{fric}}(\dot{\boldsymbol{q}}) = \boldsymbol{\tau}_{\text{act}} - \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{\text{ext}}, \tag{3.3}$$

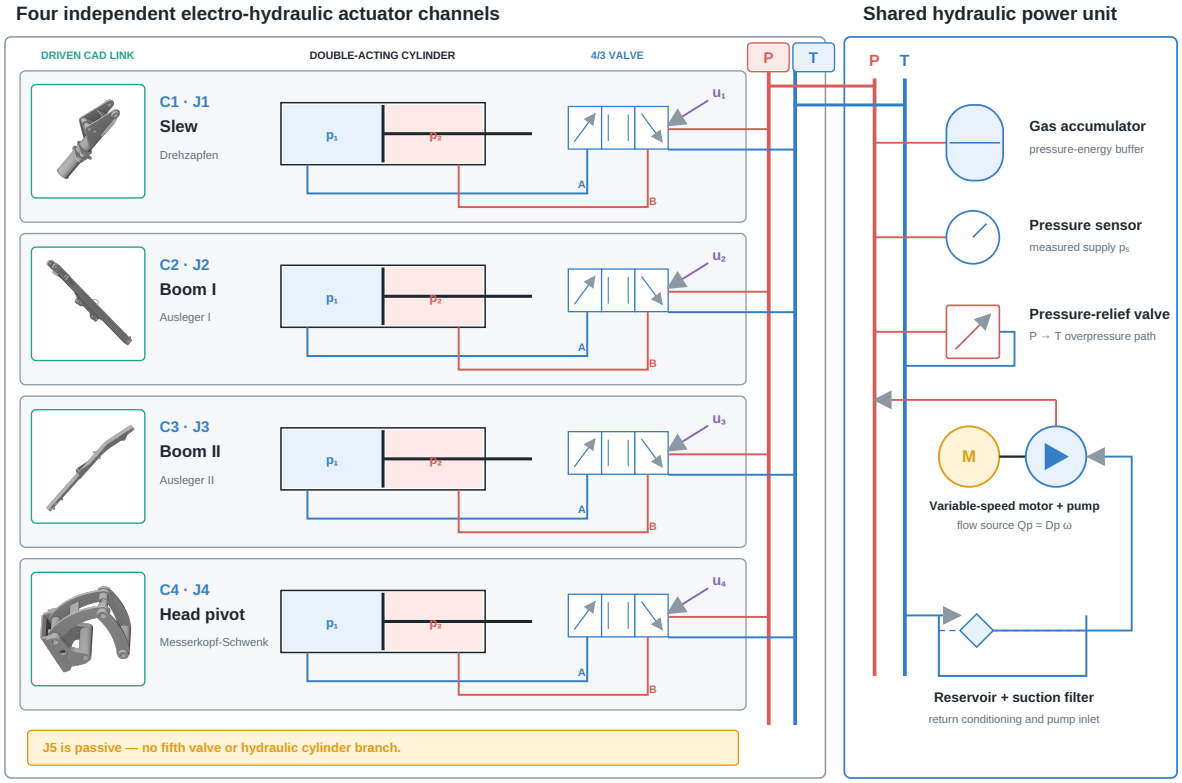

Figure 3.3: System-level hydraulic schematic. Left: the four independent actuator channels, each a 4/3 proportional directional valve driving one double-acting cylinder. Right: the power unit shared by all four channels. Link thumbnails are rendered from the CAD model of Figure 3.2.

with the inertia matrix $\boldsymbol{M}$ assembled from the imported CAD tensors. Two terms deserve comment. The friction torque $\boldsymbol{\tau}_{\text{fric}}$ combines viscous and Coulomb components with a smooth tanh approximation of the sign function, extended by a Stribeck term at the cylinder level to capture stick–slip at low velocities. The slew joint, which carries high side loads, is particularly sensitive to this. The external wrench $\boldsymbol{F}_{\text{ext}}$ models the vegetation cutting load as a nominal force plus a disturbance that scales with feed velocity and vegetation density, rather than as a constant, so mowing does not reduce to a static payload. Each cylinder is built as a closed kinematic loop (spherical–prismatic–cylindrical, shown in the top band of Figure 3.4), which lets it self-align under load without over-constraining the mechanism, and hard-stop constraints enforce the physical stroke limits of Table 3.1.

3.4.3 Hydraulic Subsystem

Figure 3.3 shows the assembled plant at system level. The hydraulic side is deliberately physical rather than phenomenological. Supply comes from a fixed-displacement pump driven at variable speed ($Q_p = D_p\,\omega$), buffered by a gas-charged accumulator (polytropic gas model) and protected by a pressure-relief valve, so supply pressure is a dynamic quantity that sags under simultaneous demand and clips under load spikes. Each of the four actuators is a 4/3 proportional directional valve driving an asymmetric double-acting cylinder. Figure 3.4 details one such channel, from the rigid-body assembly of barrel and

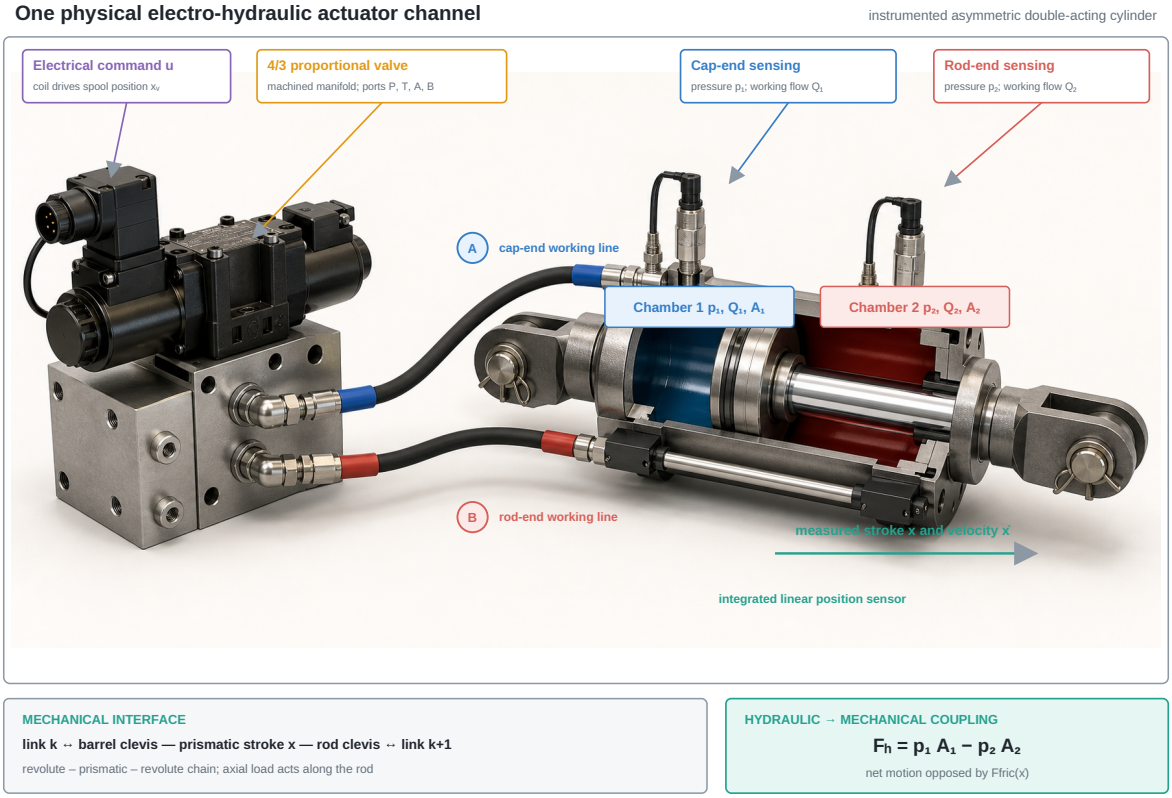

Figure 3.4: One electro-hydraulic actuator channel. Top: the mechanical interface, in which barrel and rod connect to the adjacent links through a revolute–prismatic–revolute chain whose prismatic coordinate is the stroke x. Bottom: the hydraulic interface, an asymmetric double-acting cylinder with chamber pressures and flows routed to ports A and B of the proportional valve.

rod through the mechanical–hydraulic coupling interface down to the valve, the chambers, and the measured feedback signals. The chamber pressures follow the continuity equations

$$\dot{p}_1 = \frac{E_{\text{eff}}}{V_1(x)}\Big(Q_1 - A_1\dot{x} - C_{\text{leak}}(p_1 - p_2)\Big), \qquad \dot{p}_2 = \frac{E_{\text{eff}}}{V_2(x)}\Big(-Q_2 + A_2\dot{x} + C_{\text{leak}}(p_1 - p_2)\Big), \tag{3.4}$$

with stroke-dependent chamber volumes $V_{1,2}(x)$ that include the line dead volumes, piston areas $A_1 \neq A_2$, and laminar internal leakage C_{leak}. The effective bulk modulus accounts for entrained air through

$$E_{\text{eff}}(p) = \left(\frac{1}{E_f} + \frac{\alpha}{p}\right)^{-1}, \tag{3.5}$$

so fluid stiffness itself is pressure dependent, weakest where precise low-pressure positioning is attempted. Valve flow follows the orifice law of Chapter 2 (Section 2.1.2), and the spool is not an instantaneous gain but a first-order lag, $\dot{x}_v = (u - x_v)/\tau_v$, so command transients are bandwidth limited. The net piston force $F_h = p_1 A_1 - p_2 A_2 - F_{\text{fric}}(\dot{x})$ feeds back into the mechanism through the virtual-work mapping of Section 3.2, closing the bidirectional coupling: mechanical loads shape pressure transients, and pressure transients shape motion. Because cylinder-induced stiffness is high while rigid-body motion is slow, the coupled equations are stiff, and the model runs under a variable-step implicit solver.

This stiffness is the computational reason the Simscape plant cannot serve as an RL training environment (Section 2.1.3).

3.4.4 Validated Scope of the Model

A plant model intended as a data generator must be trusted for its *dynamics*, not just its geometry. The validation strategy in the modeling study was control-based: a fixed integral backstepping controller at 0.01 s was applied as a standardized excitation instrument, and the emergent behavior of the closed loop was examined as the physical parameters varied. The headline experiment sweeps the effective bulk modulus over $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ GPa, the physically meaningful range from strongly aerated to very stiff fluid, with all controller gains held constant. Actuation delay behaves as hydraulic intuition demands. The valve-to-joint delay of the heavy slew axis falls from about 0.85 s at 0.5 GPa to 0.65 s at 2.0 GPa, the system-level delay index from roughly 0.45 s to 0.35 s, and the improvement saturates beyond 1.5 GPa, where mechanical inertia rather than fluid compliance limits the response. Task-space tracking error falls monotonically with stiffer fluid, by about 11 % over the sweep. The model, in other words, reproduces the "stiff yet oscillatory" character of real hydraulic machinery and responds to parameter changes in the physically expected direction, which is the property that Chapter 4 exploits when it randomizes these same parameters to generate plant families.

Three limitations are carried forward honestly. The model is a single machine: manufacturing spread enters only through the deliberate parameter randomization of Chapter 4, not through measured variation. Thermal dynamics are not modeled. Temperature effects are represented indirectly through the parameters they act on (bulk modulus, friction). And the logged rod-force channel failed a piston-area consistency audit on the current export (Section 5.4), so force is treated as unreliable supervision throughout the thesis.

3.5 IsaacLab Environment

Policy training, adaptation-data collection, and every closed-loop result in this thesis run in IsaacLab [Mittal 23] on the GPU-parallel PhysX engine. The same CAD-derived mechanism enters here as a USD articulation: the Unimog carrier with wheels, the five arm links, and the four cylinders represented as prismatic actuator joints, with self-collisions enabled and the solver configured for the long-chain mechanism. Figure 3.5 renders that articulation directly from the asset the environment loads. The complete machine is shown once, because the arm-only views of Section 3.1 miss the geometry that defines the task. The boom is mounted on the front of a road-going carrier and reaches out to one side. The cutter therefore works several metres from the vehicle carrying it, while that vehicle keeps driving. Every lever arm, every reaction force on the chassis, and the entire reason the vehicle's motion must be treated as part of the plant follow from this arrangement.

The full hydraulic model does not run inside these environments, for the computational reasons of Section 2.1.3. Its behavior enters only through the learned actuator representations sitting between the controller and the PhysX cylinder targets.

Figure 3.5: The complete simulated machine, rendered from the USD articulation that the IsaacLab environment loads: the Unimog carrier with the FME 500 boom on its front frame and the flail head deployed to the side. Cylinders are highlighted in orange. The hydraulics are not present here and enter only through the learned actuator models.

Table 3.4: The four rates of the simulated system. They are reported separately throughout the thesis; "the control frequency" is not a well-defined quantity here.

Layer	Step	Rate
PhysX physics	1/120 s	120 Hz
Policy / control (decimation 2)	1/60 s	60 Hz
Learned actuator model (3 control steps)	0.05 s	20 Hz
AAM latent update	0.05 s	20 Hz

3.5.1 Simulation and Control Rates

Four different clocks run in this system, and reporting any one of them as "the control frequency" misstates the interface. They are listed in Table 3.4. The physics step is 1/120 s with a control decimation of two, so the policy observes and acts at 60 Hz (16.7 ms). The learned actuator models keep the native 0.05 s transition they were identified on: three consecutive control steps make one model tick, so a learned-model prediction is issued once every three policy steps and the resulting motion is distributed over the intervening PhysX updates as held increments. Nothing is interpolated onto the model grid, and the learning samples of Chapter 4 use exactly those 50 ms transitions at control indices $0, 3, 6, \ldots$. A model asked to predict across a mismatched interval is a silent 33 % error in the quantity it is supposed to supply. The AAM of Chapter 5 updates on the same 50 ms model grid from a causal history window, and logging runs at the control rate.

3.5.2 Environment Configuration Details

Three environment details cost real debugging time and are recorded here so they need not be rediscovered. First, the articulation root of the arm sits about 1.11 m above the wheel contact plane, so the vehicle must spawn with its root at $z = 1.26$ m. Spawning

at ground height buries the truck to its axles and silently changes every contact force. Second, cylinder actuation uses PhysX position targets, and the target must be integrated in *reference* space, accumulating commanded increments, rather than re-anchored each step to the measured position: re-anchoring lets gravity sag wash out every commanded increment, removing the policy's authority entirely. Third, the vehicle's forward motion is *scripted* rather than driven through the wheels: the chassis is advanced kinematically along the road at the commanded speed and its lateral position and heading are held by a planar fixture. This is not a convenience. The authored tyres and suspension of this asset do not produce usable traction in PhysX, and leaving the chassis free turns arm reaction forces into slow drift and sway that contaminate every task-space error. The residual sway the fixture has to remove is itself measured and gated (Section 3.9) rather than assumed to be zero.

3.6 The Verge-Following Task

3.6.1 Task Definition

The primary task of this thesis, and the one every reported control result is measured on, is verge following with the vehicle in motion. The truck drives along the road at a constant speed of 4 km/h in the development configuration, reduced to 2 km/h for the final PPO–ROM task of Section 6.5, where the strip is lengthened to 26 m. Meanwhile the four-cylinder arm reaches over the embankment and holds the cutter on the verge centre line: near the target cut height, in ground contact, head down, and inside the mechanical travel limits. Tracking the line is the objective. The terrain is a *disturbance* that makes tracking nontrivial, not an obstacle course to be negotiated, and its height profile is randomized per environment so that no policy can memorize one ground surface.

Reference paths are drawn from eleven families (`straight`, `lateral_sine`, `s_curve`, `random_spline`, `offset_return`, `boom_sweep`, `lateral_chirp`, `double_s_curve`, `lane_change_hold`, `gaussian_slalom`, and `fourier_meander`), which span a range of lateral demand rather than a single nominal line. Because aggregate scores can hide one impossible family behind ten easy ones, the acceptance criteria of Section 3.9 are evaluated at the *worst* family and the *worst* terrain mesh rather than pooled.

3.6.2 Task Feasibility and Plant Capability

One finding governs the design of everything above, and it is stated here because it is a property of the plant rather than of any controller. The action contract commits at most 4 mm of cylinder travel per step on cylinders 1–3 (Chapter 6), and over the interval across which that increment is committed this bounds the lateral cutter rate at

$$v_{\text{lat}}^{\max} \approx 0.200\,\text{m/s}. \tag{3.6}$$

[**TODO: confirm the exact interval used in the maintained figure: the** $0.200\,\text{m/s}$ **ceiling is quoted as** $4\,\text{mm}/20\,\text{ms}$**, while the environment's control step is** $1/60\,\text{s} = 16.7\,\text{ms}$ **(Table 3.4), which would give** $0.24\,\text{m/s}$**. Fix one and propagate.**] At the settings used before this was measured, nine of the eleven path families demanded more lateral rate than that, one of them (`boom_sweep`) by a factor of 3.7. Tracking error then

accumulates *by construction*: no reward shaping, critic change, or policy-optimization fix can recover a target the actuator cannot reach, and indeed the privileged DLS teacher failed on those families too, which is the diagnostic that identified the problem. The terrain and path parameters are therefore now *derived* from (3.6) rather than chosen independently of it, and a preflight check asserts the resulting feasibility before any training run is allowed to start.

The methodological rule generalizes beyond this machine and is applied throughout the thesis: before attributing a tracking failure to the learning algorithm, measure whether the commanded reference is inside the actuator's authority. A task outside the plant's capability produces the symptoms of a badly tuned policy.

3.6.3 Parked Diagnostic Workflow

A second, older workflow is retained and must not be confused with the task above. With the vehicle held stationary, the cutter tracks scripted line, arc, sine, circle, and square references in free space. This is a *diagnostic* instrument for qualifying the plant and the actuator models (step response, per-axis authority, out-and-back return error, and Jacobian conditioning), and it is how several of the cylinder-level checks in this thesis are performed. It is not a fourth control stack, it is not part of the final comparison, and its numbers are never pooled with the moving-verge statistics. Where a parked result is reported, it is labelled as a separate diagnostic experiment.

3.7 Control Architecture

Figure 3.6 assembles the pieces. The vehicle layer advances the chassis at the scripted speed of Section 3.6 and is not under policy control. The reference generator supplies the verge centre line and the target cut height.

The policy is the entire task layer. It reads normalized observations of the terrain, the tracking errors and the cylinder and actuator state, and emits four desired cylinder-motion commands. It is *not* a residual: no nominal command is added at runtime and no baseline controller sits underneath it. What keeps its commands inside the learned actuator's validated envelope is the action contract. The damped least-squares inverse appears twice in this thesis, as the privileged teacher and as the controller that drove the identification runs of Chapter 4, but never inside the deployed loop. Chapter 6 specifies the observation, the action contract and the teacher in full.

Below the task layer sits the experimental variable of the whole thesis: the actuator path that realizes the desired cylinder command. PPO–ROM realizes it with the command-conditioned reduced-order model, PPO–ID–FD with the frozen inverse and strict forward pair, and PPO–ID(AAM)–FD with that same pair plus a small bounded adapter conditioned on the inferred latent $\hat{\mathbf{z}}$. All three end in the same bounded PhysX cylinder position target, and the measured state closes the loop at the rates of Table 3.4.

3.8 Problem Formulation

With the system fixed, the control problem can be stated compactly. The vehicle advances at constant speed, so the reference is not a free three-dimensional trajectory. It is a verge

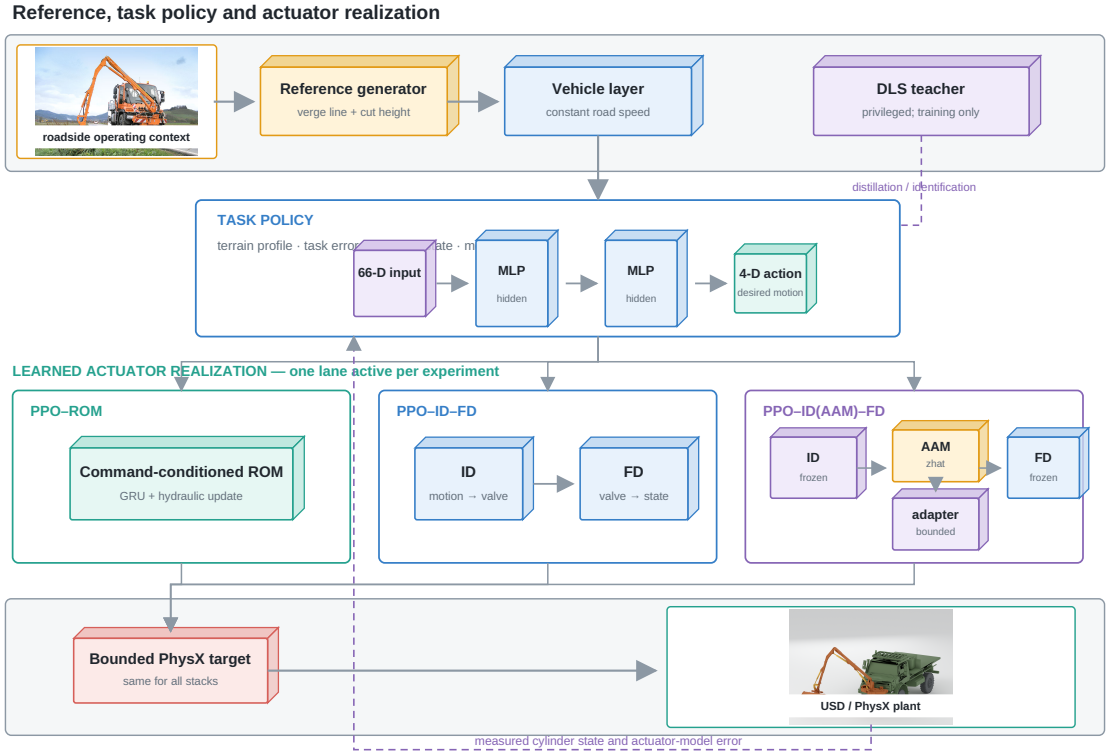

Figure 3.6: Control architecture used in all experiments, organized by layer. The vehicle layer advances the chassis and the reference generator supplies the verge centre line. The policy emits four normalized cylinder commands, which pass through the action contract to the learned actuator layer. Only that actuator layer differs between the three stacks. The DLS controller sits outside the deployed loop: it is the teacher and the identification controller, not a runtime layer.

centre line $y_{\mathrm{ref}}(t)$ and a target cut height h_{ref}, both expressed in the truck frame and drawn from the eleven path families of Section 3.6. Writing $\boldsymbol{p}_c(t)$ for the measured cutter-tip position and $g(\cdot)$ for the terrain height beneath it, the two scored errors are

$$e_{\mathrm{lat}}(t) = p_{c,y}(t) - y_{\mathrm{ref}}(t), \qquad e_{\mathrm{cut}}(t) = \Big(p_{c,z}(t) - g(\boldsymbol{p}_c(t))\Big) - h_{\mathrm{ref}}. \tag{3.7}$$

The *tracking objective* is to keep $|e_{\mathrm{lat}}|$ and $|e_{\mathrm{cut}}|$ small in mean and in the tail over complete episodes, on every path family and on unseen terrain, while the cutter stays in ground contact, the head stays down, and the motion remains smooth enough not to damage the machine. The *model objective*, developed in Chapter 5, is that each learned actuator representation predict the cylinder response to its commands accurately over one step and over recursive rollouts, with demonstrated input authority. The *robustness objective* asks that this performance persist across the randomized plant family $\{\mathcal{P}(\mathbf{e})\}$ of Chapter 4. The *adaptation objective* asks that the stack conditioned on the inferred latent $\hat{\mathbf{z}}$ recover performance on held-out factor configurations, further than the non-adaptive stack and approaching the privileged true-latent oracle, without degrading the nominal domain and without violating any acceptance gate.

All objectives are subject to the same constraint set: cylinder extensions within the ranges of Table 3.1 with a positive travel margin retained, per-step commanded increments and target leads bounded by the action contract of Chapter 6, valve or adapter outputs within the per-axis limits stored in the learned checkpoints, and the head held down throughout.

Table 3.5: The eight acceptance gates. Every gate is evaluated on at least three distinct terrain meshes and reported at its *worst* mesh, and the two tracking-critical gates additionally at the worst path family. Final claims use six held-out meshes.

Gate	Metric	Threshold	Aggregation
Cut height	mean $\lvert e_{\text{cut}} \rvert$	$\leq 0.040\,\text{m}$	worst mesh
Lateral tracking	mean $\lvert e_{\text{lat}} \rvert$	$\leq 0.300\,\text{m}$	worst mesh, worst family
Ground contact	contact fraction	≥ 0.700	worst mesh, worst family
Head alignment	p01 head-down alignment	≥ 0.900	worst mesh
Hard impacts	hard-impact step fraction	≤ 0.100	worst mesh
Tool acceleration	p95 cutter acceleration	$\leq 120\,\text{m/s}^2$	worst mesh
Cylinder headroom	minimum travel margin	$\geq 0.005\,\text{m}$	worst mesh
Safety terminations	termination rate	≤ 0.080	worst mesh

No single ingredient makes this problem hard. The difficulty is their combination, and each chapter so far has contributed one piece: the actuator chain is nonlinear and drifting (Chapter 2), only three task directions are controllable and the mapping into them varies with configuration (Section 3.2), the true actuator state is not observable at runtime (Section 3.3), the reference sits close to the plant's own rate ceiling (Section 3.6.2), the ground is a randomized disturbance rather than a known surface, and the high-fidelity plant that knows the hydraulic answers is too slow to train against (Section 3.4).

3.9 Evaluation Criteria at System Level

The thesis judges controllers and models by fixed, pre-registered gates rather than by single aggregate scores, and it separates the two levels: a controller metric can hide model weakness behind policy compensation (Section 2.7), so model metrics are always reported alongside.

3.9.1 The Eight-Gate Task Contract

At the *system level*, acceptance is an eight-gate contract, listed in Table 3.5. Each gate matches something an operator would notice. It cuts at the right height, follows the line, stays on the ground, keeps the head down, and does not slam into the terrain, it does not shake itself apart, it does not drive into its own travel limits, and its episodes complete.

Three rules about *how* these gates are used matter more than the thresholds themselves, and each of them was adopted after a measured failure of the previous practice.

Multi-Mesh Evaluation

An earlier contract scored every result on a single pinned terrain seed. When the mesh was finally varied, both the headline pass and the headline failure moved a long way: minimum head alignment ranged 0.9027 to 0.9698 across three meshes, and a checkpoint that "passed" by 0.0007 on the pinned mesh failed on the others. A gate that flips with the mesh is measuring the mesh. Results here are therefore refused unless they are measured on at least three meshes, and reported at the worst.

Gating at the Optimized Resolution

The same earlier contract read worst-of-eleven-family minima while the optimizer maximized a pooled mean. Across ten training blocks the mean ground contact rose from 0.817 to 0.870 while the worst-family contact never left the 0.66–0.75 band: nothing was optimizing what acceptance measured.

Measurement-Anchored Thresholds

A gate that nothing can pass carries no information, and several were found in that state. The tool-acceleration limit stood at $80\,\mathrm{m/s^2}$ while the environment alone, with the arm commanded to zero, measures a p95 of $100.06\,\mathrm{m/s^2}$. It was re-derived to $120\,\mathrm{m/s^2}$, $1.2\times$ the measured floor. The ground-contact and hard-impact thresholds were likewise anchored just outside what the privileged DLS teacher itself achieves, so that they separate a working controller from a non-working one rather than separating the teacher from perfection. Every threshold in Table 3.5 is therefore accompanied in the source by the measurement that fixes it, and every reported result carries its signed margin rather than a bare pass or fail, so that a pass by 0.0007 cannot masquerade as a comfortable one.

3.9.2 Cylinder- and Model-Level Criteria

At the *cylinder level*, the parked diagnostic workflow of Section 3.6.3 supplies step gates: clean response to fresh $\pm 10\,\mathrm{mm}$ commands on every axis, small terminal error on large ($80\,\mathrm{mm}$) coordinated motions, and low out-and-back return error, all without command saturation. Model-level criteria (one-step and rollout accuracy, input-ablation authority, and Jacobian conditioning) were introduced with the models in Chapter 5. Chapter 7 instantiates all three families of criteria in the full experimental matrix, and treats the eight-gate contract as the minimum acceptance criterion rather than as a substitute for reporting full distributions.

The system is now specified. The next chapter turns to the data the learned models are built from.

4. Data Generation and Preparation

Chapter 3 fixed the plant. It did not fix how its two simulated representations meet. The Simscape model knows the hydraulics but is far too slow to train a policy against, and the IsaacLab articulation is fast but has no hydraulics at all. Everything the learned models of Chapter 5 can say about the actuator has to travel from the first world to the second inside a dataset. This chapter describes how those datasets were designed, collected, audited and prepared.

The datasets come from two different sources, and keeping their provenance straight is a precondition for reading the results honestly (Table 3.3).

Historical Simscape provenance. Two Simscape campaigns identified the frozen actuator models that this thesis carries forward as comparison components: the command-conditioned reduced-order model and the inverse/strict-forward pair. The first campaign drives the arm with a closed-loop controller and produces the realistic operating distribution the inverse and reduced-order models learn from. The second removes the controller entirely and applies deliberate excitation to the valves, because the closed-loop campaign turned out to be unable, in principle, to identify what the valve does. Section 4.4 sets out that diagnosis and its repair in full, because the failure is easy to repeat and the repair is what makes the forward model usable.

Closed-loop calibration of the ROM in IsaacLab. The command ROM is not a pure Simscape artifact, whereas its partner in the comparison, the ID–FD pair, *is*. After Simscape training the ROM was fine-tuned on closed-loop rollouts collected inside IsaacLab itself, on the five plant-reachable reference shapes, with only a small bounded correction head left trainable and the Simscape material retained as a replay term (Section 4.6). It is therefore the one actuator model in this thesis that has seen the plant it is deployed in.

The IsaacLab adaptation collection. The system-dynamics surrogate, the privileged factor encoder, and the Arm Adaptation Module are not trained on Simscape data at all. They are trained on a 118-run collection made in IsaacLab through the deployment plant itself (the command ROM plus PhysX), driven by the DLS controller with dedicated PRBS excitation windows (Section 4.7). The reason is simple: those models exist to describe the plant the adaptive controller faces, and that plant is the IsaacLab one. No new Simscape

data was collected for this thesis, and no earlier adaptation or system-dynamics model was reused.

4.1 Dataset Objectives and Design Requirements

Before any run was collected I fixed four requirements that the data had to satisfy jointly. They are stated explicitly, because the rest of the chapter is essentially the record of discovering that no single collection strategy meets all four.

R1: realistic operating distribution. The learned models are deployed inside a tracking controller, so they must be accurate in the states that controller visits: coordinated multi-axis motion, gravity-loaded poses, moderate speeds. Data collected far from that distribution would be accurate where it does not matter.

R2: causal identifiability of the valve. A forward model is only useful to a policy if it responds to the command. This requires that the valve signal in the training data carry information that the state history does not already carry, which is a property of the *experiment*, not of the network [Ljung 99, Isermann 11].

R3: variation across a plant family. Robust training and, later, latent adaptation both need many plants rather than one. The variation has to be physically meaningful, it has to demonstrably change the measured response, and it has to be recorded per run, because the privileged latent of Section 5.6 is computed from those recorded factors.

R4: clean separation. Train, validation, and test material must be separable at the level of complete runs, so that no transition window straddles a split boundary and no run contributes to two splits.

R1 and R2 pull against each other, and that tension is the organising idea of this chapter. A controller that produces realistic motion computes its command from the measured state, which makes command and state history redundant and destroys identifiability (Section 2.4.2). An experiment that guarantees identifiability has to break the feedback path, which immediately makes the motion unrealistic. Rather than compromise on one of them, I collected both: a closed-loop campaign that satisfies R1 and R3, and an open-loop campaign that satisfies R2. Section 4.5 explains how the two are used together without letting either one dominate.

4.2 Randomized Parameter Space of the Simscape Campaign

This section documents the randomization design of the Simscape curriculum. Its status must be stated before its content, because the design was drawn up under an earlier plan in which randomized *Simscape* data would be the substrate for adaptation, and that plan is superseded. No model reported in this thesis is trained on this curriculum. It is retained for two reasons that survive the change: it is the provenance record of the closed-loop

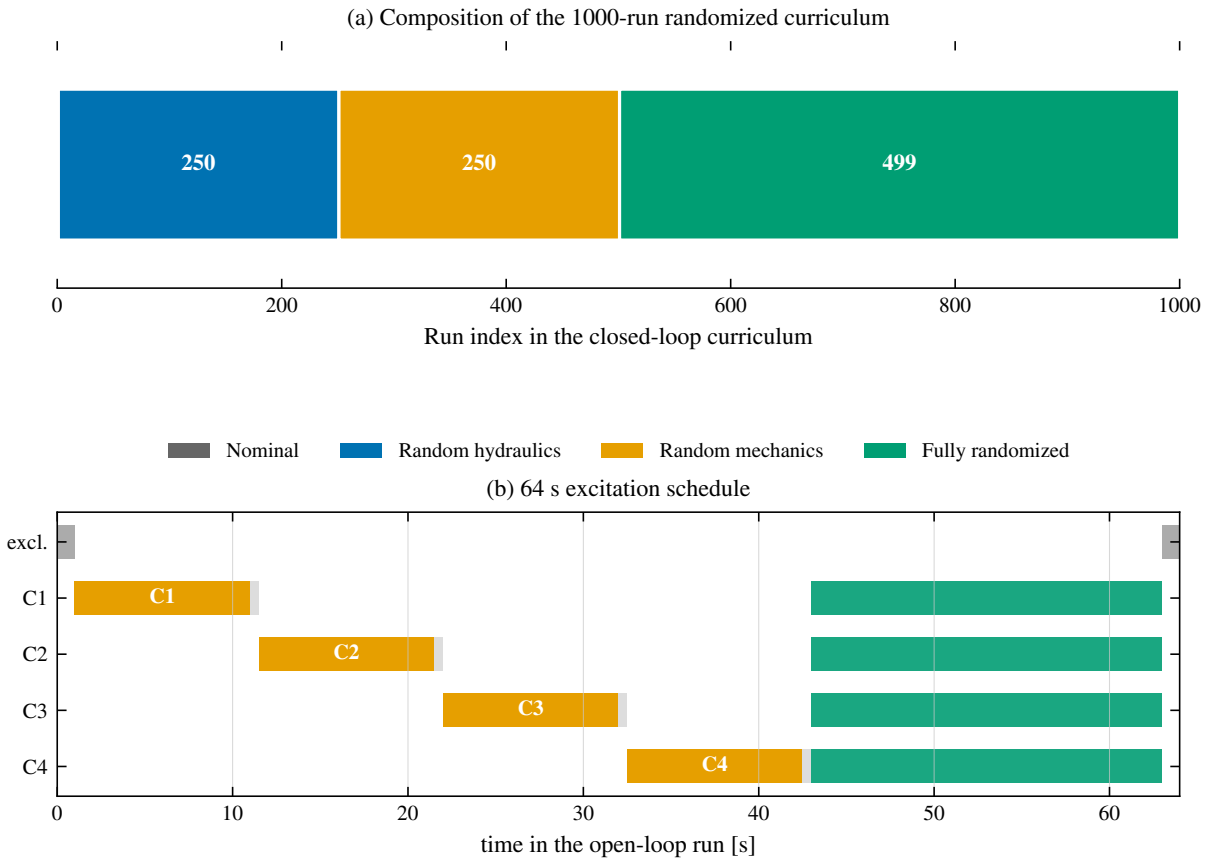

Figure 4.1: Design of the two collection campaigns. (a) Composition of the closed-loop curriculum: one nominal run, then 250 hydraulics-only, 250 mechanics-only and 499 fully randomized draws. (b) Timeline of one open-loop excitation run.

material the actuator models were identified on, and the compliance-equivalence analysis of Section 4.2.3 is what taught this project which quantity is randomizable in a hydraulic chamber model, a lesson carried directly into the factor space of Section 4.7. Readers interested only in the models that produce results may skip to Section 4.3.

4.2.1 Sampling Design

The closed-loop campaign draws its plants from a twelve-dimensional parameter space using Latin hypercube sampling [McKay 79], which spreads the draws more evenly over each marginal than independent uniform sampling would at this sample size. The twelve coordinates are fixed once for the whole batch, before any simulation runs, so the parameter table is reproducible and can be audited independently of the data it produced.

The batch is organised as a curriculum rather than a single homogeneous pool. Run 1 is the untouched nominal plant. Runs 2–251 randomize only the hydraulics, runs 252–501 only the mechanics, and runs 502–1000 both together, giving the composition shown in Figure 4.1(a). This structure is deliberate: it makes it possible to attribute a later modelling failure to the hydraulic or the mechanical axis by retraining on one block at a time, which a uniformly mixed batch would not allow.

4.2.2 Hydraulic Parameters

Table B.1 lists the hydraulic axes. They were chosen to correspond to the physical mechanisms that make one machine differ from another in service, in the sense discussed in Section 2.1.2: air works its way into the fluid, spools and seals wear, and relief settings drift.

The seal-wear coordinate deserves comment because it is the one place where a single sampled number is deliberately allowed to move several model parameters together. A worn cylinder does not become leakier without also becoming slower, so $w_{\mathrm{seal},i}$ scales the piston clearance linearly, the valve leakage area with the cube of the same factor, and the spool time constant from 8 ms to about 13 ms at the top of the range. Sampling those three independently would have produced physically incoherent machines, some of them very leaky and yet perfectly fast.

4.2.3 The Compressibility Axis

The entrained-air and dead-volume coordinates randomize the chamber dead volume, which spans 505 to 579 mL across the batch. This reaches the pressure dynamics through the continuity equation (3.4), which carries the chamber volume: a larger chamber is more compliant and slower, a smaller one stiffer and faster. The compressibility axis therefore enters the batch as capacitance variation.

4.2.4 Mechanical Parameters

The mechanical axes, listed in Table B.2, are applied to all fourteen rigid bodies of the mechanism: the six links, the four cylinder barrels, and the four pistons. For each body the mass is perturbed by up to $\pm 20\,\%$, the centre of mass by up to $\pm 5\,\%$ of its nominal offset, and the inertia tensor together with its products of inertia is perturbed consistently with the mass change, so that the body remains physically realisable rather than acquiring an inertia that its mass could not produce.

The joint stiffness and damping terms model compliance in the bearings and linkage rather than in the fluid. They matter here because they set how much of a pressure transient appears as boom oscillation instead of clean motion, which is the coupling that a purely kinematic controller cannot anticipate.

4.3 Closed-Loop Data Collection

The complete closed-loop procedure is summarized in Algorithm 4.1. Two features stand out before the prose expands on them: the parameter table is drawn in full before any simulation runs, so the plant family is fixed and reproducible independently of the collection, and the reference trajectory is resampled inside every run, so no two runs share a path.

4.3.1 The Controller as a Data-Collection Instrument

Every closed-loop run is driven by the same fixed integral-backstepping controller, operating on all five joints at a 0.005 s rate. Its gains are never retuned between runs. This is deliberate: the controller is here as a standardised instrument for putting the arm through

Algorithm 4.1: Closed-loop curriculum generation.

input : run budget $N = 1000$; joint limits $\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}$; nominal parameters $\boldsymbol{\theta}_0$
output : dataset $\mathcal{D} = \{(\text{trajectory}_i, \boldsymbol{\xi}_i)\}_{i=1}^{N}$

$\mathbf{L} \leftarrow \textsc{LatinHypercube}(N, 12)$ `// one design for the whole batch`
for $i \leftarrow 1$ **to** N **do**
  assign i to a curriculum block (nominal / hydraulic / mechanical / full)
  $\boldsymbol{\xi}_i \leftarrow \textsc{DrawParameters}(\mathbf{L}_i, \text{block})$ `//` $\varepsilon_{\text{air}}, \varepsilon_{\text{vol}}, c_d, w_{\text{seal}}, p_{\text{set}}$ `; masses, CoM, inertia, stiffness`
end
for $i \leftarrow 1$ **to** N **do**
  inject $\boldsymbol{\xi}_i$ into the Simscape model
  $n_w \leftarrow \textsc{RandInt}(12, 20)$; $\boldsymbol{W} \leftarrow \textsc{LatinHypercube}(n_w, \text{JDOF})$ within $[\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}]$
  assign each segment a speed class (slow / medium / fast / variable); **no** dwell
  $\boldsymbol{q}_{\text{ref}}(t) \leftarrow \textsc{Interpolate}(\boldsymbol{W})$ over 22 s
  simulate with `ode23t`; log on the 0.05 s grid
  store $(\boldsymbol{u}, \boldsymbol{x}, \dot{\boldsymbol{x}}, \Delta\boldsymbol{p}, \boldsymbol{F}, \boldsymbol{q}, \dot{\boldsymbol{q}}, \boldsymbol{q}_{\text{ref}}, \boldsymbol{\xi}_i)$
  if $i \bmod 100 = 0$ **then** checkpoint $\mathcal{D}$ to disk
end

realistic motion, not as a control contribution of this thesis, and holding it fixed is what makes the resulting behavioural differences attributable to the plant draw rather than to the controller.

Three details of the controller matter later. The second joint carries the gravity load of the whole boom, so it is handled by a proportional-integral law with velocity damping and an explicit gravity bias rather than by the backstepping law used on the others. A barrier term keeps the commanded configuration away from the joint limits imported from the URDF. And the commands are saturated per axis at [0.95, 0.95, 0.80, 0.80, 0.90]. That last vector matters: the first four entries are exactly the clip limits stored in the inverse-dynamics checkpoint in Section 5.2. The ID network inherited its output limits from the instrument that generated its training labels, which is correct but easy to mistake for a free design choice.

4.4 Open-Loop Excitation Experiments

4.4.1 Limitations of the Closed-Loop Batch

The first forward-dynamics model was trained on the closed-loop material and scored a one-step R^2 near 0.999. It was also, as Section 5.3.2 documents, almost completely insensitive to its own valve input: zeroing that input changed the long-horizon rollout error by a few percent. The model had learned to continue a trajectory, not to respond to a command.

This is not a training failure and no amount of retraining on the same material would have fixed it. In closed loop the controller computes the command from the state, so the command is very nearly a deterministic function of the state history that the network already sees [Ljung 99]. The information needed to separate cause from correlation is

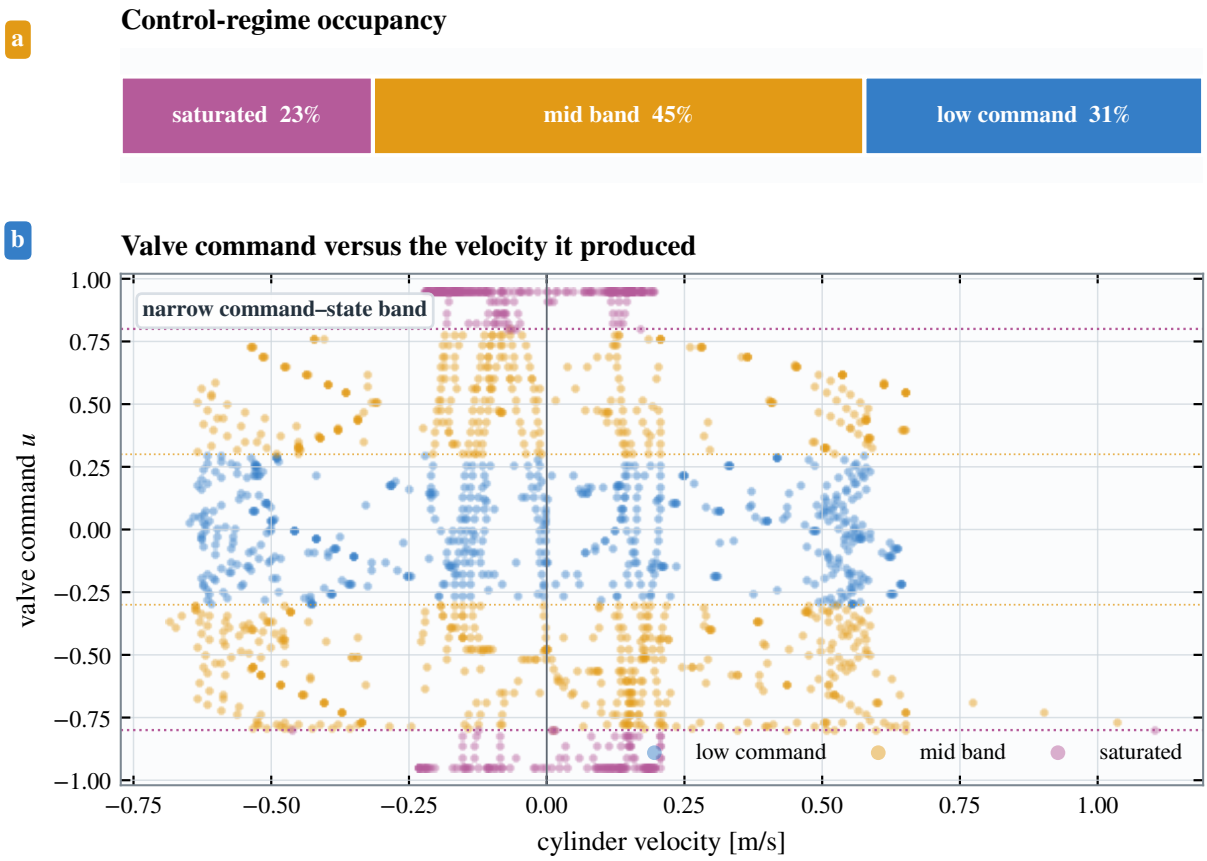

Figure 4.2: The closed-loop batch is diverse in control effort but not identifiable. (a) Fraction of time the valve spends saturated, in the mid band and in low-command creep. (b) The same run in the command–state plane, where the correlation that blocks identification is visible.

simply not present in the data. Recovering it requires an experiment in which the command is generated independently of the state, which means opening the loop.

This is not a coverage problem in disguise. Figure 4.2 shows that the closed-loop batch is in fact varied in its control effort: pooled over a run, the valve spends about a quarter of the time saturated, roughly half in the mid-authority band, and the rest in low-command creep, so all three control regimes are represented. The difficulty is visible in the same figure. Plotting the valve command against the cylinder velocity it produced shows the two riding together along a band rather than filling the plane: in a feedback loop a fast motion is when the controller commands a large valve opening, so command and state move as one. A model can read the command off the velocity, which is why it never has to learn what the command *causes*. Diverse control effort and identifiable control effort are not the same thing, and this batch has the first without the second.

4.4.2 Design of the Excitation Signal

The excitation run lasts 64 s and follows the fixed schedule in Figure 4.1(c). One second of startup is simulated and then discarded, because the plant is settling from its initial condition and those samples describe the initialisation rather than the dynamics. Four sequential slots follow, each giving one cylinder ten seconds of excitation followed by

a half-second zero-command gap. The gaps make the boundaries between single-axis segments unambiguous. A twenty-second phase then drives all four channels at once, and a final second of zero command closes the run. Commands are limited to ±0.90 of full authority.

The most consequential design decision here is one that runs against the textbook reflex. A persistently exciting input for a linear system is usually a rapidly switching signal, and the first version of this excitation was built that way. It was persistently exciting in the formal sense and still produced poor data, because the cylinders reversed before they had travelled anywhere: the commanded energy went into repeatedly accelerating and decelerating the piston near one point of the stroke instead of moving it through the workspace. The present signal replaces fast switching with long full-authority plateaus during the sequential phase and a slow, high-authority component during the simultaneous phase, so that each cylinder traverses a substantial part of its stroke while its valve is open. The simultaneous phase additionally superimposes one slow near-square cycle across the full twenty seconds with a distinct medium-frequency component on each channel, and short raised-cosine ramps at entry and exit keep the transitions differentiable.

The lesson generalises, and it is one this thesis would offer to anyone identifying a hydraulic actuator: persistent excitation is a condition on the input signal, but the quantity that decides whether a *cylinder* is identifiable is how much of the stroke the input covers. The two are not the same, and optimising the first while ignoring the second produces data that passes the classical test and still cannot support a model.

4.4.3 Plant Draws and Acceptance

The same excitation is applied to four plant draws taken from the same parameter space as the closed-loop campaign: nominal, randomized hydraulics, randomized mechanics, and fully randomized. Holding the input fixed and varying only the plant is what makes the four runs directly comparable, since any difference in the response is attributable to the parameters.

Each run is checked for stroke coverage before it is accepted. The collector requires at least 10 % of the available stroke on cylinders 1–3 and 8 % on cylinder 4, warns below a target of 30 %, and records the achieved coverage per channel in the run summary. The runs retained here cover between 12 % and 33 % of the stroke depending on the cylinder, which is above the acceptance floor but below the target. Cylinder 4 is the weakest, and improving its coverage is the clearest available next step for this campaign.

One earlier excitation export was rejected outright rather than repaired. Half of its runs turned out to contain duplicated state trajectories, so the nominally distinct plant draws were not distinct in the data, and the valve was again not causally identifiable. That export and its reports are not part of the maintained workflow. It is mentioned here because it is the reason the acceptance checks in Section 4.10 exist in their present form: they were written to make that particular failure impossible to miss a second time.

4.5 Combining Closed-Loop and Open-Loop Datasets

The two campaigns are complementary, and each learned model of Chapter 5 draws on them in a different proportion.

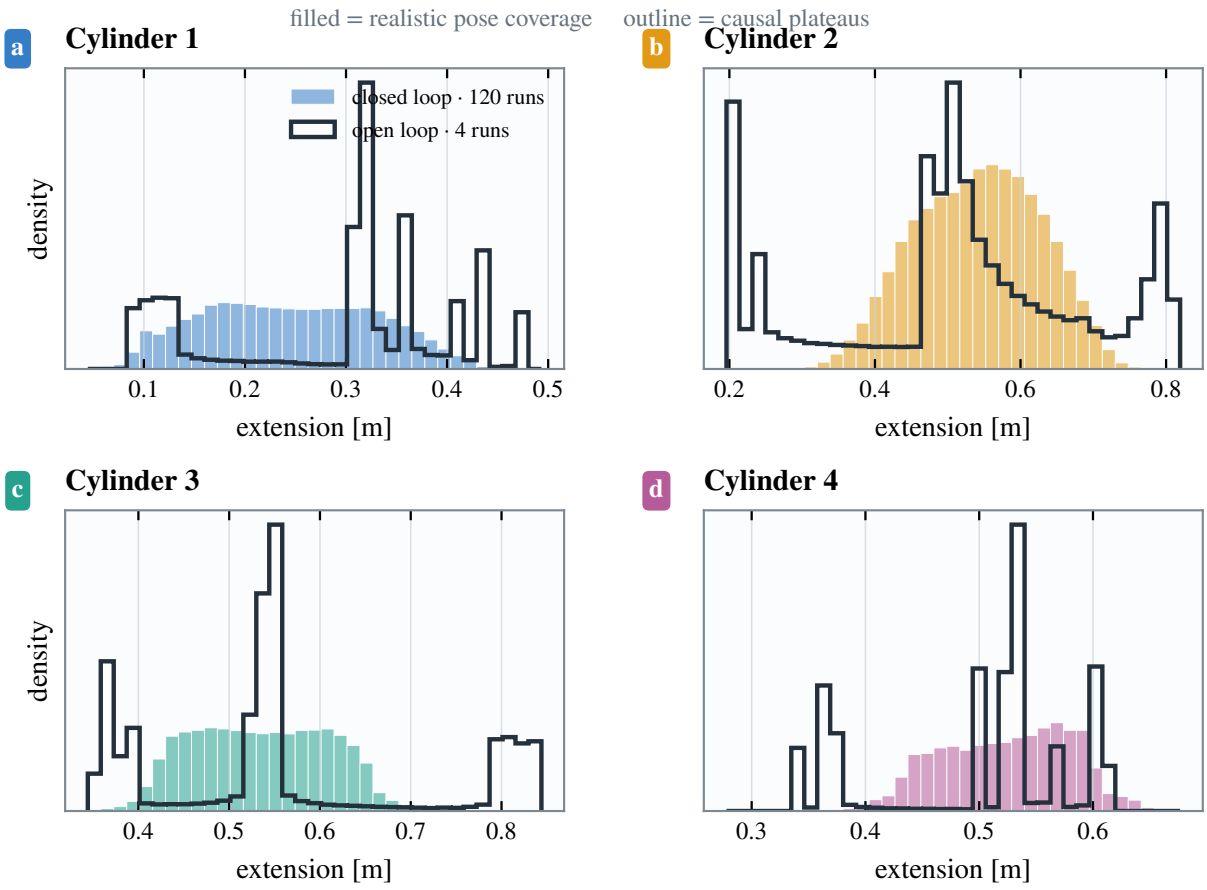

Figure 4.3: Extension coverage of the two campaigns, per cylinder and pooled over runs (closed-loop filled, open-loop outline). The closed-loop data fills the reachable range smoothly; the open-loop data concentrates into the plateaus where the valve is identifiable.

The inverse-dynamics model is trained entirely on closed-loop material, because its target is the command a competent controller issued in a given state. The forward-dynamics model begins there and is then retrained on the strict open-loop export, with the closed-loop data retained as a replay term at weight 0.25. Ablation showed that dropping the replay term costs state coverage and extension accuracy, because four excitation runs cannot span the poses a thousand operating runs visit.

Figure 4.3 shows why. The closed-loop distribution fills the reachable range smoothly; the open-loop distribution concentrates into narrow plateaus. The open-loop bands are where the valve is identifiable, the closed-loop mass is where the model must be accurate at deployment, and neither campaign covers both. The forward model is therefore trained on their union.

The reduced-order model consumes the closed-loop material in a joined form. The inverse and forward exports are aligned on exact run identifiers, time indices, and sample times, and the join is accepted only when the sixteen-dimensional cylinder state agrees between them sample for sample. The resulting command-conditioned dataset contains 452 400 transitions at 0.05 s drawn from 1 040 aligned runs. The logged valve command is carried through this join, but only as a supervision target for the model's internal valve head. It is never a runtime input, which is the property that lets the ROM be driven by desired

cylinder motion alone (Section 5.4).

The thousand-run randomized curriculum described in Section 4.3 occupies a different position in the argument, and I want to be exact about it because it is easy to overstate. The inverse, forward, and reduced-order models reported in Chapter 5 were trained on the earlier aligned export, which is a nominal-plant collection. The randomized curriculum was collected under the earlier plan, in which randomized Simscape runs would supply both the robust actuator variants and the privileged factor vectors for latent adaptation. Neither role survives: the adaptation evidence chain of this thesis is built entirely on the IsaacLab collection of Section 4.7, for the reason given there, and no result in this thesis is trained on or evaluated against the randomized Simscape curriculum. It is reported as provenance and as a negative methodological record, not as a substrate whose consequences appear later.

4.6 Closed-Loop Calibration Data for the Command ROM

The reduced-order model is the one actuator model that does not stop at the Simscape boundary. It is accurate about hydraulics, but the plant it drives at runtime is a PhysX articulation with its own contact behaviour and compliance, and the systematic offset between the extension the ROM commits to and the one PhysX realizes is invisible to any amount of further Simscape training.

The calibration set is therefore collected from the deployment loop itself. A previous ROM checkpoint is rolled out through the parked workflow of Section 3.6.3 on its five reference shapes, logged on the ROM's own 0.05 s grid, giving 4 084 transitions. Each record stores the six-step ROM history as the runtime assembles it, together with the measured PhysX transition. The targets are what the plant did, not what Simscape said it would do.

Two properties make the set safe to use. Only the bounded command-gated correction head is trainable against it, with every other parameter frozen and the correction capped at 3 mm, so the identified physics is preserved and only a deployment offset is added. And fine-tuning runs against a mixture, replaying the original dataset at weight 0.25, because five parked shapes cannot span the poses a thousand Simscape runs visit. The correction is command-gated and therefore exactly zero at zero command, so the stationary hold contract survives unchanged.

One limitation is carried forward. The calibration data comes from the *parked* diagnostic workflow rather than the moving-verge task, so it fits the actuator to PhysX under free-space tracking and not under cutter–ground contact. The calibration is therefore weakest in the regime the verge task spends most of its time in, and the bounded correction head keeps that gap contained rather than closing it. This calibration is also what qualifies the ROM as the deployment actuator, with the consequences for the three-way comparison set out in Section 3.4.1.

4.7 The IsaacLab Adaptation Collection

4.7.1 Provenance of the Adaptation Data

The models that carry the adaptation argument (the system-dynamics surrogate, the privileged factor encoder, and the AAM) describe the plant that the adaptive controller is closed around. That plant is not the Simscape model. It is the command ROM plus PhysX, the same four-dimensional cylinder command port used at deployment. A surrogate identified on Simscape data would be a model of a model, and the latent it defines would describe variation the deployment plant does not necessarily exhibit. The adaptation dataset is therefore collected *in* the deployment plant, and the frozen ROM, ID and FD checkpoints enter this chain only as fixed components, never as things to be retrained.

4.7.2 The Physical Factor Vector

The randomized quantity is a six-dimensional physical factor vector

$$\mathbf{e} = \Big[e_{\text{flow}},\ e_{\text{compliance}},\ e_{\text{seal}},\ e_{\text{pressure}},\ e_{\text{mass}},\ e_{\text{COM}_z}\Big], \tag{4.1}$$

with the ranges of Table B.3. Four coordinates are hydraulic and two mechanical, which is what makes the hydraulic-only, mechanical-only, and combined domains of Section 4.7.3 separable. Each candidate was admitted only after a one-factor sensitivity screen confirmed that moving it alone produces a measurable change in cylinder extension, velocity, pressure, or end-effector motion. Parameters whose effect could not be measured, or which could not be varied safely, were rejected rather than carried as decoration.

One omission is deliberate. Actuation delay is not a randomized coordinate. Delay in this system is an *emergent* property of flow, compliance, and load rather than a free parameter, so it is measured as a response property of each run (Chapter 7) instead of being supplied to any model as a known factor. Handing an estimator a factor that the real machine would never publish is the same mistake as handing it the true latent.

4.7.3 Domains and Factor Design

Runs are organized into five domains (nominal, hydraulic-only, mechanical-only, combined-training, and an *unseen* combined-test domain), and one factor vector is fixed for a complete run, so that no run mixes plants. Factor configurations are drawn with deterministic scrambled Sobol sequences [Sobol' 67] rather than independent uniform sampling, with explicit boundary anchors so that the corners of the space are represented and not merely approached, and with fixed per-domain seeds so the design is reproducible independently of the data it produced. Combined draws that compound into an excessive response gain are rejected at draw time rather than being collected and discarded later.

The resulting design is auditable: 98.3 % of the factor keys are unique, the active-factor matrix is full rank, every factor quartile is populated, and the factor keys of the training, validation, and test splits are disjoint. That last property is the one that matters most for the claims of Chapter 7: a test run does not merely come from a different episode, it comes from a plant the models have never seen.

4.7.4 Collection Procedure

Every run is collected through the same interface the policy uses. The DLS controller closes the loop on measured cylinder state and issues its command through the four-dimensional action port into the command ROM and PhysX. No joint-target bypass is used, and no PPO policy is used to collect identification data, because a policy trained on the same plant would reintroduce the closed-loop correlation that Section 4.4 spent a campaign removing.

Independent excitation is supplied by dedicated four-channel PRBS windows applied through that same action port, with a small amplitude and a hold of ten control steps, so that the excitation is causally independent of the state without ever leaving the safe envelope. Probe actions are logged separately from controller output, so any analysis can distinguish the two. Each run follows a fixed eleven-segment structure (settle, three postures, three PRBS windows, DLS task motion, reach-and-return, and stow), giving 11 304 control steps and 188.4 s of simulated time per run. Three postures plus line, coupled-sine, and reach-and-return motion were verified in a pilot to give full rank over all four action channels under varied geometry and load.

4.7.5 The Frozen Dataset

The final collection contains 118 complete runs: 2 nominal, 40 hydraulic-only, 12 mechanical-only, 48 combined-training, and 16 unseen combined-test. Collection ran in four vectorized IsaacLab environments with per-environment factors and logging. Slots that tripped a safety or validity check were discarded and retried individually rather than being silently kept. All 118 runs passed the final collection gate, which checks finite physics, exact model-grid alignment, command rank, factor-response authority, state coverage, and saturation limits.

The dataset is then *frozen*: the raw runs are checksummed and the run-level split is fixed at 82 training, 20 validation, and 16 reserved combined-test runs, with zero factor-key leakage between them. The reserved sixteen are not touched again: not for history-length selection, not for model selection, and not for hyperparameter choices, so that the final adaptation claim is measured on material no decision in this thesis was conditioned on.

4.7.6 Learning Timebase and Hydraulic State

The learning timebase is the actuator model grid, not the control clock. Samples are exact 50 ms ROM transitions taken at control indices $0, 3, 6, \ldots$: state row t and command row t predict state row $t + 3$. Nothing is interpolated. This is the same alignment rule as Section 3.5.1, and it is enforced rather than assumed, because a model trained across a mismatched interval is wrong by a fixed factor in the quantity it exists to supply.

The hydraulic state used by the system-dynamics models is

$$\boldsymbol{s}_t = \left[\boldsymbol{x}_t^{1:4},\ \dot{\boldsymbol{x}}_t^{1:4},\ \Delta\boldsymbol{p}_t^{1:4}\right] \in \mathbb{R}^{12}, \tag{4.2}$$

with joint position and velocity carried as *context* only, and the prediction target the next 50 ms state delta. Differential pressure occupies an awkward position in this design: it is recorded, and it is a prediction target and a diagnostic, but it is withheld from the

final deployment-mirror input. A real machine does not publish chamber pressure on this interface, so a model allowed to read it would be validated under an assumption deployment cannot honour. Withholding it is what makes a causal history *belief* necessary rather than optional, and that necessity is the whole reason the AAM has work to do.

4.8 Feature and Target Construction

4.8.1 The Actuator Models

The three learned actuator models take cylinder-space features, in the ordering and notation fixed in Section 5.1. Their input contracts differ because their questions differ. The inverse model receives the measured cylinder state, the logged rod forces, the desired increments, and the previously applied command, twenty-eight numbers in all. The forward model receives the applied valve command and the measured state, twenty numbers. The reduced-order model receives a six-sample history of state, previously predicted valve, and desired command, twenty-four numbers per step.

Three conventions apply throughout, and they are the same ones the models enforce at inference time.

Increment Targets

Every dynamic target is a within-run one-step increment, $\Delta \boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never an absolute state. This removes the large static component of the signals and, more importantly, makes the trivial persistence solution an explicit baseline that a model can be measured against rather than a hiding place it can settle into.

Causal Alignment

Features at step k contain only quantities available at step k. The extraction stage verifies this by recomputing every logged target from the within-run difference of the states and requiring exact agreement, a check reported in Section 4.10.

Time-Base Conversion

The open-loop campaign is logged at 0.01 s but the models run at 0.05 s, so five consecutive raw samples are combined into a single transition. The valve input attributed to that transition is the command held at the start of the window, which is the causally correct choice. The mean command over the window is also recorded, but only for analysis. Cylinder extension velocity is not logged directly in the closed-loop archive and is obtained by differentiating extension on the output grid, which is consistent with the trapezoidal identity that the strict data satisfies to $R^2 = 0.994$–0.9997 (Equation 5.10).

4.8.2 Observable History of the Adaptation Models

The adaptation models draw their features from the frozen IsaacLab collection, and here the interesting question is not what goes in but what is kept out. A history window carries only quantities a deployed machine could produce: the applied action, cylinder extension and velocity, and joint position and velocity. Six classes of signal are excluded by construction, and the exclusion is enforced by an explicit input mask rather than by convention:

- the physical factors $\mathbf{e}$ and the true latent $\boldsymbol{z}$, because recovering them is the estimator's job;
- any future observation, because the estimator is causal;
- the hidden internal state of the command ROM, because the deployment interface does not expose it;
- the differential pressure $\Delta\boldsymbol{p}$, for the reason given in Section 4.7;
- the desired task reference, so that the latent describes the plant rather than the trajectory being asked of it.

Windows are contained within a single run and never cross a run boundary, and each window carries the stopped-gradient target $\boldsymbol{z} = \mu(\mathbf{e})$ of its own run together with the run identifier, domain, split, and valid history bounds. Applied to the 118 frozen runs this yields 444 624 causal history ticks. What is built from them, how the history length is chosen, and how the result is validated downstream belong to Chapter 5.

4.9 Preprocessing and Data Splits

Splitting is done at the level of complete runs, never of individual samples. A random sample-level split would place transitions from the same trajectory on both sides of the boundary and report a validation score that mostly measures interpolation within a run. For the command-conditioned dataset the 1 040 runs are divided into 936 training, 52 validation, and 52 test runs, giving 407 160, 22 620, and 22 620 transitions respectively, with no run identifier appearing in more than one split. The IsaacLab adaptation collection is split by run *and* by factor configuration (82/20/16, Section 4.7), which is the stronger condition: separating runs alone would still let two runs of the same plant fall on opposite sides of the boundary and turn a claim about unseen plants into a claim about unseen episodes.

The strict open-loop export needs a different treatment because it contains only four runs, and holding one out would remove an entire plant draw from training. Instead each run is divided into twenty temporal blocks, every fifth block is assigned to validation, and a five-sample guard interval is discarded at each block boundary so that no 0.05 s transition window spans the split. A simple last-twenty-percent split was tried first and rejected: it changed the spread of the targets between training and validation by a factor of 1.3 to 2.3, because the final fifth of the run is the simultaneous excitation phase and is dynamically unlike everything before it. That split would have measured extrapolation to a different regime and called it generalisation.

Normalization statistics are computed on the training split only and stored inside each checkpoint, so that a model always normalizes with the statistics it was fitted with. Mixing statistics between checkpoints silently invalidates any comparison, and the loader enforces the stored contract rather than recomputing it.

Finally, samples are carried with an explicit validity flag rather than being silently dropped. In the strict export 93.7 % of samples are marked valid. The remainder are the excluded startup and shutdown seconds and the zero-command gaps. Keeping them in the archive with a flag, instead of deleting them, means the time base stays contiguous and the within-run difference checks remain meaningful.

4.10 Dataset Diagnostics

No dataset in this thesis is used before it passes an explicit audit, for the reason given at the end of Section 4.4. The audit of the strict open-loop export is reproduced in Table B.4.

The two rank figures are the ones that speak directly to requirement R2. The instantaneous command matrix has rank four, so no channel is a linear combination of the others, and the lagged regressor matrix built from the command history has rank eighty, so the excitation is persistently exciting to the order the models need. The condition number of that lagged matrix is about 4.6×10^3, which is finite and workable rather than comfortable. It is a consequence of using long plateaus, and it is the price paid for the stroke coverage those plateaus buy.

The exactness of the three increment checks deserves a word, since a maximum error of exactly zero invites suspicion. It is expected here: the targets are computed from the logged states by the same within-run difference that the check recomputes, so the check verifies that the extraction stage did what it claims, not that the simulation is accurate. It catches misalignment, off-by-one indexing, and rows that have been reordered or concatenated across run boundaries, which are the failure modes that occur. Physical fidelity is a separate question, addressed by the validation study in Section 3.4.4.

Three limitations of the Simscape material are carried forward honestly. Two limitations are carried forward. Stroke coverage reaches only 12 % on the head-pivot cylinder, which is also the cylinder the strict forward model predicts least well. The rod-force channel fails a piston-area consistency check on this export, so it carries zero supervised weight in the reduced-order model and no result rests on it.

The IsaacLab collection is audited against different failure modes. Design adequacy is checked before use (unique factor keys, full active-factor rank, disjoint keys across splits), physical validity per run (finite physics, exact model-grid alignment, full command rank, bounded saturation), and leakage structurally, with privileged channels removed by an explicit input mask rather than by trusting the feature list. Its limitation is scale: 118 runs over six factors is a sparse design, which is why the adaptation claims of Chapter 7 are reported with paired bootstrap intervals and per-run detail rather than pooled point estimates.

With the data described and audited, the next chapter turns to what was learned from it: the inverse and forward models that form the PPO–ID–FD actuator path, the command-conditioned reduced-order model that replaces them in PPO–ROM, and the system-dynamics, encoder, and adaptation models built on the IsaacLab collection.

5. Learned Models and Online Adaptation

The previous chapter established how the Simscape datasets were generated and why both closed-loop operation and open-loop valve excitation were required. This chapter develops the learned actuator representations that consume those datasets: the inverse-dynamics (ID) and forward-dynamics (FD) networks that form the PPO–ID–FD actuator path, and the command-conditioned reduced-order model (ROM) that replaces them in PPO–ROM. It then develops the system-dynamics surrogate, the privileged factor encoder, and the Arm Adaptation Module that together carry the adaptation argument, and is careful throughout to keep the surrogate distinct from the deployed actuator: the system-dynamics model is an instrument for establishing that a compact dynamics latent exists and is useful, not a component of any control stack. Each model is presented with its exact input–output contract, architecture, training objective, and offline validation evidence, because a recurring result of this work is that standard supervised scores alone were not sufficient to accept any of these models: a forward model can achieve near-perfect one-step R^2 while having learned almost nothing about valve authority (Section 5.3). All models are trained on cylinder-space signals only. The mapping between cylinder space and task space is handled by the kinematic layers of Chapter 3 and the control integration is described in Chapter 6.

5.1 Modeling Overview and Common Notation

5.1.1 Cylinder-Space Signals

All learned models in this thesis operate on the four hydraulic cylinders of the MULAG arm, indexed $i \in \{1, \ldots, 4\}$ in the fixed order slew cylinder (*Drehzapfen*), boom cylinder I (*Ausleger I*), boom cylinder II (*Ausleger II*), and head pivot cylinder (*Messerkopf-Schwenk*). The passive fifth joint is not actuated and does not appear in any model input. At the learned-model sample time $T = 0.05\,\mathrm{s}$, the discrete time index k refers to the cylinder-space state

$$\boldsymbol{x}_k \in \mathbb{R}^4, \qquad \dot{\boldsymbol{x}}_k \in \mathbb{R}^4, \qquad \Delta \boldsymbol{p}_k \in \mathbb{R}^4, \qquad \boldsymbol{F}_k \in \mathbb{R}^4, \tag{5.1}$$

Table 5.1: Cylinder-space notation used by all learned models.

Symbol	Dimension	Meaning
T	–	learned-model sample time, 0.05 s
$\boldsymbol{x}_k$, $\dot{\boldsymbol{x}}_k$	$\mathbb{R}^4$	cylinder extension [m] and extension velocity [m/s]
$\Delta\boldsymbol{p}_k$	$\mathbb{R}^4$	chamber pressure difference [Pa]
$\boldsymbol{F}_k$	$\mathbb{R}^4$	logged rod force [N]
$\boldsymbol{u}_k$	$[-1,1]^4$	normalized proportional-valve command
$\boldsymbol{c}_k = [\Delta\boldsymbol{x}_k^*, \Delta\dot{\boldsymbol{x}}_k^*]$	$\mathbb{R}^8$	desired extension and velocity increments
$\Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k$	–	within-run one-step increment
$\hat{(\cdot)}$	–	model prediction
$\boldsymbol{h}_k$	$\mathbb{R}^{128}$	recurrent latent state of the ROM encoder

where $\boldsymbol{x}$ collects the cylinder extensions in meters, $\dot{\boldsymbol{x}}$ the extension velocities, $\Delta\boldsymbol{p}$ the chamber pressure differences, and $\boldsymbol{F}$ the logged rod forces. The valve command vector $\boldsymbol{u}_k \in [-1,1]^4$ is the normalized spool command of the four proportional valves. Desired quantities carry a star; the desired cylinder command produced by the task controller is the pair

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8, \tag{5.2}$$

i.e. the extension increment and the extension-velocity increment that the controller wants to realize over the next sample. Predictions of a model carry a hat. Table 5.1 summarizes the notation used throughout this chapter.

5.1.2 Two Actuator Representations

Figure 5.1 shows the two learned actuator paths side by side. In the PPO–ID–FD path, actuation is represented by two separate networks: the ID network maps the current hydraulic state and a desired increment to a valve command, and the FD network maps a valve command and the current state to the predicted state increment. Their composition $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ predicts what a commanded increment will actually do to the cylinders. In the PPO–ROM path, one command-conditioned ROM replaces this composition: it receives the same desired command $\boldsymbol{c}_k$, predicts the valve internally, and propagates pressure, velocity, and extension through an explicit, physically structured update (Section 5.4). Both paths terminate in the same bounded PhysX cylinder position target inside the IsaacLab environment [Mittal 23], and both are driven by the same policy under the same action contract [Schulman 17], distilled from the same damped-least-squares teacher, as described in Chapter 6. The experimental variable between PPO–ID–FD and PPO–ROM is therefore the actuator representation alone.

5.1.3 Common Conventions

Three conventions apply to every model in this chapter.

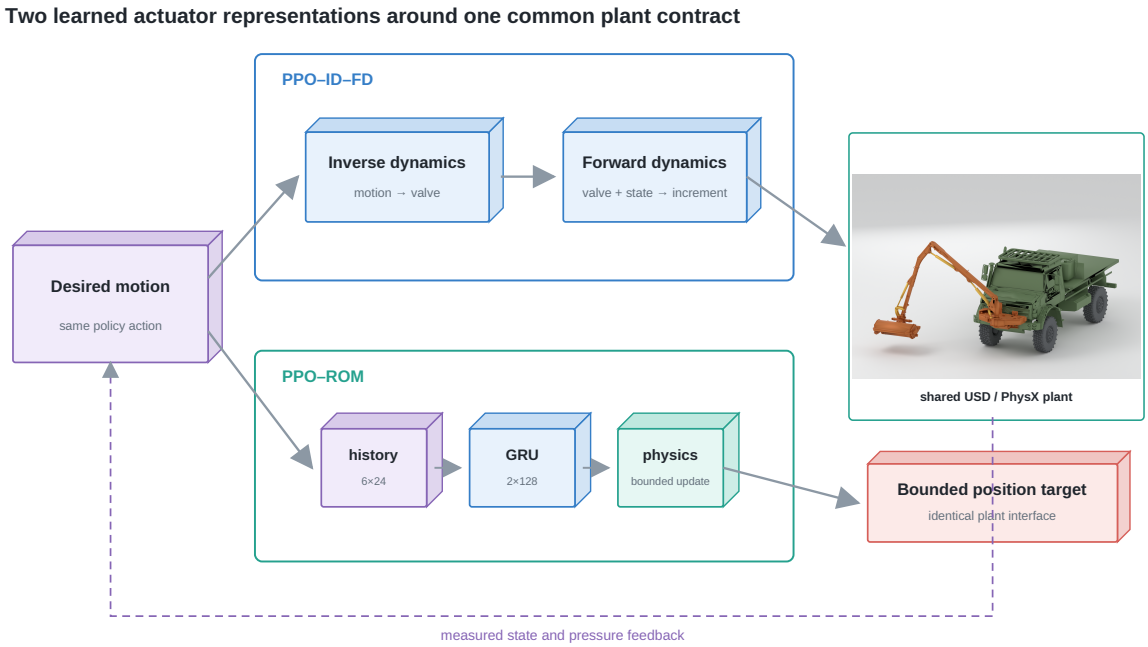

Figure 5.1: The two learned actuator representations as parallel lanes around one shared contract. Both consume the same desired cylinder command and the same measured cylinder state from the feedback bus, and both produce the predicted increment that becomes the bounded PhysX target. The PPO–ID–FD lane composes two separately trained networks. The PPO–ROM lane realizes the command inside a single grey-box model whose valve signal is internal. Training-time data flow is described in Chapter 4.

Increment Targets

All dynamic targets are within-run one-step increments, $\Delta \boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never absolute states. This removes the dominant static component of the signals and makes the persistence solution ($\Delta \hat{\boldsymbol{x}} = \mathbf{0}$) an explicit, testable baseline rather than a hidden local minimum.

Per-Checkpoint Normalization

Every checkpoint stores the mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$ of its own training inputs and targets, and inference normalizes each feature as

$$\tilde{\xi} = \frac{\xi - \mu_\xi}{\sigma_\xi}. \tag{5.3}$$

Statistics are never shared between checkpoints. Mixing the statistics of two checkpoints silently invalidates any comparison between them, so the loader enforces the stored contract.

Causal Timing

Model inputs at step k contain only quantities available at step k (including the previously applied command), and targets are the measured transition to step $k+1$. During dataset construction this alignment is checked explicitly against the within-run differences of the logged states (Chapter 4).

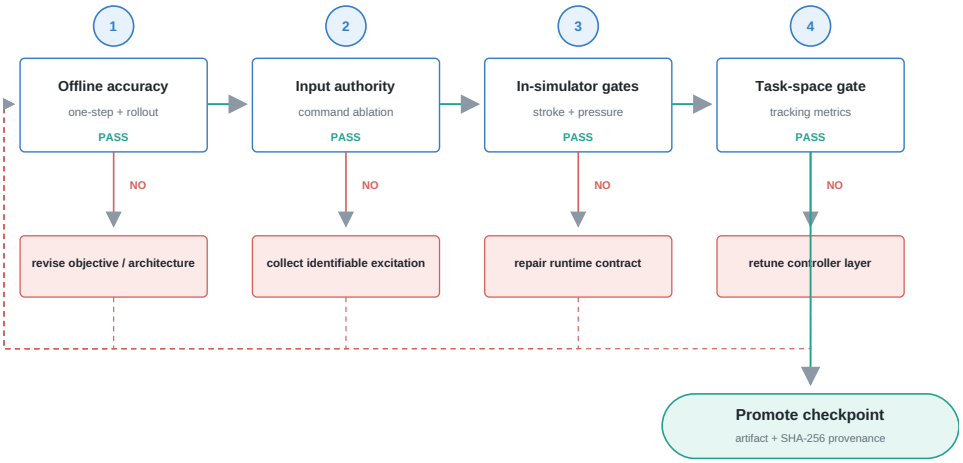

Figure 5.2: The acceptance ladder that every learned model in this thesis must climb before deployment. A candidate that fails a gate exits right into the corresponding repair loop and re-enters from the top; only a candidate that passes all four gates is promoted, and the promoted artifact is recorded together with its SHA-256 hash.

Gate-Based Acceptance

Finally, no model in this thesis is promoted on a supervised score. Every candidate passes the fixed gate sequence of Figure 5.2: offline accuracy, input-ablation authority, in-simulator cylinder gates, and the task-space tracking gate, in that order. Each "no" exit names the repair that experience showed to be the right one, and the promoted checkpoints are recorded with their hashes for provenance. The sections that follow report, for each model, where in this ladder it initially failed and what fixed it.

5.1.4 Data Provenance

Two dataset families from Chapter 4 are consumed here. The *closed-loop* family consists of the aligned Simscape exports [The MathWorks, Inc. 24] used for supervised ID and FD training and, after exact run-and-timestamp alignment, for the command-ROM dataset: 452 400 transitions at $T = 0.05\,\mathrm{s}$ from 1 040 runs, split by complete runs into 936 training, 52 validation, and 52 test runs (407 160 / 22 620 / 22 620 samples). The *open-loop* family is the strict valve-excitation export used to repair the FD model: four 64-s runs sampled at 0.01 s, in which the same rank-4 multi-input open-loop excitation is applied to four distinct plant draws (nominal, randomized hydraulics, randomized mechanics, both). Its properties and the reasons it was necessary are analyzed in Section 5.3.

5.2 Inverse Dynamics Model

5.2.1 Input–Output Contract

The ID network answers the question the low-level controller asks: *given the current hydraulic state, which valve command realizes a desired cylinder increment?* Its contract is

$$\Delta \boldsymbol{u}_k = f_{\mathrm{ID}}\left(\boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta \boldsymbol{p}_k,\, \boldsymbol{F}_k,\, \Delta \boldsymbol{x}_k^*,\, \Delta \dot{\boldsymbol{x}}_k^*,\, \boldsymbol{u}_{k-1}\right) \in \mathbb{R}^4, \tag{5.4}$$

a 28-dimensional input mapped to a valve-command *change*. The deployed command is the accumulated

$$\boldsymbol{u}_k = \mathrm{clip}\Big(\boldsymbol{u}_{k-1} + \Delta\boldsymbol{u}_k,\ -\boldsymbol{u}_{\max},\ \boldsymbol{u}_{\max}\Big), \qquad \boldsymbol{u}_{\max} = [0.95,\ 0.95,\ 0.8,\ 0.8], \tag{5.5}$$

where the per-axis limits are stored in the checkpoint. Predicting the command change rather than the absolute command conditions the problem on the previous operating point of the valve, which matters because proportional valves are direction- and history-dependent around their dead zone (Section 2.1.2). The desired increments $\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*$ in the training data are the *achieved* transitions of the logged Simscape runs, used as a proxy for desired motion because no independent cylinder-space reference was logged. The consequences of this proxy are examined below.

5.2.2 Architecture and Supervised Training

f_{ID} is a multilayer perceptron with an input projection to width 512 followed by four residual blocks [He 16] with SiLU activations [Elfwing 18], layer normalization [Ba 16], and dropout of 0.02 [Srivastava 14], implemented in PyTorch [Paszke 19]. Figure 5.3(a) shows the trunk, which the FD network shares, and Figure 5.3(b) the recurrent encoder that the ROM uses instead (Section 5.4).

Training minimized the smooth-L_1 (Huber) loss [Huber 64] with transition point $\beta = 0.5$ on normalized targets using Adam [Kingma 15] with learning rate 10^{-3}, weight decay 10^{-4}, batch size 8 192, gradient-norm clipping at 10, and early stopping with patience 45. Samples in valve saturation were up-weighted by a factor 1.25 so that the rare saturated regime is not ignored. The promoted checkpoint (seed 42) was selected at epoch 120 of at most 300 with a validation loss of 0.0336.

The supervised result was, on its face, excellent: per-valve validation R^2 of 0.9996, 0.9973, 0.9981, and 0.9974. It must be stated clearly what this number does and does not mean. It means the network reproduces the logged valve commands from the logged states and achieved transitions. It does not mean the composite map $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ has authority over the plant: since the achieved transitions in closed-loop data are themselves produced by the logged commands, a model can score highly by exploiting this correlation without ever being asked to *cause* a transition that was not already in the data (Section 2.4.2).

5.2.3 Paired Control Fine-Tuning

This deficiency became measurable when the ID and FD networks were composed. For control, the relevant object is the *paired differential response*: the difference between what the FD predicts under the ID's command for the actual target and under the ID's command for a zero target,

$$\begin{aligned} \boldsymbol{u}_k^{\mathrm{tgt}} &= f_{\mathrm{ID}}(\cdot,\ \boldsymbol{c}_k,\ \boldsymbol{u}_{k-1}), \qquad \boldsymbol{u}_k^{0} = f_{\mathrm{ID}}(\cdot,\ \mathbf{0},\ \boldsymbol{u}_{k-1}), \\ \Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}} &= f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{\mathrm{tgt}}, \cdot\Big) - f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{0}, \cdot\Big). \end{aligned} \tag{5.6}$$

If the pipeline is to be useful, $\Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}}$ must track the requested $\Delta\boldsymbol{x}_k^*$. Before fine-tuning it did not: the normalized control error of the composite was 1.00, exactly the score of

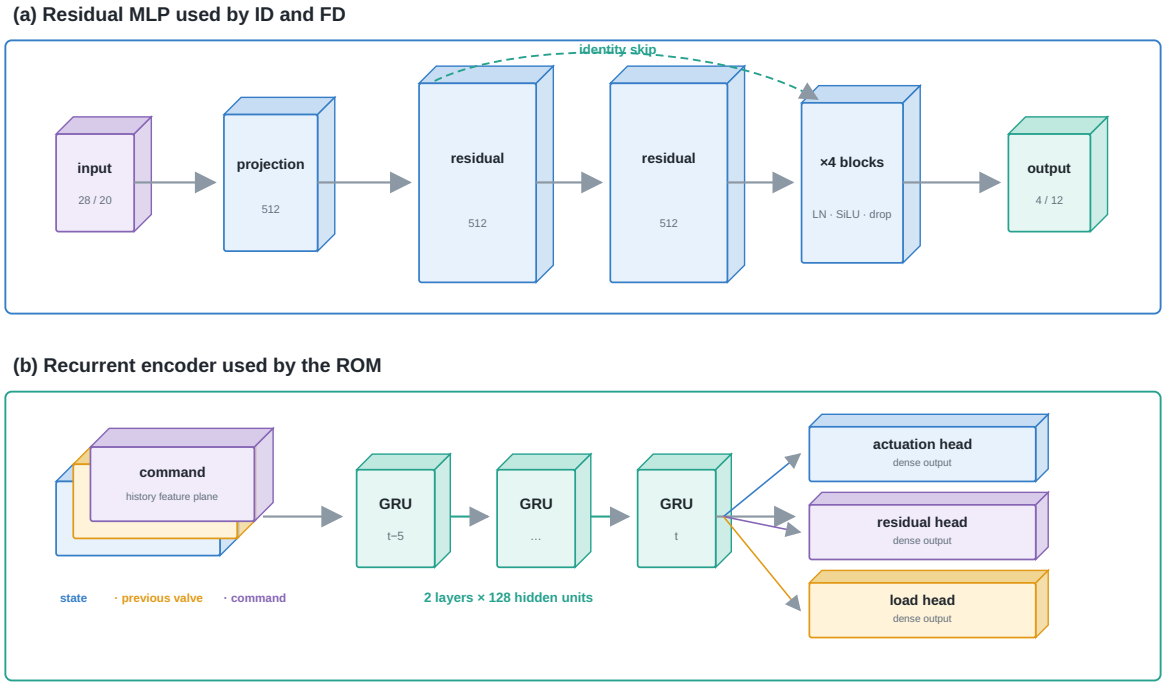

Figure 5.3: The two network families used by the learned actuator models. (a) The residual MLP trunk shared by the ID and FD networks: an input projection to width 512, four pre-activation residual blocks (Linear–LayerNorm–SiLU–Dropout–Linear with an identity skip), and a linear output head. Only the input and output widths differ between ID and FD. (b) The ROM's recurrent encoder, unrolled over its six history steps: each step consumes the 24-D feature vector, the hidden state carries pressure and command transients across time, and the final latent $\boldsymbol{h}_k$ feeds the actuation, residual, and load heads.

predicting nothing, with per-axis regression slopes of at most 0.004 (Table 5.2). Both networks were therefore fine-tuned jointly on the control objective

$$\mathcal{L}_{\text{ctl}} = \left\| \Delta\hat{\boldsymbol{x}}^{\text{pair}} - \Delta\boldsymbol{x}^{*} \right\|_{\text{norm}}^{2} + \lambda_{\text{ID}}\,\mathcal{L}_{\text{ID}} + \lambda_{\text{FD}}\,\mathcal{L}_{\text{FD}}, \tag{5.7}$$

where the supervised losses $\mathcal{L}_{\text{ID}}$ and $\mathcal{L}_{\text{FD}}$ are retained to limit regression from the original data contracts. Crucially, the target blocks $(\Delta\boldsymbol{x}^{*}, \Delta\dot{\boldsymbol{x}}^{*})$ were shuffled across valid states during this stage, so the networks could no longer succeed by reconstructing the logged trajectory. They had to respond to targets that the logged controller never issued in that state.

After 30 epochs of paired tuning the composite control MSE dropped to 0.109 and all four axes reached slopes above 0.8 and correlations above 0.92. The resulting ID checkpoint (`best_id_cylinder_control.pt`) is the ID model deployed in all PPO–ID–FD and PPO–ID(AAM)–FD experiments, where it is frozen. The fine-tuned FD from this stage, in contrast, was accepted only as an intermediate: its remaining weakness under independent valve excitation is the subject of the next section.

Table 5.2: Effect of paired control fine-tuning on the composite $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ differential response (5.6), evaluated per cylinder axis on validation data. Before fine-tuning, the composite had essentially no control authority despite near-perfect supervised scores of both networks; the composite normalized control MSE fell from 1.0011 to 0.1086.

	Regression slope		Correlation	
Cylinder axis	before	after	before	after
1 (slew)	0.003	0.818	0.135	0.920
2 (boom I)	0.002	0.928	0.090	0.967
3 (boom II)	0.001	0.849	0.037	0.933
4 (head pivot)	0.004	0.912	0.249	0.958

5.3 Forward Dynamics Model

5.3.1 Input–Output Contract and Supervised Baseline

The FD network predicts the one-step cylinder response to a valve command,

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{FD}}\Big(\boldsymbol{u}_k,\ \boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k\Big) \in \mathbb{R}^{12}, \tag{5.8}$$

a 20-dimensional input mapped to the three groups of state increments over $T = 0.05\,\mathrm{s}$. The architecture is the same residual MLP trunk as the ID network (width 512, four residual blocks, SiLU, layer normalization, dropout 0.02). Supervised pretraining on the closed-loop Simscape export used normalized MSE on all three output groups, Adam with learning rate 10^{-4}, weight decay 10^{-4}, batch size 8 192, and an auxiliary recursive rollout loss (weight 0.25, horizons of 2, 5, 10, and 20 steps), with the checkpoint selected by validation rollout loss [Nagabandi 18]. The selected model (epoch 291) reached grouped one-step R^2 near 0.999 for the extension increments.

5.3.2 Assessment of the Supervised Score

A model intended to give a policy gradient information about actuation must respond to the *valve*, not merely to the state history. This property was tested directly with an input ablation: zeroing the valve input of the pretrained FD barely changed its long-horizon rollout error (valve-zero error ratio ≈ 0.94 at a 0.5 s horizon. Figure 5.5, dashed orange versus solid orange). The model had learned to extrapolate the smooth kinematics of closed-loop operation, effectively a sophisticated persistence model, because in feedback data the controller makes valve command and state history nearly redundant, the identifiability trap of Section 2.4.2. Ablation studies on the source MAT exports (Chapter 4) confirmed that the valve signal was not causally identifiable from the closed-loop distribution at all. No amount of retraining on the same distribution could fix this. The data had to change.

5.3.3 Strict Open-Loop Excitation Dataset

The repair dataset is the strict open-loop excitation export shown in Figure 5.4. All four valves receive a rank-4 multi-input open-loop sequence built from long, full-authority command plateaus: the first four segments excite one cylinder at a time, and the final

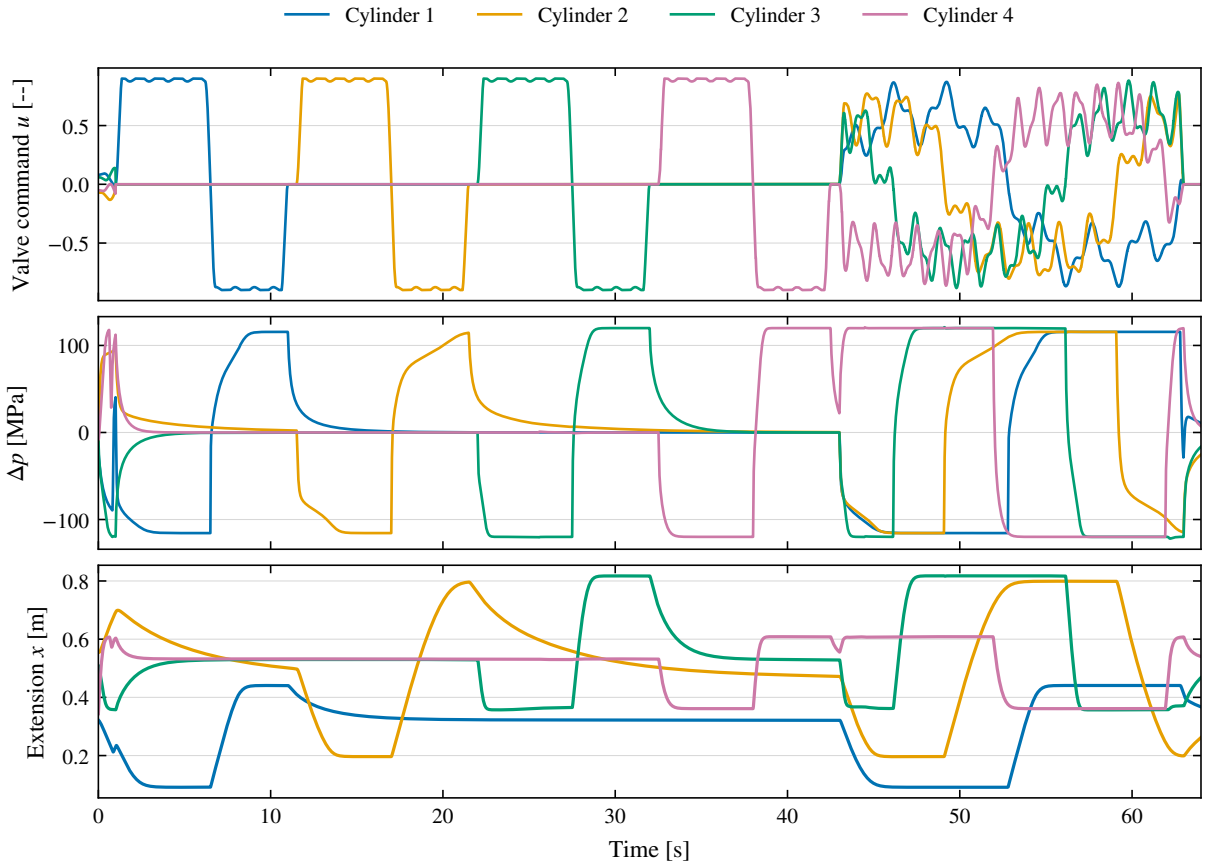

Figure 5.4: One run of the strict open-loop excitation dataset (nominal plant draw, 0.01 s sampling). Top: the rank-4 open-loop valve commands. The channels are excited individually in the first four segments and simultaneously in the final segment. Middle: resulting chamber pressure differences, including relief-limit episodes. Bottom: cylinder extension responses. The same excitation is repeated on three further randomized plant draws.

segment drives all four channels simultaneously (Section 4.4). The same excitation is applied to four distinct plant draws (nominal, randomized hydraulics, randomized mechanics, and both randomized), giving four 64-s runs at 0.01 s (6 400 samples each, 49 logged columns). Dataset validation established the properties that the closed-loop export lacked: all four state trajectories are distinct; every logged target increment agrees with the within-run difference $\boldsymbol{x}_{k+1} - \boldsymbol{x}_k$; valve history alone predicts the pressure response with $R^2 \approx 0.80$–0.85; the valve regressors have variance-inflation factors of only 7–10; and pressure-relief saturation occupies 9–22 % of the samples, so the saturated regime is represented but not dominant. An earlier excitation export was rejected outright because half of its runs duplicated state trajectories and the valve was again not causally identifiable, a reminder that excitation data must itself be audited, not assumed informative [Isermann 11].

For FD training, five consecutive 0.01 s samples are combined into one 0.05 s transition, with the valve input taken as the command held at the beginning of the window. Each run is divided into 20 temporal blocks with every fifth block assigned to validation and five-sample guard intervals removed at each boundary, so no transition window crosses a split boundary while train and validation samples still cover comparable pose and energy regimes. A plain last-20 % split was tried and rejected because it changed the target

spread between train and validation by a factor of 1.3–2.3 and would therefore have tested extrapolation to a different operating regime rather than generalization within one.

5.3.4 Strict Retraining Objective

Starting from the paired-control FD checkpoint, the promoted retraining run (150 epochs, Adam, learning rate 3×10^{-4}) minimized

$$\mathcal{L}_{\mathrm{FD}} = \mathcal{L}^{\mathrm{strict}}_{\mathrm{Huber}} + 0.25\,\mathcal{L}^{\mathrm{MAT}}_{\mathrm{Huber}} + 2.0\,\mathcal{L}^{(10)}_{\mathrm{roll}} + 10.0\,\mathcal{L}_{\mathrm{kin}}, \tag{5.9}$$

combining the one-step Huber loss on the strict data, a replay term on the original closed-loop data (weight 0.25, retained because ablation showed it improves state coverage and extension accuracy), a 10-step (0.5 s) recursive rollout loss, and a kinematic-consistency term enforcing the trapezoidal identity

$$\Delta \boldsymbol{x}_k = T\,\dot{\boldsymbol{x}}_k + \tfrac{1}{2}\,T\,\Delta \dot{\boldsymbol{x}}_k, \tag{5.10}$$

which holds in the measured strict data with $R^2 = 0.994$–0.9997 and ties the predicted extension and velocity increments to each other. The Huber loss is used because rare relief events otherwise dominate a squared-error objective. One further, initially attractive regularizer was evaluated and rejected: anchoring the model to a constant valve Jacobian. The measured valve authority varies by a factor of 15–75 across pressure quartiles and can change sign, so pinning a single global gain produced a pleasing mean response while collapsing one-step accuracy. This is consistent with the strongly operating-point-dependent valve–flow–pressure characteristics of hydraulic servo-systems discussed in Section 2.1.2, and it motivates the pressure-dependent structure adopted in the ROM (Section 5.4).

5.3.5 Validation of Valve Authority

Figure 5.5 and Table 5.3 summarize the outcome. The strict FD reduces the 0.5 s extension rollout RMSE from 75.2 mm to 12.4 mm and the 1.0 s RMSE from 157.9 mm to 27.3 mm, and, decisively, its valve-zero error ratio rises from 0.94 to 1.81 at 0.5 s: removing the valve input now roughly doubles the rollout error, which is the sensitivity a control-relevant forward model must exhibit. Per-axis one-step extension-increment R^2 improved from the range $[-5.7, 0.60]$ to $[0.76, 0.98]$. Two honest limitations remain. First, at very short horizons (≤ 0.25 s) the zero-order hold baseline is still competitive, because extension simply does not move far in 0.05 s. The learned model's value appears at the horizons over which a target increment must be committed. Second, the velocity-increment outputs remain poorly predicted (near-zero or negative R^2 on two axes, dominated by relief transients that a memoryless 0.05 s model cannot resolve). The deployed controller therefore integrates only $\Delta\hat{\boldsymbol{x}}$, the accurately modeled and directly validated quantity, and treats the velocity output as diagnostic.

For completeness, an intermediate rank-preserving fine-tuning stage between the supervised and paired stages regularized the valve-to-extension differential map with a finite-difference Jacobian penalty. The resulting checkpoint has a full-rank (rank-4) valve Jacobian with condition number ≈ 2.3 and is retained as provenance, but the strict FD supersedes it as the active model. The strict FD (`best_fd_cylinder_strict.pt`) together with the paired-control ID constitutes the learned actuator pair of every PPO–ID–FD experiment in Chapter 7.

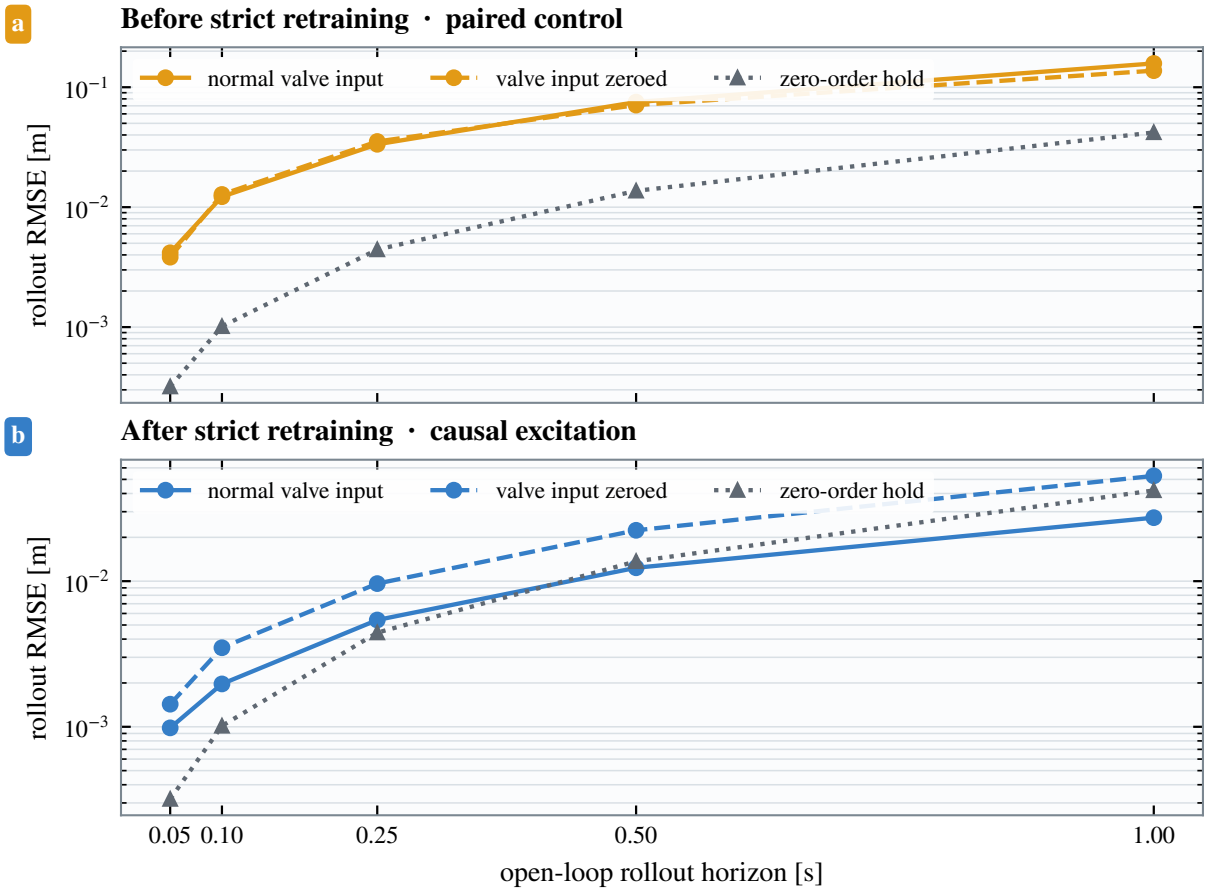

Figure 5.5: Open-loop extension rollout RMSE on held-out strict-excitation data as a function of rollout horizon, before (orange) and after (blue) strict retraining. Dashed curves show the same model with the valve input zeroed. The gap between solid and dashed is the model's usable valve authority. The paired-control FD is insensitive to its valve input, whereas the strict FD degrades by a factor of 1.8–1.9 when the valve is removed and outperforms the zero-order hold baseline (gray) beyond 0.5 s.

5.4 Reduced-Order Model

5.4.1 Motivation and Design Position

The PPO–ID–FD actuator path composes two separately trained networks and, as the previous sections showed, needed three repair stages before the composition had authority. The PPO–ROM path replaces the composition with a single *command-conditioned reduced-order model* that is trained end-to-end on the quantity the controller consumes: the response of the cylinders to a desired cylinder command. Two design decisions distinguish the ROM from a generic sequence model. First, it is *grey-box*: pressure, force, velocity, and extension are propagated through explicit physical update equations with trainable effective parameters, and neural components only supply what the physics cannot: the valve realization, unmodeled load, and bounded residual corrections [Karniadakis 21, Lutter 19]. This follows the observation from Section 5.3 that valve authority is strongly pressure-dependent: encoding the orifice-flow–continuity structure [Merritt 67] makes that dependence architectural rather than something to be rediscovered from data. Second,

Table 5.3: Strict-excitation validation of the FD model before and after strict retraining. The valve-zero error ratio is the rollout RMSE with zeroed valve input divided by the RMSE with the true valve input at a 0.5 s horizon; a ratio near 1 means the model ignores the valve.

Metric	Paired-control FD	Strict FD
Valve-zero error ratio at 0.5 s	≈ 0.94	1.81
Extension rollout RMSE at 0.5 s	75.2 mm	12.4 mm
Extension rollout RMSE at 1.0 s	157.9 mm	27.3 mm
One-step Δx R^2 (per-axis range)	-5.7 to 0.60	0.76 to 0.98

it is *recurrent*: a GRU encoder [Cho 14] over 0.25 s of history replaces the memoryless state input of the FD, in the spirit of the learned actuator networks of Hwangbo et al. [Hwangbo 19], because hydraulic response depends on recent pressure and command history that a single sample cannot represent.

5.4.2 Input–Output Contract

The ROM consumes a history of $L = 6$ samples (0.25 s at $T = 0.05$ s) of the 24-dimensional feature vector

$$\boldsymbol{z}_k = \left[\boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \hat{\boldsymbol{u}}_{k-1},\ \boldsymbol{c}_k\right] \in \mathbb{R}^{24}, \tag{5.11}$$

i.e. the cylinder state, the *previously predicted* valve command fed back as recurrent state, and the current desired command $\boldsymbol{c}_k \in \mathbb{R}^8$ of (5.2). The actual valve command appears only as a training target for the internal valve head. At runtime no valve signal enters the model. From $\mathcal{H}_k = (\boldsymbol{z}_{k-L+1}, \ldots, \boldsymbol{z}_k)$ the ROM predicts the internal valve realization and the full next cylinder state,

$$\left(\hat{\boldsymbol{u}}_k,\ \Delta\hat{\boldsymbol{p}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\hat{\boldsymbol{x}}_k\right) = f_{\mathrm{ROM}}\left(\mathcal{H}_k\right), \tag{5.12}$$

which jointly replaces the ID–FD composition of the PPO–ID–FD path.

5.4.3 Grey-Box Structure

Figure 5.6 shows the computation. The normalized history is encoded by a two-layer GRU of width 128 (dropout 0.05). The final latent $\boldsymbol{h}_k$ and the command $\boldsymbol{c}_k$ feed a small actuation head whose output passes through a learnable smooth dead-zone element (per-cylinder trainable width and asymmetric gains, built from softplus and tanh primitives so it remains differentiable), reflecting the dead band and direction-dependent gain of proportional valves (Section 2.1.2). Model-based designs treat the same nonlinearity by inverting an identified dead-band analytically [Yang 26]. Here its width and asymmetric gains are trained per cylinder instead, so no separate dead-band identification experiment is required. The resulting internal actuation $\hat{a}_i \in [-1, 1]$ drives an orifice-flow and continuity update per cylinder i,

$$\begin{aligned} q_i &= \hat{a}_i\, k_{q,i} \sqrt{\tfrac{1}{2}\left(p_{s,i} - \mathrm{sgn}(\hat{a}_i)\,\Delta p_i\right)_+}, \\ \Delta\hat{p}_{i,k+1} - \Delta p_{i,k} &= \frac{4\beta_i}{V_i(x_i)}\left(q_i - A_i\dot{x}_i - k_{\ell,i}\,\Delta p_i\right) T \ +\ r_{p,i}, \end{aligned} \tag{5.13}$$

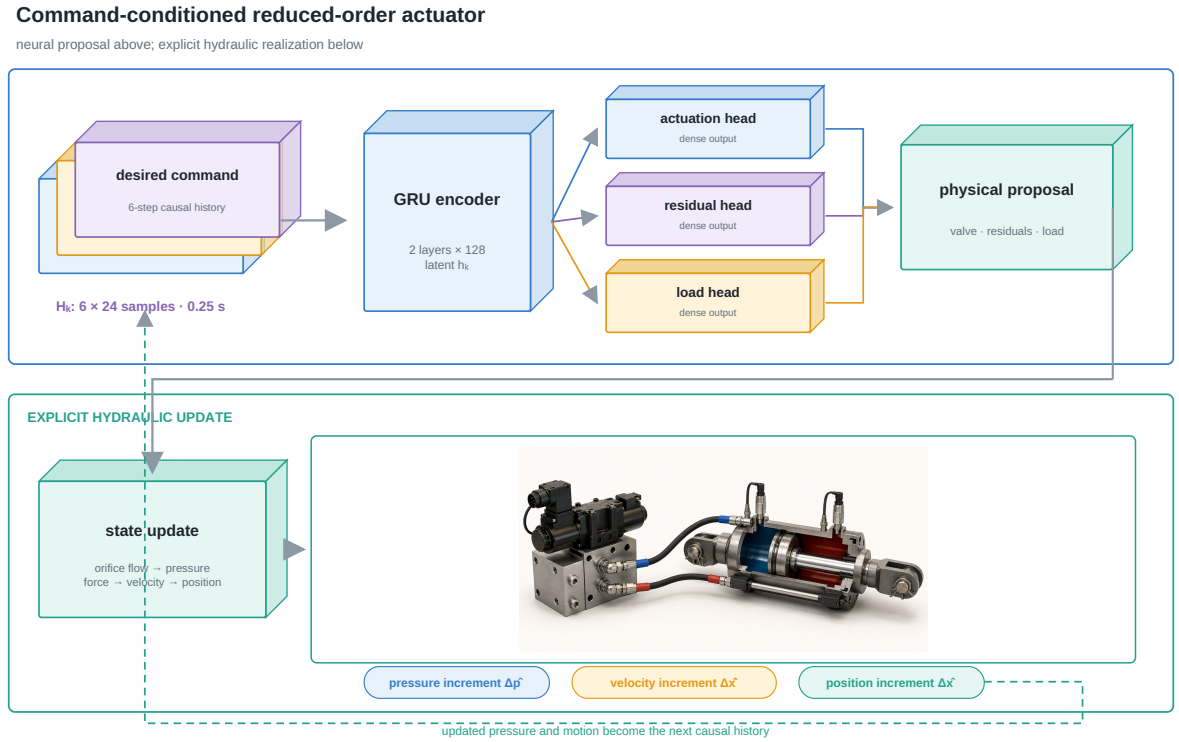

Figure 5.6: Structure of the command-conditioned reduced-order model as a data-flow pipeline. The 0.25 s history is split into its three feature groups (state, previously predicted valve, desired command), normalized, and passed through the recurrent GRU encoder whose hidden state carries pressure and command history across the six steps. The final latent feeds three fully connected heads: the actuation head realizes the command as an internal valve signal, the residual heads supply tanh-bounded corrections, and the load head the configuration load. The state is then propagated through the explicit pressure (5.13), velocity (5.14), and position (5.15) updates with trainable effective hydraulic parameters, so the neural parts propose and the physics disposes.

with effective bulk modulus β_i, supply pressure $p_{s,i}$, flow gain $k_{q,i}$, leakage coefficient $k_{\ell,i}$, piston area A_i, and extension-dependent chamber volume $V_i(x_i) = V_{0,i} + A_i(x_i - x_{\mathrm{ref},i})$; the softplus-smoothed pressure margin $(\cdot)_+$ keeps the square root differentiable near the supply limit. The pressure then produces force, and force produces motion:

$$\hat{F}_i = \Delta\hat{p}_{i,k+1} A_i + r_{F,i}, \qquad \ddot{x}_i = \frac{\hat{F}_i - c_{v,i}\,\dot{x}_i - \ell_i(\boldsymbol{h}_k)}{m_i},$$

$$\dot{\hat{x}}_{i,k+1} = \dot{x}_{i,k} + \mathrm{sat}\Big(T\,\ddot{x}_i + r_{v,i}\Big), \tag{5.14}$$

$$\hat{x}_{i,k+1} = x_{i,k} + \tfrac{T}{2}\Big(\dot{x}_{i,k} + \dot{\hat{x}}_{i,k+1}\Big) + r_{x,i}, \tag{5.15}$$

where $c_{v,i}$ is a viscous friction coefficient, m_i an effective modal mass, and $\ell_i(\boldsymbol{h}_k)$ a neural load head that captures the configuration-dependent gravitational and coupling load. All physical parameters are trained in log-space with hard clamps to physically plausible ranges, so they can adapt to the data without leaving their meaning behind. The trapezoidal position update (5.15) is the same identity (5.10) that the strict data satisfies almost exactly. The neural residuals r_p, r_v, r_x are tanh-bounded to 10 % of the pressure step limit,

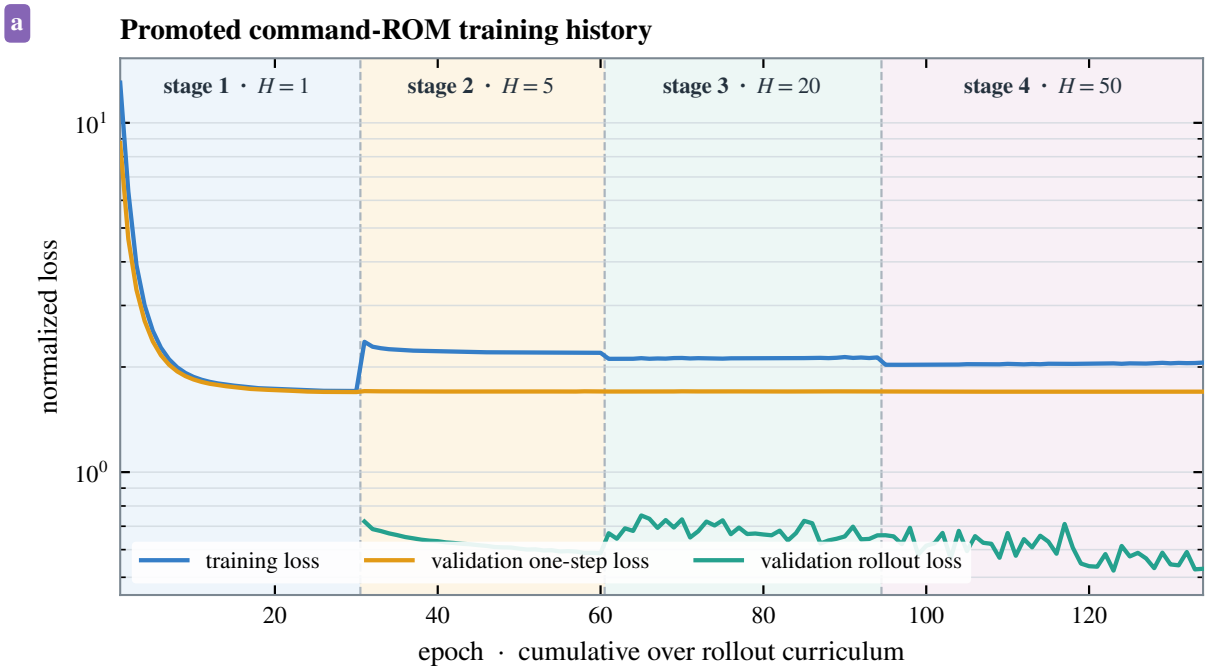

Figure 5.7: Training history of the promoted command-ROM run over the four-stage rollout curriculum (H is the recursive rollout horizon of each stage). The step in training loss at each stage boundary reflects the added longer-horizon rollout term; the validation one-step loss remains stable while the validation rollout loss continues to improve through stage 4.

10 % of the velocity step limit, and 1 cm respectively: large enough to absorb model error, too small to let the network bypass the physics. This bounded-residual layering is the central robustness argument of the grey-box design: even a poorly trained residual cannot produce an unphysical state jump.

5.4.4 Dataset and Training

The command-ROM dataset joins the aligned ID and FD closed-loop exports on exact run identifiers and time indices, yielding the 452 400 transitions and run-level 936/52/52 split summarized in Section 5.1. The logged valve commands enter only as supervision for the internal valve head. Training (seed 17, Adam, learning rate 3×10^{-4}, weight decay 10^{-5}, batch size 2 048, gradient-norm clipping at 5) follows a four-stage rollout curriculum with horizons $H \in \{1, 5, 20, 50\}$ and stage lengths of 30/30/30/40 epochs: each stage optimizes the weighted one-step losses plus a recursive rollout loss at its horizon, with scheduled sampling ramping from fully teacher-forced to 75 % self-fed within each rollout [Bengio 15]. The loss weights (pressure 1.0, velocity 2.0, extension 1.0, rollout 2.0, valve supervision 1.0, residual penalty 0.1, actuation regularization 0.5) penalize the residual heads explicitly so that explanatory burden stays on the physical terms. The logged force channel receives zero supervised weight: it failed a piston-area consistency audit on the current curriculum ($F \neq \Delta p \cdot A$ within tolerance), and supervising a head on a label that violates the model's own structure would corrupt the pressure chain. The ROM still exposes its pressure-derived force internally. Force supervision can be enabled once a reliable chamber-force label is available.

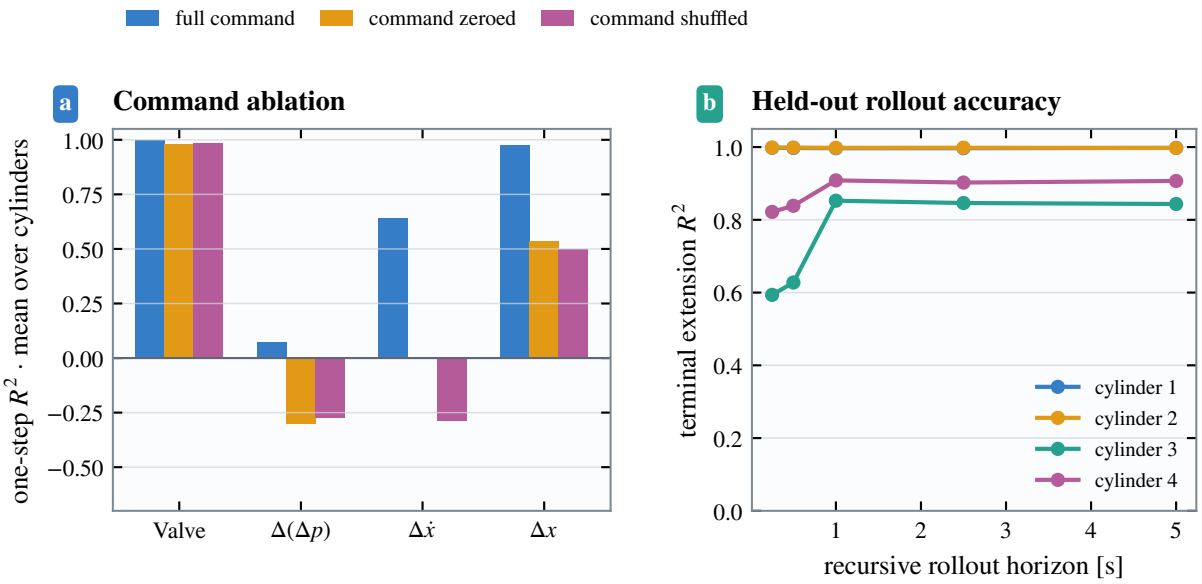

Figure 5.8: Offline validation of the command ROM on held-out runs. (a) One-step R^2 per output group (mean over the four cylinders) with the full desired command, with the command zeroed, and with the command shuffled across the batch. The collapse of the velocity and extension scores without the true command demonstrates command authority. (b) Terminal extension R^2 of recursive rollouts up to 5 s (100 steps) per cylinder.

Figure 5.7 shows the resulting training history. The characteristic step in the training loss at each stage boundary is the newly added, harder rollout term. The important observation is that the validation one-step loss does not degrade as longer horizons are introduced, while the validation rollout loss improves. The curriculum buys long-horizon stability without sacrificing one-step accuracy.

5.4.5 Offline Validation

The offline acceptance report (4 096 held-out samples, all gates passed) is summarized in Figure 5.8. Three findings matter for the control experiments. First, *command authority*: with the true command, mean one-step R^2 is 0.998 for the internal valve prediction, 0.64 for the velocity increment, and 0.97 for the extension increment. Zeroing or shuffling the command collapses the velocity score to below zero and the extension score to ≈ 0.5, and the 4×4 Jacobian of predicted extension increment with respect to the commanded increment is diagonally dominant with singular values 1.00–0.47 and condition number 2.14. The model responds to what it is asked to do, per axis, which is the property whose absence broke the pretrained FD. Second, *rollout stability*: recursive rollouts remain accurate and bounded to at least 5 s (terminal extension R^2 of 0.998/0.998/0.84/ 0.91 per cylinder), so PPO–ROM training rollouts do not ride on a diverging model. Third, *honest weaknesses*: the one-step pressure-increment R^2 is low (0.06–0.09): pressure transients at 0.05 s resolution remain largely unpredictable, and the model's pressure state is best understood as an internal latent that supports the velocity chain rather than as a calibrated pressure estimator. Cylinders 3 and 4 (boom II and head pivot) are consistently harder than cylinders 1 and 2 in both velocity and rollout scores. Inference costs 21.5 μs per step in evaluation, which is compatible with the vectorized IsaacLab training loop.

5.4.6 Closed-Loop Calibration Against the Deployment Plant

Everything above identifies the ROM against Simscape. The deployed checkpoint has one further stage, and it is stated here because describing this model as purely Simscape-trained would be inaccurate and would misrepresent the comparison of Chapter 7.

At runtime the ROM does not drive Simscape. It drives a PhysX articulation whose contact behaviour, solver, and effective compliance are its own. A small systematic difference between the extension the ROM commits to and the extension PhysX realizes is invisible from Simscape data by construction. The promoted checkpoint therefore adds a bounded correction term to the extension update,

$$\hat{x}_{i,k+1} \leftarrow \hat{x}_{i,k+1} + \delta_i(\boldsymbol{h}_k, \boldsymbol{c}_k), \qquad |\delta_i| \le 3\,\mathrm{mm}, \qquad \delta_i\Big|_{\boldsymbol{c}_k=\mathbf{0}} = 0, \tag{5.16}$$

produced by a command-gated correction head and trained on 4 084 closed-loop transitions collected in IsaacLab on the five plant-reachable reference shapes, with the Simscape command-ROM dataset retained as a replay term at weight 0.25 (Section 4.6). Every other parameter of the model (the flow, continuity, force and integrator chain and all of its physical coefficients) is frozen during this stage, so the identified hydraulics are preserved and only a deployment offset is placed on top of them. The gating in (5.16) is what keeps the stationary hold contract intact: at zero command the correction is exactly zero, so a commanded hold remains a hold.

This is the reason the command ROM, and not the ID–FD pair, serves as the deployment plant of Section 3.4.1, and it is a difference between the two actuator paths that must be reported whenever their tracking numbers are compared.

Beyond these aggregate scores, the acceptance protocol requires pressure–velocity phase-plane checks, frequency-response comparisons, and an in-simulator cylinder step gate before any PPO–ROM training run. A state-only variant trained with the command zeroed throughout serves as the mandatory comparison model for the identifiability ablations in Chapter 7. The command ROM is *frozen* for the final comparison: it is the actuator of PPO–ROM and, separately, the plant through which the adaptation dataset of Section 4.7 was collected. It is not conditioned on any latent. The adaptation mechanism developed in Sections 5.5–5.7 acts on the inverse model of the PPO–ID–FD path instead, and leaves this model untouched.

5.5 System Dynamics Model

The three actuator models of the previous sections are alternatives for the same job. The system-dynamics model is not. It is an *instrument*: a learned approximation of the deployment plant itself, the command ROM followed by PhysX, trained from scratch on the IsaacLab collection of Section 4.7. It never serves as the actuator of any PPO stack. Its only purpose is to answer whether a compact factor-dependent context exists at all, and whether conditioning on that context improves prediction of held-out hydraulic rollout. If it does not, there is nothing for an online estimator to recover and the adaptation argument stops here.

The contract is a 50 ms transition. From the observable state, the applied action and a causal history the model predicts the next increment of the twelve-dimensional hydraulic

state, four cylinder extensions, four velocities and four pressure differences. Joint position and velocity enter as context. The action is the pair of consecutive valve commands $[\boldsymbol{u}_{k-1}, \boldsymbol{u}_k]$, which is what the command ROM itself consumes at a tick, so the model is asked the same question the plant is.

Two architectural choices follow from the data rather than from preference. The model carries a belief module of width 16 over a six-tick history, because a single sample cannot summarize the pressure state that drives the next transition. Rollout is trained with a hold-start joint mode and an unroll of five transitions, with the rollout term ramped in over the first 40 epochs and a teacher-forcing floor of 0.2, so that the model is optimized for recursive use rather than for one-step accuracy alone (Section 2.4).

The matched control is SD_0, identical in every respect except that no latent is supplied. Both models train on the same 82 runs with the same splits, normalizers and rollout protocol; the factor normalization is computed on training domains only. SD_0 selects epoch 58 and SD_z epoch 48. Their scalar validation losses are close and are not the criterion: the two models consume different inputs, so the comparison that decides the question is the rollout study of Section 5.6, run at a fixed horizon on runs neither model trained on.

5.6 Privileged Latent Representation

The privileged encoder $\boldsymbol{z} = \mu(\boldsymbol{e})$ compresses the six-dimensional physical factor vector of Section 4.7.2 into a two-dimensional latent. It is trained *jointly* with the latent-conditioned model SD_z, so the representation is shaped by what helps predict rollout rather than by a reconstruction objective; the factor reconstruction weight is zero. Factor normalization uses training-domain statistics only.

The claim that a latent *matters* is tested directly and adversarially. At a horizon of 20 ticks, one second of plant time, SD_z is evaluated with the true latent against five controls: zero, shuffled, random, the mean encoded latent over training runs, and the nominal factor setting. All controls are constructed from the training pool alone, with fixed seeds. The test is paired per run and reported with bootstrap confidence intervals on validation and on the sixteen reserved test runs separately.

The true latent wins on both pools. Against the zero control the pooled rollout error falls by 7.20×10^5 on validation, with a 95 % confidence interval of $[-1.03 \times 10^6, -4.30 \times 10^5]$ over 20 runs, and by 4.41×10^5 on the reserved test runs, interval $[-7.61 \times 10^5, -1.74 \times 10^5]$ over 16 runs. Neither interval crosses zero, and the gate records no failures.

What this establishes, and what it does not, must be stated precisely. It justifies freezing the representation and attempting online inference, because a two-dimensional context demonstrably carries information about held-out hydraulic rollout that the state and action do not. It is *not* evidence that the AAM works, because it uses the true $\boldsymbol{z}$ and not an estimate. It is *not* evidence of any control improvement. No latent dimension is named after a physical parameter anywhere in this thesis, because that relationship was not measured.

The representation is then frozen. The encoder, SD_z, the belief module, the normalizers, the input mask, the split and the rollout protocol are written to `stage31_frozen/z2_rom` with a recorded protocol stamp (2026-08-13, `stage31_frozen_sd_v1`), and everything downstream reads that frozen copy.

5.7 Arm Adaptation Module

The Arm Adaptation Module estimates the latent from history that exists at deployment: $\hat{\boldsymbol{z}} = \mathrm{AAM}(\boldsymbol{H}_t)$. Its input row is eighteen-dimensional, comprising five joint positions, five joint velocities, four cylinder pressure differences and the four previous valve commands. Five consecutive 50 ms rows form the window, so the estimator reads 0.25 s of past and 90 inputs in total.

The pressure entry deserves a precise description, because it is the signal that carries most of the hydraulic factor information and it is not a measurement. The four pressure differences are the learned actuator's own recursive pressure state, produced by the command ROM in the PPO–ROM stack and by the frozen forward model in PPO–ID(AAM)–FD. They are available at every deployment step because the actuator model that generates them runs inside the deployed loop. Nothing privileged enters: the factor vector, the true latent, the desired reference, future rows and the model's hidden recurrent state are excluded by construction, and the previous valve command is used rather than the current one so that the row is causal at the instant the estimate is needed.

History length was selected from a predeclared set of 5, 10, 20 and 40 ticks, corresponding to 0.25, 0.5, 1.0 and 2.0 s, on validation runs only. The rule was to take the shortest history that remains statistically comparable with the best downstream rollout, and it selected the shortest candidate, 0.25 s. A shorter window is preferable on a machine that must react within a tick, and the selection rule, not the outcome, is what makes that defensible.

Training adds a causal temporal-consistency term at unit weight to the latent regression, which penalizes an estimate that jitters between adjacent windows. Input normalization is computed on training windows only. The network is a three-layer network of width 256 with dropout 0.05, and the selected checkpoint is epoch 10.

Against a constant-prediction baseline the estimator reduces latent error substantially on runs it never saw. On validation, over 76 720 windows, the root-mean-square latent error is 0.828 against 1.605, a mean-squared reduction of 73.4 %. On the sixteen reserved test runs, over 61 376 windows, it is 0.743 against 1.326, a reduction of 68.6 %. The gate records no failures.

Deployment behaviour is fixed by the online runtime. Inference runs every three control steps, which places it on the 20 Hz ROM grid inside a 60 Hz control loop, under a declared latency budget of 50 ms. The first valid estimate appears only after five complete rows have accumulated; until then the adapter is bypassed and the frozen inverse model's action is returned unchanged. A past-only rolling mean over twenty estimates smooths the output. That smoother is causal by construction, which the earlier non-causal variant discussed in Section 5.8 was not.

5.8 Validation of the Adaptation Chain

The chain is only useful if the deployable estimate retains a worthwhile share of the advantage that the true latent buys. That is tested by comparing $SD_0(\boldsymbol{s}, \boldsymbol{u})$, $SD_z(\boldsymbol{s}, \boldsymbol{u}, \boldsymbol{z})$ and $SD_z(\boldsymbol{s}, \boldsymbol{u}, \hat{\boldsymbol{z}})$ under the frozen protocol, on validation and on the reserved test runs, with paired bootstrap intervals.

The gate passes with the causal estimator, and the two components of that result should be read together rather than separately. Section 5.6 establishes that the latent carries

information, with intervals that exclude zero on both pools. Section 5.7 establishes that the causal estimator recovers it well enough to cut latent mean-squared error by roughly two thirds on runs it never saw. The chain gate confirms that the second survives substitution into the first.

One audit artifact is retained deliberately. The original Stage 25 to 27 evaluation scored the estimator using a full-run mean of held-out predictions. That statistic is not a deployable estimator, because it uses the whole run to predict any instant within it, and the result obtained under it is therefore superseded rather than reported. The corrected causal score is what selects the 0.25 s history and what the numbers above refer to. Keeping the superseded verdict visible, and labelled, is the point: the failure mode it represents is easy to reintroduce.

5.9 Latent-Conditioned Inverse-Model Adapter

The adapter is where adaptation reaches the plant. The frozen inverse model produces a base command $\boldsymbol{u}_{\text{base}} = \text{ID}(\boldsymbol{s}, \boldsymbol{r})$, and the adapter adds a small bounded residual conditioned on the estimated latent,

$$\boldsymbol{u} = \text{clip}\Big(\boldsymbol{u}_{\text{base}} + g_\psi(\boldsymbol{s}, \boldsymbol{r}, \boldsymbol{u}_{\text{base}}, \hat{\boldsymbol{z}})\Big). \tag{5.17}$$

The inverse model, the forward model and the policy stay frozen. The residual is bounded by construction and scaled by a declared factor, so a warm-up period or an invalid latent degrades the stack exactly to PPO–ID–FD rather than to something new. That fallback is not a safety afterthought; it is what makes the comparison in Chapter 7 a controlled one.

Training targets the valve command the command ROM actually issued at the exact ROM tick, minimizing the error between the clipped, adapter-corrected action and that recorded command. The split is the frozen one, 82 training and 20 validation runs, and the sixteen reserved test runs are recorded as untouched in the run manifest. The selected checkpoint is epoch 116.

Deployment evidence exists but does not yet isolate the adapter's contribution, and the distinction matters. A policy fine-tuned with the adapter active passes all eight gates on six held-out meshes over the full 26 m strip, with worst-mesh cut-height error 1.1 mm, worst-family lateral error 66 mm, ground contact 0.982 and head-alignment p01 0.995. That is a complete, gate-passing adaptive stack. It is not, on its own, evidence that adaptation caused the result, because no adapter-off run of the same checkpoint at the same length exists.

The one matched comparison available is weaker and shorter. On a single mesh, with the adapter scaled to zero and then to 0.25 on the same initial checkpoint, tracking does not improve: worst-family lateral error rises from 50.0 mm to 67.4 mm and ground contact falls slightly, while tool acceleration and cylinder headroom improve marginally. That run covers 8.9 m of the 26 m strip, so it exercises a third of the task. It is reported here because it is the only paired evidence in hand, and because a favourable full-length result without a control would be a weaker claim than an unfavourable short one with a control.

The experiment that settles this is a single run: the adapter-trained checkpoint evaluated at zero residual scale over the full strip on the same six meshes. Until it exists, this section describes a working adaptive stack and declines to claim a causal improvement.

Table 5.4: Provenance of each model in the adaptation chain. Frozen components are loaded unchanged and never receive gradients in this chapter.

Model	Status	Selected checkpoint
Control ID	frozen	historical, Section 5.2
Strict FD	frozen	historical, Section 5.3
Command ROM	frozen	`blend25`, Section 5.4
SD_0 (no latent)	scratch	epoch 58
SD_z + encoder	scratch	epoch 48, frozen after the gate
AAM	scratch	epoch 10, 0.25 s history
Latent adapter	scratch	epoch 116

5.10 Training and Model-Selection Protocol

All models in this chapter share one split. Runs are divided 82 / 20 / 16 into training, validation and reserved test, by complete run *and* by factor configuration, so no factor key appears on both sides of a boundary. Windows never cross run boundaries. The sixteen reserved runs were untouched after the latent-matters gate of Section 5.6 and are recorded as unused in the adapter's own manifest.

Table 5.4 states, for every model, whether it is frozen or trained from scratch, since the comparison of Chapter 7 depends on that answer being unambiguous.

Selection is by held-out criterion rather than by training loss throughout. SD_z and SD_0 are selected on validation rollout, the AAM on downstream rollout quality rather than on latent error alone, and the adapter on validation command error. The state loss weights extension, velocity and pressure equally in normalized units, which is why the pooled rollout figure of Section 5.6 is dominated by the pressure channel: its raw magnitudes are six orders larger, and the normalized comparison is the one that carries meaning.

5.11 Offline Model Evaluation

The offline picture divides into three questions, and it is worth keeping them apart because they fail independently.

Does the plant model predict?

SD_z with the true latent reaches a one-tick extension error of 0.22 mm and a velocity error of 7.6 mm/s. At five ticks these grow to 1.0 mm and 12.6 mm/s, and at ten ticks to 1.7 mm and 14.9 mm/s. Growth is sublinear in the horizon for both channels, which is what the multi-step rollout objective was added to produce. The pressure channel is the weak one, as it was for the reduced-order model in Section 5.4, and for the same reason: pressure transients at a 50 ms resolution are largely unpredictable from the information available.

Does the latent carry information?

Answered in Section 5.6 at a 20-tick horizon against five controls, on validation and reserved test pools separately, with bootstrap intervals that exclude zero on both.

Can the latent be recovered causally?

Answered in Section 5.7: a 68.6 % mean-squared reduction against the constant baseline on the reserved test runs, from 0.25 s of causal history containing no privileged signal.

Three limitations are carried forward rather than buried. The pressure channel that dominates the pooled rollout statistic is also the least well predicted, so the pooled figure should be read alongside the per-channel numbers above and not instead of them. The evaluation domains are reported separately in Chapter 7 rather than pooled, because a model that adapts well on hydraulic draws and poorly on mechanical ones would otherwise be indistinguishable from one that adapts uniformly. And the adaptation-delay measurement implied by the warm-up of Section 5.7, five rows before the first valid estimate, is stated as a design parameter here and measured as a response time in Chapter 7.

6. Control Framework and Simulation Integration

Chapter 5 left three actuator representations sitting on a bench, each with its own contract and its own offline evidence. This chapter puts a controller in front of them and turns them into three experiments.

Section 6.1 describes the classical controller, because it is where the competence comes from even though it never runs at deployment. Section 6.2 specifies the policy: what it sees, what it emits, what stops it from emitting something dangerous, and what it is paid for. Section 6.3 then explains how the first becomes the second, and reports a result about that transfer which changed the training protocol of this thesis. The individual stacks appear only after those three are fixed, because they differ in one place only.

6.1 The DLS Teacher and Its Two Roles

6.1.1 Scope and Role of the Controller

The classical controller occupies a narrower position in this architecture than its prominence suggests. There is no damped-least-squares layer underneath the policy at runtime. Nothing is added to the policy's output. If the policy emits zero, the arm does not move.

The DLS controller is nevertheless the reason the policies work at all, and it appears twice, both times at training or collection time and never at deployment (Figure 6.1). It is the *privileged teacher* whose behaviour the policies are distilled from (Section 6.3), and it is the closed-loop controller that drove the identification runs of Section 4.7. Calling it a nominal layer would misdescribe the deployed system and, worse, would make every reported result a property of a controller pair rather than of the policy.

6.1.2 Privileged Information

The teacher is *privileged* rather than better tuned: it reads two quantities that no deployable controller on this machine could obtain. The first is the exact terrain height directly under

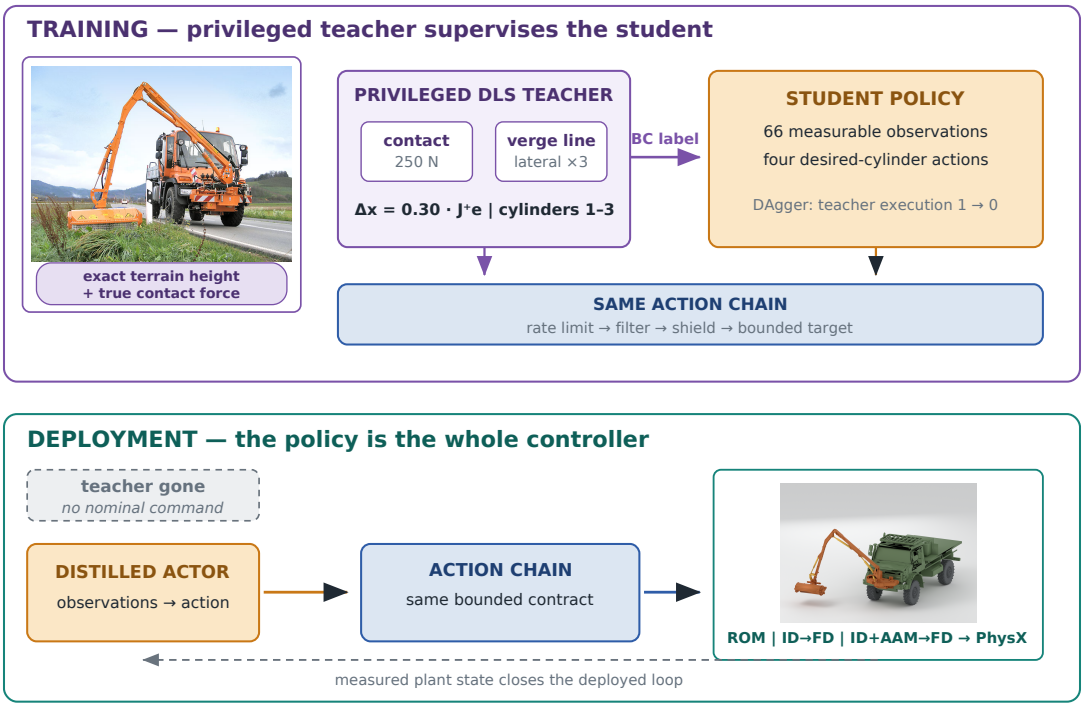

Figure 6.1: Where the classical controller lives. At training time the privileged teacher reads quantities the deployed system cannot (the exact terrain height under the cutter and the true contact force) and converts them into a desired cylinder target, which is then pushed through *the student's own* action chain and plant. At deployment the teacher is gone entirely: the policy is the whole controller, and no nominal command is summed with its output. The distilled actor is the only thing that crosses the boundary.

the cutter, queried from the simulator rather than inferred from a forward- looking scan. The second is the true cutter–ground contact force. Together these turn the hardest part of the task, knowing where the ground actually is, right now, underneath a 1.2 m flail head that is occluding it, into a lookup.

The teacher uses that privilege in a specific way. Rather than commanding a height, it runs a contact regulator: it maintains a vertical offset relative to the ground and adjusts that offset to hold the measured contact force near a target of 250 N within a 100 N deadband. Below the band it presses down by 0.25 mm per step; above it, it backs off proportionally; if contact is lost entirely it searches downward at 1 mm per step; on first acquisition it applies a small preload; and a hard-force branch retreats immediately if the force spikes. The accumulated offset is clamped to $[-0.30, -0.05]$ m so the regulator cannot drive the head into the ground or lift it clear.

The teacher does not know how to mow well because it has better kinematics. Its kinematics are the same DLS inverse available to anything. It knows how to mow well because it can *feel the ground*. That is the quantity the student must infer, from a noisy forward scan and its own contact history, and it is a large part of why distillation is a nontrivial transfer rather than a formality.

6.1.3 From Task Error to Cylinder Command

The rest of the teacher is conventional and is stated for completeness. A goal point is maintained in the truck frame: laterally it tracks the verge centre line, vertically it follows the measured ground height plus the contact regulator's offset. The task error is the difference between that goal and the measured cutter position, with the lateral component weighted by a factor of three, because on this machine lateral error is what the acceptance gate measures and vertical error is largely handled by contact.

That error is mapped to cylinder space through the damped least-squares inverse of the measured cutter Jacobian $\boldsymbol{J}_c$ of Section 3.2.2,

$$\Delta \boldsymbol{x}^*_{1:3} = \mathrm{clip}\Big(g\, \boldsymbol{J}_c^{+}\, \boldsymbol{W} \boldsymbol{e},\ \pm \Delta x_{\max} \Big), \qquad g = 0.30, \qquad \Delta x_{\max} = 3\,\mathrm{mm}, \tag{6.1}$$

with $\boldsymbol{W}$ the lateral weighting and the per-step clamp bounding how far any cylinder may be asked to move in one tick. Only cylinders 1–3 are solved. Cylinder 4 is held at the reference extension that keeps the head down, for the reason given in Section 3.1. The gain g is small on purpose: this is a stiff, heavy mechanism in contact, and a resolved-rate controller that closes quickly on the task error simply excites it.

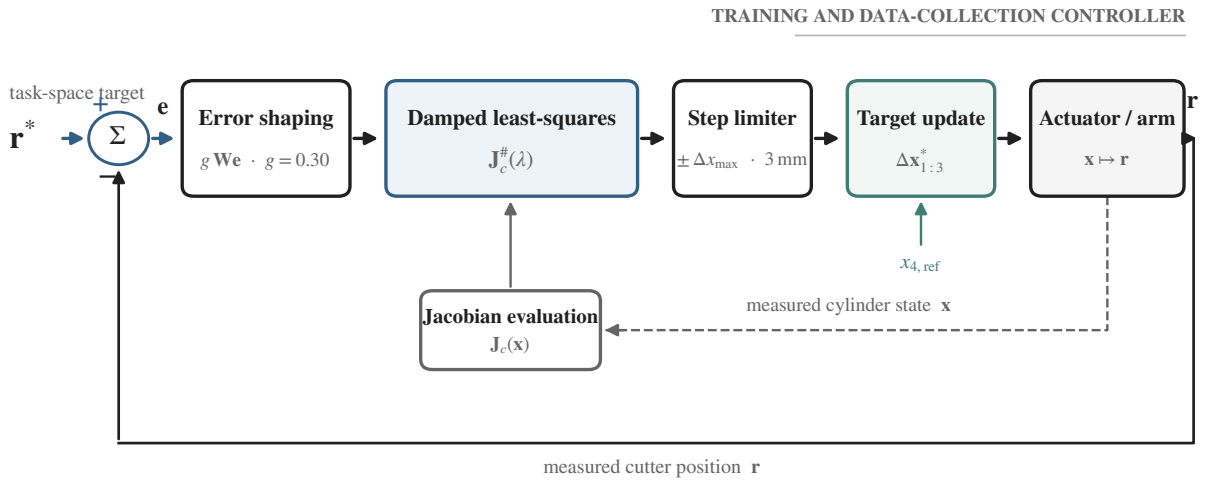

Figure 6.2: Standard feedback realization of the DLS teacher in (6.1). The weighted task error is scaled, mapped to the first three cylinder coordinates by the measured damped inverse, and clipped. Cylinder 4 is held at its head-down reference. Cutter position closes the task-space loop and the measured cylinder state updates $\boldsymbol{J}_c$. The loop is used only for teaching and data collection, not at deployment.

6.1.4 Execution Through the Student Interface

One implementation detail determines whether the distillation of Section 6.3 converges to something usable.

The teacher's output, $\Delta \boldsymbol{x}^*$, is a desired cylinder target. It is *not* written to PhysX directly. Instead it is converted into the same normalized four-value action the student emits, and then pushed through the student's entire action chain (rate limit, exponential filter, limit shield, command scale, and target integrator) and into the same command ROM and

Table 6.1: The privileged DLS teacher, measured on the stage-6 verge task through the command-ROM plant (128 environments, 480 steps, all eleven path families, seed 17). It is competent, not perfect, and several acceptance thresholds sit deliberately just outside what it achieves.

Quantity	Teacher	Gate
Mean $\lvert e_{\text{cut}} \rvert$	0.0234 m	≤ 0.040 m
p95 $\lvert e_{\text{cut}} \rvert$	0.1524 m	—
Mean $\lvert e_{\text{lat}} \rvert$	0.1221 m	≤ 0.300 m
Ground-contact fraction	0.7807	≥ 0.700
p01 head alignment	0.9638	≥ 0.900
Minimum head alignment	0.8317	—
p95 tool acceleration	64.9 m/s^2	≤ 120 m/s^2

PhysX plant (Section 6.2). The consequence is that every label the student is asked to imitate is, by construction, an action the student is capable of producing. A teacher that wrote targets directly to the plant would generate labels outside the student's reachable set, and the imitation error would have a floor that no amount of training could remove. This project has measured that failure mode in a related setting: when an earlier action-rate limit was set below the teacher's own structural bound, 0.43 % of the distillation labels became unreachable by construction, and that fraction acted as such a floor.

6.1.5 Teacher Performance

A teacher is not an oracle, and quoting its performance is necessary because distillation cannot exceed it and the gates of Section 3.9 were anchored to it. Table 6.1 reports the privileged teacher measured on the stage-6 verge task through the ROM plant, over 128 environments and 480 control steps.

Two readings follow. First, the teacher is a competent but visibly imperfect controller: it loses ground contact about a fifth of the time and its cut-height tail is six times its mean. Second, and this is why Section 3.9 re-derived several thresholds, a gate set above these numbers would be a gate that the ground-truth controller itself cannot pass, which carries no information about the policy.

6.2 Policy Formulation

6.2.1 Policy Definition

The deployed controller is a memoryless Gaussian multilayer perceptron. It maps a 66-value observation to four normalized actions, one desired motion command per hydraulic cylinder, and that is the entire control law. Actor and critic are separate networks with hidden widths 256–128–64 and ELU activations, over empirically normalized observations.

Three things it is *not* must be stated, because each was a design option that was considered and rejected. It is not a residual: there is no nominal command to correct. It is not recurrent: temporal inference in this architecture belongs to the AAM of Chapter 5, and giving the policy its own memory would confound the two. And it does not command

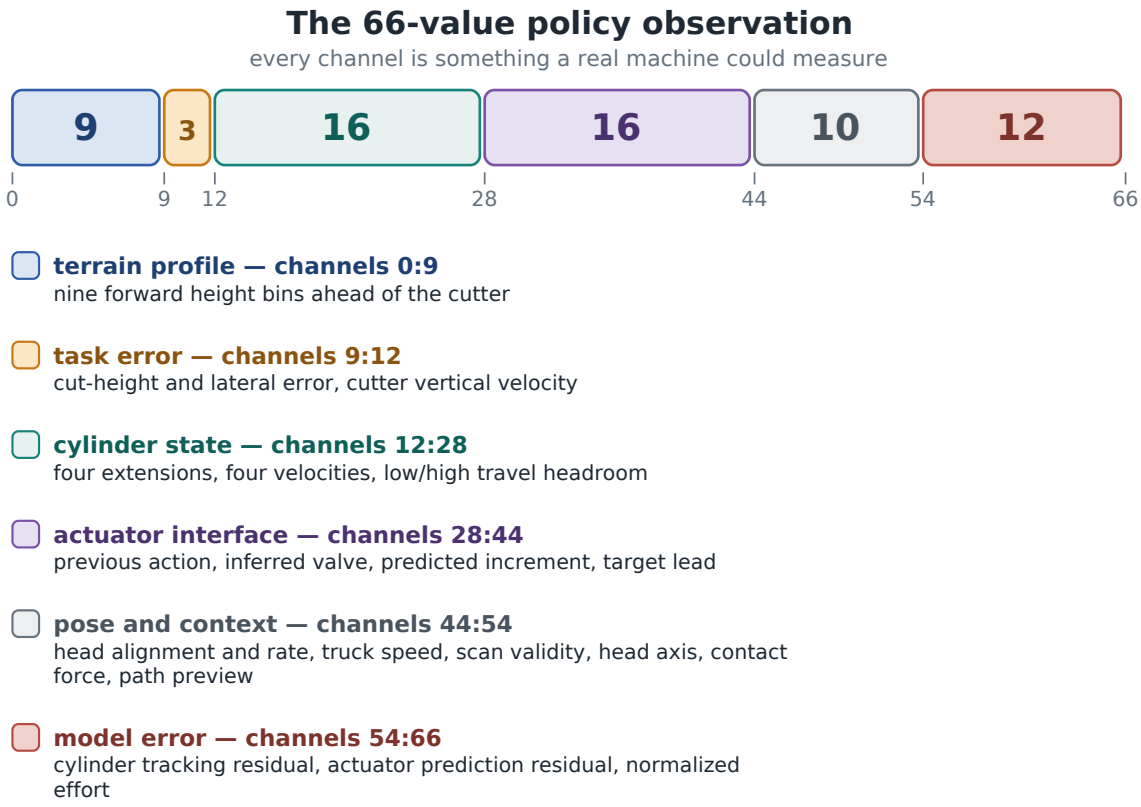

Figure 6.3: The 66-value policy observation, drawn to scale by group. Every channel is something a real machine could measure. No trajectory-family label, no physical factor vector, and no true latent enters the actor; if the optional AAM-to-policy conditioning is used, its two added channels must be declared as a different architecture rather than compared as though the policy were unchanged.

the vehicle: the truck's forward motion is scripted, so the policy cannot solve a tracking problem by driving differently.

6.2.2 The Observation Vector

Figure 6.3 lays out the 66 channels by group. The grouping tells the story of what this controller has to reason about: nine terrain-height samples ahead of the cutter, three task-error channels, sixteen channels of cylinder state and remaining travel, sixteen describing the actuator interface, ten of pose and context, and twelve of model error.

The last group is the one that deserves an argument, because twelve of sixty-six channels is a substantial fraction of the observation spent on something that is not the task. Those channels carry the actuator's standing error: the cylinder tracking residual (measured extension minus commanded target), the actuator prediction residual (what the cylinder actually did minus what the actuator model predicted it would do on the previous step), and the normalized cylinder effort.

The reason they exist is a direct consequence of the policy being memoryless. The observation already contains the actuator's prediction and the current measured extension.

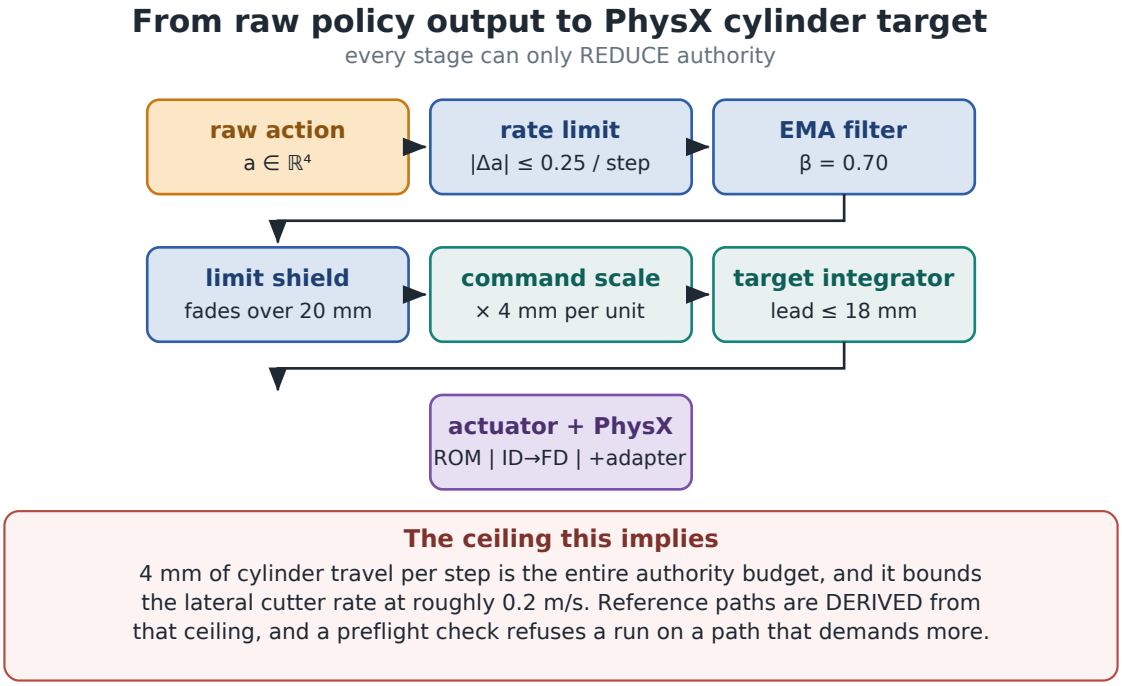

Figure 6.4: The action chain. With no nominal controller to explore around, the contract itself is the safety argument: a rate limit, an exponential filter, a limit shield that fades authority as travel headroom is consumed, a small per-step command scale, and an anti-windup cap on how far the integrated target may lead the measured extension.

But forming the *error* requires differencing position across two steps and comparing against a prediction issued on the previous step, and a feedforward network cannot construct either from a single observation. Without these channels, the model error the policy is being asked to compensate is not merely hard to see: it is not present in the input at all. This was measured rather than reasoned about: across ten training blocks the p01 head alignment sat at 0.79–0.80 and did not respond to changes in the feedforward source or the levelling gain, because the policy had no observable evidence of the error it was being asked to correct.

This group is also what makes the three stacks comparable. Under the ROM the tracking residual is near zero, because the ROM reaches what it is told. Under a learned ID–FD path it is the model's standing error. The same observation contract therefore gives each policy an honest view of its own actuator's shortcomings, which is what a fair comparison requires.

6.2.3 The Action and Its Bounds

The policy emits $\boldsymbol{a} \in \mathbb{R}^4$, sampled from its Gaussian at training time and taken at the mean for evaluation. Between that sample and the PhysX cylinder target sit five stages, shown in Figure 6.4. Every one of them can only *reduce* authority. None can introduce motion the policy did not request.

The rate limit bounds $|\Delta \boldsymbol{a}|$ at 0.25 per control step and is held *constant* across every curriculum stage. It is not a curriculum knob, and the reason is the reachability argument

of Section 6.1: the teacher's own command stream has a structural per-step bound near 0.20, so a student limit below that makes part of the label set unreachable. The exponential filter ($\beta = 0.70$ at the final stage) smooths the command. The limit shield scales the command down, and only down, and only in the direction that consumes headroom, over the last 20 mm of cylinder travel, which is the soft replacement for the teacher's hard clamp into the learned range. The command scale converts the filtered action into 4 mm of cylinder travel per unit action. Finally the target integrator accumulates the increments in reference space, with the integrated target permitted to lead the measured extension by at most 18 mm.

That last cap is an anti-windup device and it is load-bearing. Measured in evaluation on a trained checkpoint, relaxing it from 18 mm to 50 mm halved the lateral error on the two families that demand a sustained standing offset (`lane_change_hold` from 0.4833 to 0.2310, `boom_sweep` from 0.3745 to 0.1602), which proves the clamp was bounding the standing effort those paths need. The same relaxation applied to cylinder 4 drove head alignment from 0.9507 to 0.4636. The cap is therefore a real trade rather than a safety margin.

The final PPO–ROM task resolves that trade per axis, and this is the configuration every reported PPO–ROM result was produced under: the caps are 50 mm on cylinders 1 to 3, where the standing offset is needed, and 20 mm on cylinder 4, where relaxation costs head alignment. The 18 mm scalar quoted above is the inherited default that the measurement was made against.

Two properties of this chain follow. First, it is what replaces the residual band: exploration is bounded by how much motion a single action can command, not by proximity to a baseline. Second, it fixes the plant's capability ceiling. Four millimetres of cylinder travel per step is the entire authority budget, and it is what produces the roughly 0.2 m/s lateral rate limit of Section 3.6.2, which is why reference paths are derived from it and a preflight check refuses to start a run on a path that demands more.

6.2.4 The Reward Function

The reward of the final PPO–ROM task has ten bounded terms, listed in Table 6.2. Three are positive and describe the task. Seven are penalties and describe how the machine must behave while doing it. All are bounded, and all retain a live gradient beyond the acceptance boundary, so that a policy already inside a gate still has a direction to improve in.

It replaces an inherited formulation carrying 53 weighted reward fields. Isaac Lab's own locomotion task uses eleven. The reduction was not cosmetic. At 53 terms the optimum is not something a designer can reason about, and three separate terms turned out to be working against the task without being noticed for thirteen training campaigns. The three rules below were each adopted after a measured failure.

No Gaussian Shaping

The earlier reward priced tracking as $w \exp[-(e/s)^2]$, which is effectively flat beyond about $2s$. Measured on the lateral term, at an error of 0.50 m the gradient is -2.07 at $s = 0.30$ but -0.004 at $s = 0.15$: a factor of 520 weaker where the failing path families live. Every

shaping term is therefore linear or Huber in the error. Writing $x = |e|/s$, the two shapes used are

$$B(e; s, c) = \min(x,\, c), \qquad H(e; s, c) = \min\Big(c,\, \begin{cases} \frac{1}{2}x^2, & x \le 1 \\ x - \frac{1}{2}, & x > 1 \end{cases}\Big). \tag{6.2}$$

B has the same gradient at 0.5 m of error as at 0.05 m, so a policy far from the reference is still told which way to move; H buys quadratic precision near zero where cut height needs it, without a vanishing far field.

Bounded Terms

A term that can grow without limit can dominate the sum and destabilize the critic. Each contribution is clipped, which gives the reward a published budget: $6 + 12 + 4 = 22$ for the task terms against -24 for the safety terms. A non-finite value is replaced by -25, just below that worst case, so a numerical fault can never outscore legitimate behaviour.

Fixed Sign Convention

The three task terms are non-negative and the seven safety terms non-positive. This is what makes worst-case (CVaR) weighting safe to add later: scaling a non-positive term by a factor ≥ 1 can only lower the return, whereas up-weighting a *positive* reward for the worst environments pays the policy to be bad. Measured in the earlier stack, weighting the positive contact reward four-fold for the worst quarter of environments made a contact quality of 0.49 pay 7.84 against 2.00 for a quality of 0.50, and collapsed worst-family contact within a single training block.

Three terms of the earlier formulation are absent, and the reasons are results rather than preferences.

The *action-magnitude* penalty on mean(a^2) was removed outright. A terrain action is a desired cylinder motion, not a pose offset, so the term was read as charging for a standing bias that keeps driving the arm. What it actually charged for was the sustained offset that `lane_change_hold`, `offset_return` and `boom_sweep` structurally require: the policy correctly learned to return to neutral, which is the one thing "hold an offset" forbids. Removing it produced the first training block in a full session that did not degrade, improving the held-out score from 1.2058 to 0.3678.

The *head barrier* was replaced. In its earlier form (a quadratic of weight -20 with a -8 far field, a tail multiplier, and additional progress and memory terms) the head group dwarfed lateral tracking at $3.0 + 3.0$, so line following could never compete for the optimizer's attention. The two-scale bounded penalty of Table 6.2 keeps a sharp gradient just below the alignment threshold while remaining alive down to an inverted head, at a weight commensurate with the task terms it must trade against.

The *clearance barrier* was disabled for the final configuration, because measured physical contact, cylinder headroom and cut-height error proved more reliable signals of the same condition, and three overlapping motion penalties (magnitude, rate and acceleration), none of which distinguishes following the path from tilting the head, were reduced to two.

One apparent inconsistency is deliberate. The tool-acceleration term saturates at 300 m/s^2 while the corresponding gate sits at 120 m/s^2, which is anchored to the measured physical

Table 6.2: The ten reward terms of the final PPO–ROM task, with weights and shaping scales. B and H are the bounded shapes of Equation (6.2). The lateral and head terms each combine a coarse and a fine scale, so far-field correction and near-field precision are priced separately.

Term	Weight	Shape and purpose
Cut-height tracking	+6	H, scale 0.04 m, cap 2.5; the primary mowing-quality term
Lateral tracking	+12	B, coarse 1.50 m and fine 0.60 m mixed 0.6/0.4; the primary line-following term
Ground contact	+4	pays for the cutter being in contact
Head alignment	−12	B on the deficit below 0.98, fine scale 0.02 and coarse scale 1.00 mixed evenly
Contact loss	−3	charges directly for loss of contact
Action rate	−1	B on the mean absolute command change, scale 0.25; penalizes chatter
Tool acceleration	−1	B, scale 300 m/s^2; mechanical-wear proxy on the cutter
Contact force	−1	B on force above 30 kN, scale 150 kN
Cylinder headroom	−2	B on encroachment inside a 0.02 m margin, scale 0.10 m
Hard impact	−4	indicator on the impact event

floor of the stage-6 environment near 100 m/s^2. The reward scale is a saturation scale for a bounded penalty and not a target, so the gate remains the binding criterion. The reward's role is only to keep a gradient against gross excursions. The same caution applies to the action-rate term, which is held deliberately mild: cylinder 4 owns head pitch, so a rate tax falls hardest on the fast corrections that keep the head down as boom attitude shifts, and under a heavier tax the optimizer chose head drift over paying it.

Every term is logged individually and by name during training. That is a direct consequence of the failure above: the earlier stack logged an aggregate and a partial subset, with the two largest lateral terms not logged at all, which is part of why a reward term opposing the task survived thirteen campaigns.

Reward weights on this system are not preferences but measured trades, and several natural-looking choices are actively harmful. This is also the reason the reward is *not* the result criterion. Promotion uses the eight physical gates of Section 3.9, evaluated at the worst mesh and, for the tracking-critical gates, the worst path family. A policy may raise total reward while failing the thing the reward was written to encourage, as Section 6.3 documents.

6.2.5 Optimization

Policies are optimized with clipped-surrogate PPO over generalized advantage estimation, in the form of Section 2.7. The maintained configuration uses clip parameter $\epsilon = 0.2$, discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, five learning epochs over four minibatches per update, an adaptive learning-rate schedule targeting a KL divergence of 0.02 from a base

rate of 3×10^{-4}, gradient-norm clipping at 1.0, and a clipped value loss with coefficient 1.0. The terrain task collects 32 steps per environment per update. At 512 environments that is 16 384 transitions per update, chosen because the inherited 128-step horizon exceeded two minutes per update on the available GPU without adding useful batch diversity.

The final PPO–ROM fine-tuning blocks depart from that configuration, because they run under the reference anchor of Section 6.4 rather than under an adaptive learning-rate schedule. Read from the run manifest, they use clip parameter $\epsilon = 0.1$, discount $\gamma = 0.97$, GAE parameter $\lambda = 0.95$, three learning epochs, fixed learning rates of 10^{-5} for the actor and 10^{-4} for the critic, entropy coefficient 0.001, an exploration standard deviation fixed at 0.15 and excluded from the gradient, frozen observation normalizers, and 128 environments.

Three of those departures are structural rather than tuning. The discount is reduced from 0.99 because the episode is long in simulation steps but locally controlled: an action should be credited for correcting the terrain state it faces, not for a return hundreds of steps later that the critic cannot resolve at this horizon. The actor learning rate is an order of magnitude below the critic's so that the actor is deliberately the slow variable, and trust-region budget is not spent on steps driven by a value function that has not yet converged within the block. The adaptive schedule is dropped because the anchored formulation already constrains the update through an explicit divergence bound, and two competing step-size controllers would make neither attributable.

Two hyperparameters were changed specifically because the policy stopped being a residual. The initial policy standard deviation was raised from 0.05 to 0.30, and the entropy coefficient raised with it: a residual policy explores around a controller that already tracks, whereas here a zero action means the arm does not move at all, so the policy must first discover the command directions before it can refine them. On the terrain task, where distillation supplies a competent starting point, both are then reduced again to an entropy coefficient of 0.001 and an exploration standard deviation of 0.15 that is *held fixed* rather than initialized, because exploration around an already-competent actor is a cost rather than a benefit. Fixing the standard deviation has a second purpose specific to the anchored formulation, given in Section 6.4: with a free standard deviation, entropy collapse shrinks the measured divergence from the reference while the commanded action moves, so the constraint would report compliance during the drift it exists to prevent. Exact learning rate, entropy coefficient, initial standard deviation, environment count and iteration budget for any reported run must be read from that run's frozen manifest, because these values changed during development.

6.3 Training Protocol and the Distillation Finding

6.3.1 Distillation by DAgger

Policies are initialized by imitation, not by reward. The privileged teacher of Section 6.1 is rolled out in the student's own environment, and the student is fitted to the teacher's raw action by regression, in the DAgger scheme of Ross et al. [Ross 11]: the teacher's probability of *executing* the action, as opposed to merely labelling it, is annealed from one to zero over the first 70 % of iterations, so the later part of training collects states the student itself visits rather than states the teacher would have visited. Labels are

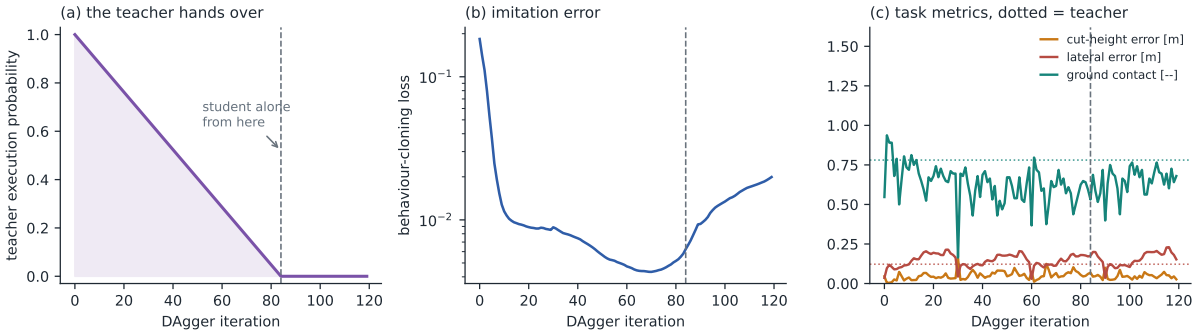

Figure 6.5: A measured distillation run for the PPO–ROM stack, from the stored record. (a) The teacher's execution probability is annealed to zero over the first 70 % of iterations. (b) Behaviour-cloning loss falls by over an order of magnitude under teacher execution, then rises once the student is alone, the distribution shift that DAgger is designed to surface, not a regression. (c) Closed-loop task metrics during distillation. Dotted lines are the teacher's own measured values from Table 6.1.

accumulated in a replay buffer of 262 144 transitions and refitted over three epochs per iteration in minibatches of 8 192.

Figure 6.5 shows a complete distillation run for the ROM stack, taken from the stored record. Panel (a) is the handover schedule. Panel (b) is the imitation error: the behaviour-cloning loss falls by more than an order of magnitude while the teacher is still driving, reaches its minimum around the handover, and then *rises* once the student is alone. That rise is not a failure. It is the distribution shift DAgger exists to expose: the student is now visiting states the teacher never chose, and the teacher's advice on those states is new information rather than a repetition of what has already been fitted. Panel (c) confirms that the closed-loop task metrics stay in the neighbourhood of the teacher's own, without matching them.

6.3.2 Objective Mismatch in Unconstrained Fine-Tuning

The natural next step is to treat the distilled actor as an initialization and improve it with PPO. That is what the residual literature does, it is what the plan for this thesis originally assumed, and in unconstrained form, on this task, it does not work. The finding is reported here in full because it is what motivates the anchored formulation of Section 6.4, which is how the final policies are trained.

PPO fine-tuning was run repeatedly from distilled checkpoints, and the outcome was consistent enough to be treated as a property of the problem rather than as bad luck. The critic was healthy throughout, with explained variance in the 0.85–0.95 band, so the failure is not a value-estimation failure. Total reward rose. And the primary tracking objective degraded, in the worst case by a factor of 12.9.

The mechanism follows from placing the reward and the gates side by side. The reward pays for ground contact and cut height as well as lateral tracking, and the optimizer discovers that it can buy the first two by spending the third. That trade increases total reward and moves the policy away from the gate that decides acceptance. The reward's optimum is simply not where the tracking gate is. Four earlier fine-tuning campaigns on a

related configuration failed the same way in a different currency: every block degraded head alignment, with p01 falling from 0.9854 to the 0.48–0.78 range while safety terminations rose from 1.3 % to 3.6–7.7 %.

This is a claim about two functionals over the same policy space rather than about the optimizer. Training maximizes the expected return $J(\vartheta)$, while acceptance is decided by the vector of physical gates $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$ of Section 3.9. Writing $\mathcal{F} = \{\vartheta : \boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star\}$ for the feasible set, the measured content of the finding is

$$\arg\max_{\vartheta} J(\vartheta) \notin \mathcal{F}, \qquad \text{while} \qquad \vartheta_{\text{distilled}} \in \mathcal{F}. \tag{6.3}$$

Distillation lands inside $\mathcal{F}$ because the teacher was constructed to satisfy the gates, not to maximize J. Unconstrained ascent on J from that point therefore leaves $\mathcal{F}$ by construction. This is the optimizer working correctly on the wrong functional, and it is why the remedy is a constraint on the ascent rather than a repair of the reward.

Three consequences follow, and all three are protocol rather than interpretation.

Admissibility of Fine-Tuning

For several development cycles the reported policy was the distilled actor, because no bounded form of fine-tuning was available. That restriction is lifted by Section 6.4 and no longer describes the maintained pipelines: the selected PPO–ROM policy is a fine-tuned actor. Unconstrained fine-tuning remains inadmissible, and both the pre- and post-fine-tune checkpoints are reported in either case.

Predeclared and Matched Protocol

Either all three stacks are distilled with identical teacher data and budget and then evaluated, or all three are distilled identically and given the *same* bounded fine-tuning budget, with both checkpoints reported. What is not permitted is comparing a distilled baseline against a separately optimized adaptive policy and attributing the difference to the AAM.

Training Instrumentation

Reward decomposition, critic explained variance, policy entropy and standard deviation, and the gate metrics themselves are logged during training, not only at the end. The whole point is that total reward went *up* while the system got worse. A training curve that reports only the scalar reward would have shown a success.

The general form of this result applies beyond this machine. When a competent teacher exists and the acceptance criterion is not the reward, reward-based fine-tuning is not automatically an improvement step. PPO optimizes the written objective, and that objective is a proxy. The teacher, by contrast, was constructed to do the task. Distillation transfers the intent. PPO then optimizes the proxy, and on this problem the proxy and the intent point in measurably different directions.

The constructive form of that lesson is the subject of the next section. Since the divergence between proxy and intent grows with distance travelled from a policy where the two were measured to agree, the remedy is not to abandon reward optimization but to bound that distance explicitly.

6.4 Reference-Anchored PPO

Equation (6.3) says that reward ascent leaves the feasible set. It does not say that reward ascent is useless, only that it is unbounded. The construction of this section keeps PPO and constrains how far it may travel from a policy already known to satisfy the gates. Writing π_{ref} for a frozen reference policy and J for the expected return, each training block solves

$$\max_{\vartheta} J(\vartheta) \quad \text{s.t.} \quad \mathbb{E}_{\boldsymbol{o}\sim d_{\pi_\vartheta}}\Big[\mathrm{KL}\Big(\pi_{\text{ref}}(\cdot\,|\,\boldsymbol{o})\,\|\,\pi_\vartheta(\cdot\,|\,\boldsymbol{o})\Big)\Big] \le \delta_{\text{ref}}, \tag{6.4}$$

with the expectation taken over the states the policy itself visits.

6.4.1 Critic Warm-Up

A resumed checkpoint carries an actor ϑ and a critic ψ. After distillation the actor already behaves sensibly, but the critic has never been fitted to the long terrain return, so the first PPO updates would move a useful actor using a value estimate that is not yet meaningful. The actor is therefore frozen for 150 iterations while only the value loss is optimized, under the asserted invariant $\vartheta_{\text{after}} = \vartheta_{\text{before}}$. The observation normalizers are frozen at the same point: without that, the same raw observation is normalized differently from one iteration to the next, and the actor's behaviour changes while its weights do not.

6.4.2 The Reference Anchor

Let π_{ref} be a frozen copy of the accepted warm actor. It is queried only on states the live policy visits and never supplies an executed action, so PPO remains the sole controller of the plant. The penalty is the *forward* divergence, which for diagonal Gaussians is available in closed form,

$$D_{\text{ref}} = \mathrm{KL}\Big(\pi_{\text{ref}}\,\|\,\pi_\vartheta\Big) = \sum_i \left[\log\frac{\sigma_{\vartheta,i}}{\sigma_{\text{ref},i}} + \frac{\sigma_{\text{ref},i}^2 + (\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2\sigma_{\vartheta,i}^2} - \frac{1}{2}\right], \tag{6.5}$$

at the cost of one additional forward pass and no sampling. Because both scales are fixed at $\sigma = 0.15$, this collapses to a pure quadratic on the drift of the commanded action,

$$D_{\text{ref}} = \sum_i \frac{(\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2(0.15)^2}, \tag{6.6}$$

which is the second reason for fixing σ: with a free standard deviation, entropy collapse shrinks D_{ref} while the commanded action moves, and the constraint would report compliance during drift.

The direction is load-bearing. Forward KL is mass-covering: it charges π_ϑ wherever π_{ref} has density and π_ϑ does not, and therefore penalizes the *abandonment* of reference behaviour. The reverse direction is mode-seeking and would permit the policy to collapse onto a subset of reference behaviour at no cost, which is the observed failure, in which one competence was dropped to buy another. It follows that the term is a behaviour-cloning anchor evaluated on the policy's own state distribution, rather than the reverse-KL regularizer of soft reinforcement learning. Its lineage is demonstration-regularized policy gradient [Rajeswaran 18], not the Boltzmann-optimum family.

The actor objective is

$$\mathcal{L}(\vartheta) = \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\,\mathcal{H}(\pi_\vartheta) + \lambda_{\text{ref}}\,\mathbb{E}_k\Big[\mathrm{KL}(\pi_{\text{ref}}\|\pi_\vartheta)\Big], \qquad \lambda_{\text{ref}} = 0.10. \tag{6.7}$$

6.4.3 Trust Regions and Rollback

A penalty is not a constraint: at a fixed coefficient, a sufficiently large advantage always buys its way past. Feasibility is therefore also verified on the update taken. Two bounds are imposed,

$$D_{\text{update}} = \mathrm{KL}(\pi_{\text{old}} \| \pi_\vartheta) \leq 0.01, \qquad D_{\text{ref}} \leq 0.05, \tag{6.8}$$

and they do different work. D_{update} is the classical step-size control that keeps the surrogate a valid local model of J. D_{ref} is the new object: because the anchor is *fixed* within a block rather than moving with the policy, it bounds cumulative drift. A sequence of updates each individually satisfying D_{update} can still walk arbitrarily far from feasibility, and that is the trajectory the earlier campaigns took, monotonically, with the reward rising the whole way. Only a constraint referred to a fixed feasible point can observe it.

Enforcement is in two stages. During minibatch optimization the epoch is early-stopped as soon as D_{update} exceeds its bound. After the update both divergences are recomputed over the full rollout. If either bound fails, or any quantity is non-finite, the actor and its optimizer state are restored *atomically*. On a divergence rejection the critic fit is retained when finite, since the bounds constrain the policy and not the value function, and rewinding valid critic fits computed on the same on-policy rollout was wasting most batches near the boundary. On a non-finite update both networks are restored. In the final block, five of one hundred logged updates were rejected, all of them on the reference bound and none on the update bound. That is, the drift caught was of the kind that ordinary PPO clipping does not see.

6.4.4 The Block Ratchet

The reference is not fixed forever. Within a block it is frozen. When a checkpoint is scored and passes the gates, it is promoted to reference,

$$\vartheta_{\text{ref}}^{(m+1)} = \vartheta^\star, \qquad \vartheta^\star \in \mathcal{F}, \quad \vartheta^\star \in B_{\delta_{\text{ref}}}\left(\vartheta_{\text{ref}}^{(m)}\right), \tag{6.9}$$

and the actor's optimizer moments are reset, because momentum accumulated relative to the old anchor is not meaningful relative to the new one. The critic's are retained: it is fitting the same return, and its calibration is expensive to rebuild.

This is the structural point of the construction. The policy is at all times within a bounded divergence of a checkpoint that has been verified against $\boldsymbol{G}$, and the anchor advances only through verified checkpoints. The acceptance criterion therefore enters the training loop instead of appearing only at the end. The optimizer still ascends the proxy. It simply can never travel far from the last point at which proxy and intent were measured to agree. In this construction distillation is no longer the endpoint: it is $\vartheta_{\text{ref}}^{(0)}$.

Algorithm 6.1 states the construction in full. Three details in it are easy to overlook and each was a measured correction. The early stop at line 15 acts *within* the epoch, so a minibatch sequence cannot accumulate past the bound before the post-hoc check sees it. The rejection branch at line 23 restores the actor but keeps a finite critic fit, because the bounds constrain the policy and not the value function. And the promotion at line 27 resets the actor's optimizer moments only: momentum accumulated against the old anchor is meaningless against the new one, whereas the critic is still fitting the same return.

Algorithm 6.1: Reference-anchored PPO for the PPO–ROM verge policy. The teacher appears only in layer 1 and the reference only as a divergence target: at every step of layer 3 the plant is driven by π_ϑ alone.

input : privileged teacher π_T; gates $\boldsymbol{G}$ with thresholds $\boldsymbol{g}^\star$; bounds $\delta_{\text{upd}} = 0.01$, $\delta_{\text{ref}} = 0.05$; anchor weight $\lambda_{\text{ref}} = 0.10$; fixed $\sigma = 0.15$; $\epsilon = 0.1$, $\gamma = 0.97$, $\lambda_{\text{GAE}} = 0.95$; $\eta_{\text{actor}} = 10^{-5}$, $\eta_{\text{critic}} = 10^{-4}$

output : accepted policy $\vartheta^\star$ with $\boldsymbol{G}(\vartheta^\star) \preceq \boldsymbol{g}^\star$

`// Layer 1: a physically feasible student`

$\vartheta \leftarrow \arg\min_\vartheta \mathbb{E}[\|\mu_\vartheta(\boldsymbol{o}) - \boldsymbol{y}^T\|_2^2]$ on DAgger states `// on the mean, not a sample`

discard π_T `// never in the loop again`

`// Layer 2: critic warm-up, actor frozen`

freeze ϑ and the observation normalizers

for 150 *iterations* **do**

  $\psi \leftarrow \psi - \eta_{\text{critic}} \nabla_\psi \mathcal{L}_V(\psi)$

end

assert $\vartheta_{\text{after}} = \vartheta_{\text{before}}$

$\vartheta_{\text{ref}} \leftarrow \vartheta$ `// block-0 anchor`

`// Layer 3: anchored PPO in blocks`

for *block* $m \leftarrow 0, 1, 2, \ldots$ **do**

  for *each iteration of block* m **do**

    collect an on-policy rollout with π_ϑ as sole controller

    compute GAE advantages from $\gamma, \lambda_{\text{GAE}}$; normalize

    snapshot $(\vartheta, \psi, \text{optimizer})$

    for 3 *epochs, each over the minibatches* **do**

      if $\text{KL}(\pi_{\text{old}} \| \pi_\vartheta) > \delta_{\text{upd}}$ **then break**

      $\mathcal{L} \leftarrow \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\mathcal{H} + \lambda_{\text{ref}} \text{KL}(\pi_{\text{ref}} \| \pi_\vartheta)$

      step Adam on $\mathcal{L}$ with $\eta_{\text{actor}}, \eta_{\text{critic}}$

    end

    $D_{\text{upd}} \leftarrow \text{KL}(\pi_{\text{old}} \| \pi_\vartheta)$, $D_{\text{ref}} \leftarrow \text{KL}(\pi_{\text{ref}} \| \pi_\vartheta)$ `// over the whole rollout`

    if D_{upd} *or* D_{ref} *non-finite* **then**

      restore ϑ, ψ and the optimizer

    else if $D_{\text{upd}} > \delta_{\text{upd}}$ **or** $D_{\text{ref}} > \delta_{\text{ref}}$ **then**

      restore ϑ and the actor optimizer group; **retain** the critic fit `// the bounds constrain the policy, not` V

  end

  score the block's checkpoints on ≥ 3 held-out terrain meshes; aggregate $\boldsymbol{G}$ to the worst mesh, and to the worst path family for the tracking gates

  if *some candidate* $\vartheta^\star$ *has* $\boldsymbol{G}(\vartheta^\star) \preceq \boldsymbol{g}^\star$ **then**

    $\vartheta_{\text{ref}} \leftarrow \vartheta^\star$; reset the *actor* optimizer moments, retain the critic's

  else

    keep ϑ_{ref}; the block is not promoted

  end

end

6.4.5 Theoretical Status

The components are standard and citable. The bound on D_{update} is conservative policy iteration and its trust-region descendants [Kakade 02, Schulman 15]; the atomic rollback is that literature's backtracking line search reduced to a single trial point; the penalty in Equation (6.7) is a Lagrangian relaxation of the constrained problem (6.4) evaluated at a fixed multiplier, with the rollback supplying the feasibility enforcement a fixed multiplier cannot.

What does not follow is a certificate. Bounding the divergence from a verified policy does bound gate degradation in form: the chain rule of KL over trajectories, Pinsker's inequality, and the boundedness of every reward and gate metric together give

$$\left|\mathbb{E}_{\pi_\vartheta}[g] - \mathbb{E}_{\pi_{\text{ref}}}[g]\right| \leq 2G_{\max}\sqrt{H\delta_{\text{ref}}/2} \tag{6.10}$$

for a gate metric g with $\|g\|_\infty \leq G_{\max}$ over a horizon H. This bound is available *only* because the reward of Table 6.2 is bounded term by term, and one that an exponential term would destroy. At the production horizon of 2808 steps and $\delta_{\text{ref}} = 0.05$, however, the right-hand side exceeds eight, which for a metric bounded in $[0, 1]$ is vacuous. There is also a measure mismatch, in that the constraint is taken under the live policy's state distribution while the chain rule requires the reference's.

The honest claim is therefore the following. The construction is not a certificate. It is a principled restriction of reward ascent to a neighbourhood of a policy where proxy and acceptance criterion were measured to agree, assembled from standard trust-region and demonstration-regularized results, with feasibility re-established empirically at every block boundary. The gates, not the theory, certify each promotion.

6.5 PPO–ROM: Certified Actuator Baseline

PPO–ROM is the reference condition of this thesis. The policy emits desired cylinder motion, the frozen command ROM converts it to a cylinder position target, and PhysX closes the loop. There is no online adaptation anywhere in the stack. The truck is scripted at 2 km/h over a 26 m strip, the eleven path families of Section 3.6 are distributed across environments within a single run, and the terrain mesh is drawn per environment so that no policy can memorize one ground surface.

Its status is that of a reference condition rather than a competitor, as developed in Chapter 7: the command ROM is not a pure Simscape artefact but a Simscape-trained model with a bounded closed-loop calibration against IsaacLab and PhysX, and ROM-plus-PhysX *is* the deployment plant. PPO–ROM's actuator therefore coincides with the plant. Its tracking advantage over PPO–ID–FD is consequently not evidence that a reduced order model approximates the actuator better than an identified ID/FD pair: one of the two defines the target.

The policy is produced by the three-layer construction of this chapter: distillation from the privileged DLS teacher to a feasible student, critic warm-up with the actor frozen, and reference-anchored PPO in blocks under Equations (6.7) and (6.8). The final block produced checkpoints 280 to 329. The last iterate was not taken, because PPO can raise mean return while damaging a single path family. Candidates were scored instead

on held-out meshes against the eight gates, aggregated to the worst mesh and, for the tracking-critical gates, the worst path family.

[**TODO: Insert the worst-mesh worst-family gate table for the selected checkpoint once the** 2808**-step six-mesh revalidation has been run. The numbers currently on record were measured on the superseded** 16 m **rollout and the checkpoint's own selection record marks it provisional for that reason.**]

6.6 PPO–ID–FD: Non-Adaptive Learned Baseline

[**TODO: write**]

6.7 PPO–ID(AAM)–FD: Proposed Adaptive Method

[**TODO: write**]

6.8 IsaacLab Integration and Timing

[**TODO: write**]

6.9 Stability, Constraints and Fair-Comparison Rules

[**TODO: write**]

7. Experimental Evaluation and Discussion

7.1 Hypotheses and Fairness Controls

[TODO: write]

7.2 Model-Level Experiments

[TODO: write]

7.3 System Response

[TODO: write]

7.4 Delay

[TODO: write]

7.5 Nonlinearity

[TODO: write]

7.6 End-Effector and Mowing Performance

[TODO: write]

7.7 Robustness and Generalization

[TODO: write]

7.8 Adaptation Ablations

[TODO: write]

7.9 Statistics and Reporting

[TODO: write]

7.10 Integrated Discussion

[TODO: write]

7.11 Threats to Validity

[TODO: write]

8. Conclusion and Outlook

8.1 Summary of the Approach

[TODO: write]

8.2 Answers to the Research Questions

[TODO: write]

8.3 Main Contributions

[TODO: write]

8.4 Limitations

[TODO: write]

8.5 Future Work

[TODO: write]

A. System Parameters and Coordinate Mappings

[TODO: write]

B. Dataset Manifests and Factor Ranges

B.1 Trajectories

Each run receives its own reference trajectory, sampled rather than scripted. Between twelve and twenty waypoints are drawn by Latin hypercube sampling within the joint limits, and the arm moves between them without dwelling, so that the data contains transitions rather than long static holds. Segment durations are drawn from four speed classes, slow (1.5–2.5 s), medium (0.8–1.5 s), fast (0.3–0.7 s), and variable (0.5–2.5 s), and about a third of the runs receive additional exploratory perturbations on top of the waypoint path.

An earlier version of this generator used four to eight waypoints with dwell periods between them. It produced data in which the arm was stationary for a large fraction of the time, which flatters any model that learns to predict persistence. The current settings were chosen to remove that easy mass of near-static samples, and the increase in waypoint count is the single change that most improved the usefulness of the batch.

B.2 Execution and Logging

Each run simulates 22 s of operation. The solver is the stiff implicit `ode23t` integrator in accelerator mode with relative and absolute tolerances of 10^{-2} and 10^{-3}. The loose tolerances are a deliberate throughput compromise, and their effect is bounded by the consistency checks of Section 4.10. Output is written on a fixed 0.05 s grid, giving 441 logged samples per run, and the batch is checkpointed to disk every hundred runs so that a failure late in a multi-day batch does not cost the whole collection.

For every run the archive stores the time vector, the five valve commands, the four cylinder extensions, the four chamber pressure differences, the four rod forces, the five joint positions and velocities, the reference position and velocity signals, and, crucially, the complete parameter structure that produced the run. That last item is what makes the batch usable for the latent-adaptation work: the privileged plant descriptor $\boldsymbol{\xi}$ of Section 2.6 is not something that has to be reconstructed later, it is recorded alongside the trajectory.

The cost quantifies the argument made in Section 2.1.3. A single 64 s open-loop run takes roughly 2000 s of wall time on the collection machine, a real-time factor of about 0.03. The Simscape plant runs some thirty times slower than the machine it models. No policy-optimisation loop that needs millions of environment steps can be built on that, which is why the learned actuator surrogates exist.

B.3 Parameter Ranges and Dataset Audit

Table B.1: Randomized hydraulic parameters. The seal-wear coordinate is drawn independently for each of the four cylinders and drives three physically linked quantities at once.

Parameter	Nominal	Randomized range	Acts on
Entrained air fraction ε_{air}	0.005	0.001 – 0.080	fluid compliance
Dead-volume fraction ε_{vol}	0.020	0.005 – 0.080	fluid compliance
Discharge coefficient c_d	0.70	0.56 – 0.84	valve flow gain
Seal wear $w_{\text{seal},i}$	0	0 – 0.25 (per cylinder)	clearance, leakage, spool lag
Relief setting p_{set}	27.6 MPa	24.8 MPa – 30.4 MPa	pressure ceiling

Table B.2: Randomized mechanical parameters. The rigid-body perturbations are applied independently to each of the fourteen bodies.

Parameter	Nominal	Randomized range
Body mass	CAD value	±20 %
Centre-of-mass offset	CAD value	±5 %
Inertia tensor and products	CAD value	consistent with mass draw
Joint stiffness, J1 and J2	60 kN/m	40 – 120 kN/m
Joint stiffness, J3 and J4	60 kN/m	40 – 120 kN/m
Joint stiffness, J5	25 kN/m	15 – 60 kN/m
Joint damping	250	100 – 600

Table B.3: The six randomized physical factors of the IsaacLab adaptation collection and their final ranges. One factor vector is drawn per run and held fixed for the whole run.

Coordinate	Range	Domain	Acts on
e_{flow}	$[-0.25, 0.25]$	hydraulic	valve flow gain
$e_{\text{compliance}}$	$[-0.25, 0.10]$	hydraulic	effective chamber compliance
e_{seal}	$[0, 1]$	hydraulic	seal wear, leakage, spool lag
e_{pressure}	$[-0.15, 0.15]$	hydraulic	supply/relief pressure level
e_{mass}	$[0.8, 1.5]$	mechanical	cutter mass scale
e_{COM_z}	$[-0.10, 0.05]$	mechanical	cutter centre-of-mass height

Table B.4: Audit of the strict open-loop excitation export. All checks are computed by the extraction stage and stored with the data; the export is accepted only if every one of them passes.

Check	Result	Interpretation
Rows, runs, columns	25 600 / 4 / 49	four complete runs
Median sample time	0.01 s	as commanded
Maximum sample-time error	9.1×10^{-15} s	numerically exact grid
Maximum Δextension error	0	targets match state differences
Maximum Δvelocity error	0	targets match state differences
Maximum $\Delta(\Delta p)$ error	0 Pa	targets match state differences
Non-finite values	0	no solver escapes
Valid fraction	0.937	startup, gaps, shutdown excluded
Command rank	4	all four channels independent
Lagged excitation rank	80	history matrix full rank
Valve saturation	8.5 % per channel	limits reached but not dominant
Stroke coverage	12 % – 33 %	above floor, below target

C. Model Architectures, Normalizers and Checkpoint Hashes

[TODO: write]

D. PPO and IsaacLab Configuration

[TODO: write]

E. Supplementary Results

[TODO: write]

Bibliography

[Åström 95] K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Reading, MA: Addison-Wesley, 1995.

[Ba 16] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization", arXiv preprint arXiv:1607.06450, 2016.

[Bengio 15] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. 2015, pp. 1171–1179.

[Benner 15] P. Benner, S. Gugercin, K. Willcox, "A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems", *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.

[Cho 14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[Chua 18] K. Chua, R. Calandra, R. McAllister, S. Levine, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31. 2018.

[Egli 20] P. Egli, M. Hutter, "Towards RL-Based Hydraulic Excavator Automation", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2692–2697.

[Egli 22] P. Egli, M. Hutter, "A General Approach for the Automation of Hydraulic Excavator Arms Using Reinforcement Learning", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5679–5686, 2022.

[Elfwing 18] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", *Neural Networks*, vol. 107, pp. 3–11, 2018.

[He 16] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[Hochreiter 97] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[Huber 64] P. J. Huber, "Robust Estimation of a Location Parameter", *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[Hwangbo 19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[Isermann 11] R. Isermann, M. Münchhof, *Identification of Dynamic Systems: An Introduction with Applications*, Berlin, Heidelberg: Springer, 2011.

[Jelali 03] M. Jelali, A. Kroll, *Hydraulic Servo-systems: Modelling, Identification and Control*, London: Springer, 2003.

[Johannink 19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, S. Levine, "Residual Reinforcement Learning for Robot Control", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.

[Kakade 02] S. Kakade, J. Langford, "Approximately Optimal Approximate Reinforcement Learning", in *Proceedings of the 19th International Conference on Machine Learning (ICML)*. 2002, pp. 267–274.

[Karniadakis 21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, "Physics-Informed Machine Learning", *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[Kingma 15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", in *International Conference on Learning Representations (ICLR)*. 2015.

[Kumar 21] A. Kumar, Z. Fu, D. Pathak, J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots", in *Proceedings of Robotics: Science and Systems (RSS)*. 2021.

[Lee 20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning Quadrupedal Locomotion over Challenging Terrain", *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[Ljung 99] L. Ljung, *System Identification: Theory for the User*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 1999.

[Lutter 19] M. Lutter, C. Ritter, J. Peters, "Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning", in *International Conference on Learning Representations (ICLR)*. 2019.

[Maitan 26] L. Maitan, L. Toschi, C. Zanette, E. G. Vergamini, L. F. Santos, T. Boaventura, "Contraction Metric Based Safe Reinforcement Learning Force Control for a Hydraulic Actuator with Real-World Training", arXiv preprint arXiv:2602.08977, 2026.

[Makoviychuk 21] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning", arXiv preprint arXiv:2108.10470, 2021.

[Mattila 17] J. Mattila, J. Koivumäki, D. G. Caldwell, C. Semini, "A Survey on Control of Hydraulic Robotic Manipulators With Projection to Future Trends", *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 2, pp. 669–680, 2017.

[McKay 79] M. D. McKay, R. J. Beckman, W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code", *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[Merritt 67] H. E. Merritt, *Hydraulic Control Systems*, New York: John Wiley & Sons, 1967.

[Mittal 23] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments", *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[MULAG Fahrzeugwerk, Heinz Wössner GmbH u. Co. KG 26] MULAG Fahrzeugwerk, Heinz Wössner GmbH u Co KG, "FME 500 Front Mower — Product Documentation", Oppenau, Germany. `https://www.mulag.de/en/roadside-equipment/products/unimog-equipment/front-mower/fme` 2026. Accessed July 2026.

[Nagabandi 18] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 7559–7566.

[Nakamura 86] Y. Nakamura, H. Hanafusa, "Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control", *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[Nguyen-Tuong 11] D. Nguyen-Tuong, J. Peters, "Model Learning for Robot Control: A Survey", *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, 2011.

[Paszke 19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. 2019, pp. 8024–8035.

[Peng 18] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810.

[Rajeswaran 18] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, "Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations", in *Proceedings of Robotics: Science and Systems (RSS)*. 2018.

[Ross 11] S. Ross, G. J. Gordon, J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning", in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 627–635.

[Rudin 22] N. Rudin, D. Hoeller, P. Reist, M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", in *Proceedings of the Conference on Robot Learning (CoRL)*. 2022, pp. 91–100.

[Schoukens 19] J. Schoukens, L. Ljung, "Nonlinear System Identification: A User-Oriented Road Map", *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 28–99, 2019.

[Schulman 15] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, P. Abbeel, "Trust Region Policy Optimization", in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. 2015, pp. 1889–1897.

[Schulman 16] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation", in *International Conference on Learning Representations (ICLR)*. 2016.

[Schulman 17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv preprint arXiv:1707.06347, 2017.

[Siciliano 09] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, London: Springer, 2009.

[Silver 18] T. Silver, K. Allen, J. Tenenbaum, L. Kaelbling, "Residual Policy Learning", arXiv preprint arXiv:1812.06298, 2018.

[Slotine 91] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Englewood Cliffs, NJ: Prentice Hall, 1991.

[Sobol' 67] I. M. Sobol', "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals", *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[Spinelli 24] F. A. Spinelli, P. Egli, J. Nubert, F. Nan, T. Bleumer, P. Goegler, S. Brockes, F. Hofmann, M. Hutter, "Reinforcement Learning Control for Autonomous Hydraulic Material Handling Machines with Underactuated Tools", in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. arXiv:2410.05093.

[Srivastava 14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[Sutton 18] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.

[The MathWorks, Inc. 24] The MathWorks, Inc , "Simscape Fluids: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-fluids.html`.

[Tobin 17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[von Szadkowski 20] K. von Szadkowski, S. Reichel, "Phobos: A Tool for Creating Complex Robot Models", *Journal of Open Source Software*, vol. 5, no. 45, p. 1326, 2020.

[Wampler 86] C. W. Wampler, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods", *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[Yamamori 26] S. Yamamori, K. Ishihara, K. Minamikawa, R. Ohmori, T. Yasaki, N. Sugimoto, J. Morimoto, "Actuator Reality Shaping for Zero-Shot Sim-to-Real Robot Learning", arXiv preprint arXiv:2607.02205, 2026.

[Yang 26] N. Yang, C. Ji, J. Chen, H. Zheng, "State-Constrained Control for Hydraulic Manipulator Position Servo Systems with Valve Dead-Band Compensation", *Actuators*, vol. 15, no. 4, p. 196, 2026.

[Yin 25] P. Yin, T. Westenbroek, S. Bagaria, K. Huang, C.-A. Cheng, A. Kolobov, A. Gupta, "Rapidly Adapting Policies to the Real World via Simulation-Guided Fine-Tuning", arXiv preprint arXiv:2502.02705, 2025.

[Yu 26] S.-Y. Yu, J.-W. Wang, M. Horii, M. Tomizuka, V. Garg, "Learning Context-Aware Neural ODE Dynamics for Adaptive Robotic Control", arXiv preprint arXiv:2606.15469, 2026.